

MENTORIA 360

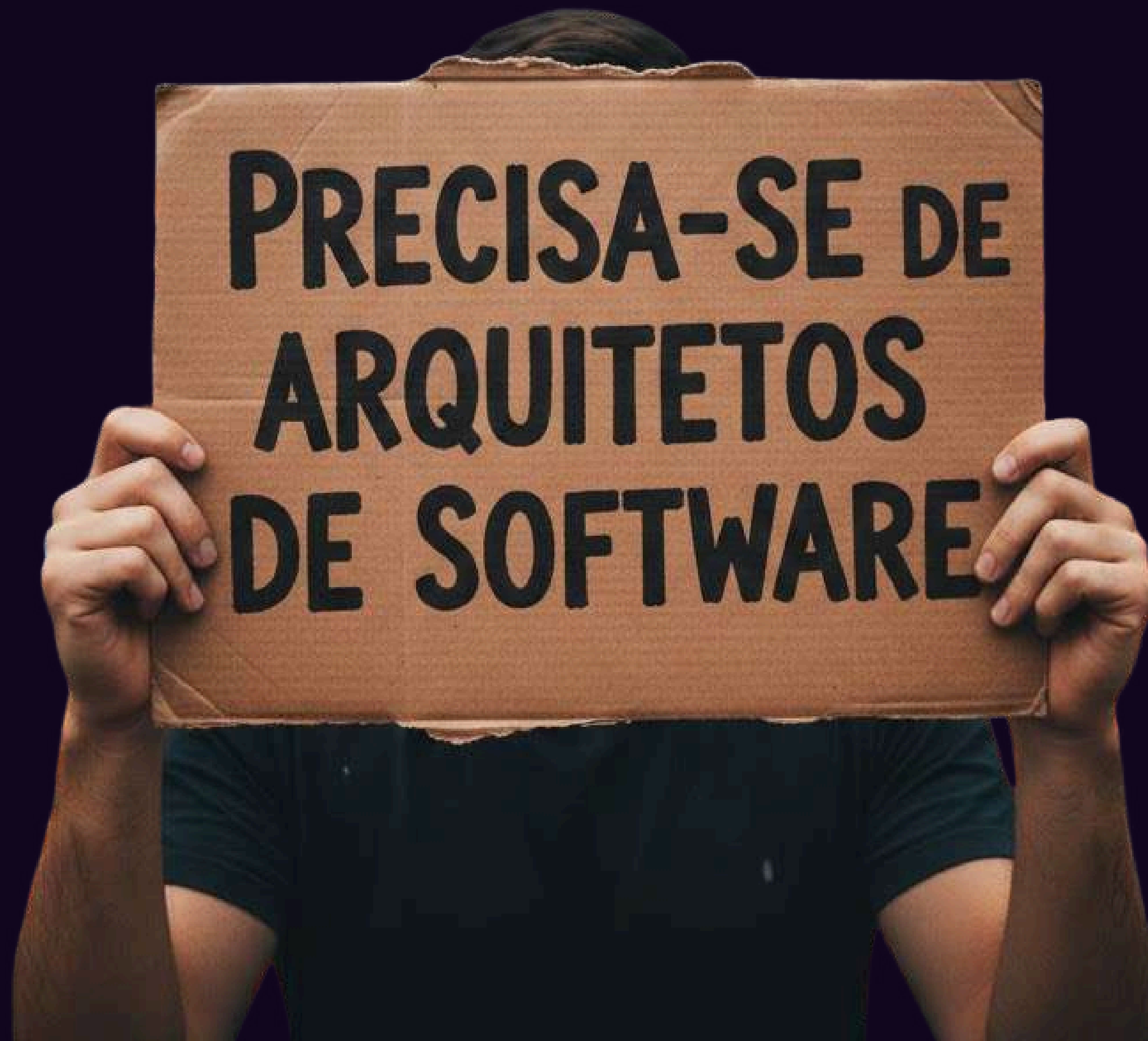
INFORMAÇÕES IMPORTANTES

INTRODUÇÃO

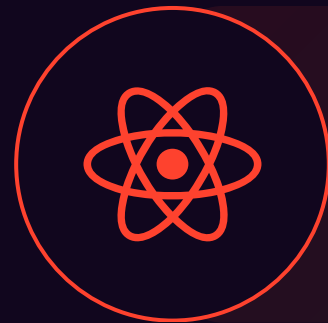


Nossos Objetivos

Formar arquitetos de software capazes de transformar regras de negócio em aplicações reais, modulares e alinhadas ao mercado, com clareza de visão, decisões técnicas conscientes e foco em manutenibilidade e escala.



A Transição de 2026



Profissional com profundo domínio de **uma tecnologia ou stack específica**, altamente eficiente na execução, mas com visão restrita do sistema como um todo.



Profissional com **visão integrada do desenvolvimento**, capaz de conectar negócio, arquitetura e implementação, e guiar decisões técnicas com clareza.



#RADARMARTHAGABRIEL

PROFISSÕES QUE MAIS VÃO CRESCER ATÉ 2030

X in @MARTHAGABRIEL

CRESCIMENTO LÍQUIDO

ESPECIALISTA DE BIG DATA	+110%
ENGENHEIRO DE TECNOLOGIAS FINANCEIRAS	+95%
ESPECIALISTA EM IA E MACHINE LEARNING	+85%
DEV. DE SOFTWARE E APPS	+60%
GESTÃO DE CIBERSEGURANÇA	+55%
ESP. ARMAZ. DADOS	+50%
ESP. VEÍC. AUTÔNOMOS	+45%
UI E UX DESIGNER	+45%
MOTORISTA CAMINHÃO	+45%
ESPECIALISTA IOT	+40%
CIENTISTA DE DADOS	+40%
ENG. AMBIENTAL	+40%

Fonte: Fórum Econômico Mundial / Visual Capitalist

Martha Gabriel,...

Best seller AUTHOR, KEYNOTE...

4d

+ Follow

X

As PROFISSÕES que mais vão CRESCER até 2030 já estão sendo moldadas pela inteligência artificial e pela automação.

Segundo o Future of Jobs Report 2025 do Fórum Econômico Mundial:

✓

Especialistas em BIG DATA lideram a lista, com crescimento previsto de 110%.

✓

Engenheiros de FINTECH (+95%) e Especialistas em IA e MACHINE LEARNING (+85%) vêm logo em seguida.

✓

Outras funções em destaque: desenvolvedores de SOFTWARE (+60%), especialistas em ...more

Show translation

1,473

76 comments · 109 reposts

Like

Comment

Repost

Send

Add a comment...

Most relevant

SERGIO LOPES -Top 1 Ambi...

Top 1% do Setor Ambiental pelo LinkedIn / ...

3d

...

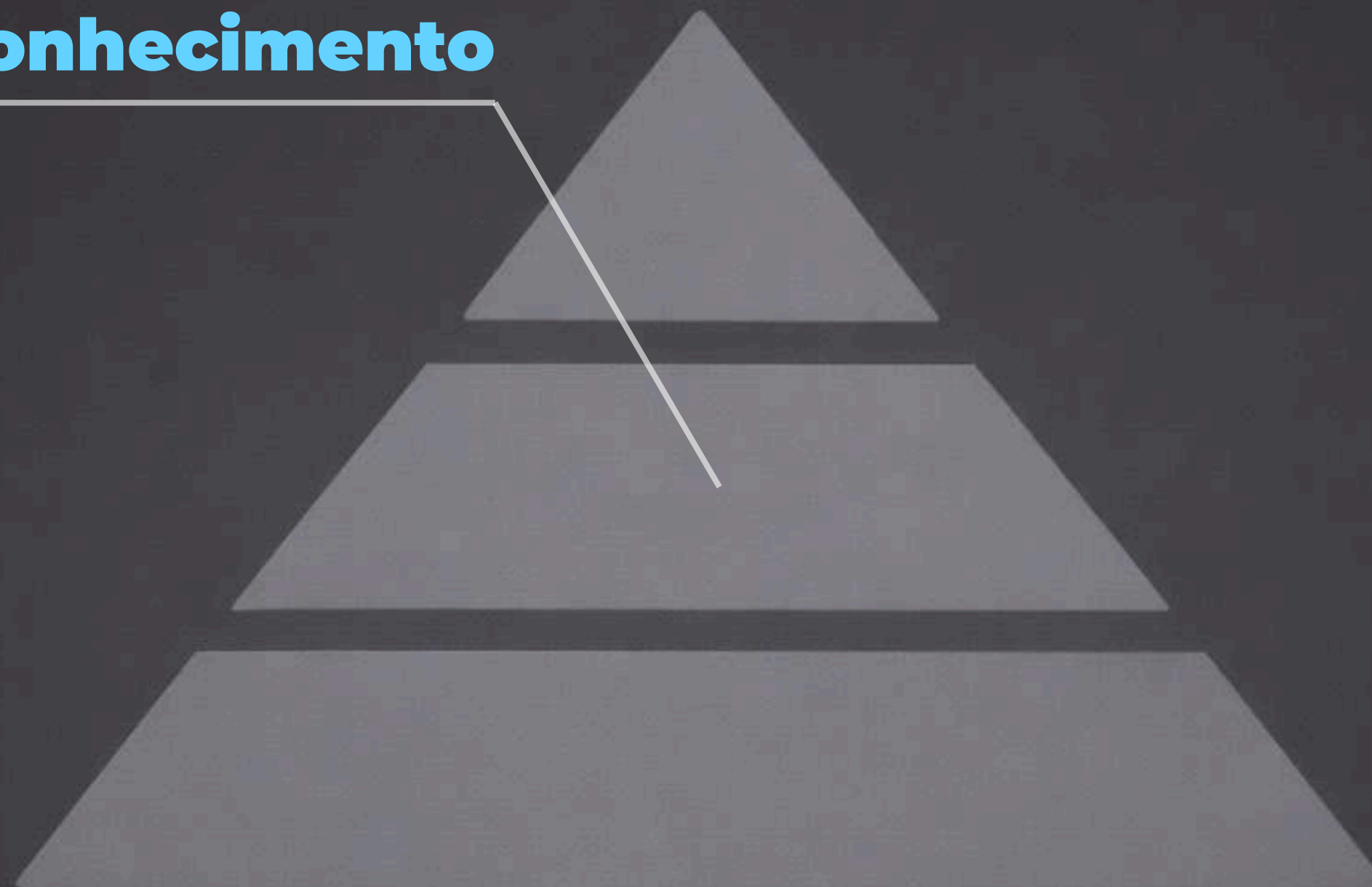
Esse gráfico é relevante, mas ele costuma ser lido de forma incompleta. A tecnologia vai crescer, sem dúvida. Dados, IA e automação são vetores claros. O ponto cego está em achar que esses vetores operam sozin ...more

Show translation

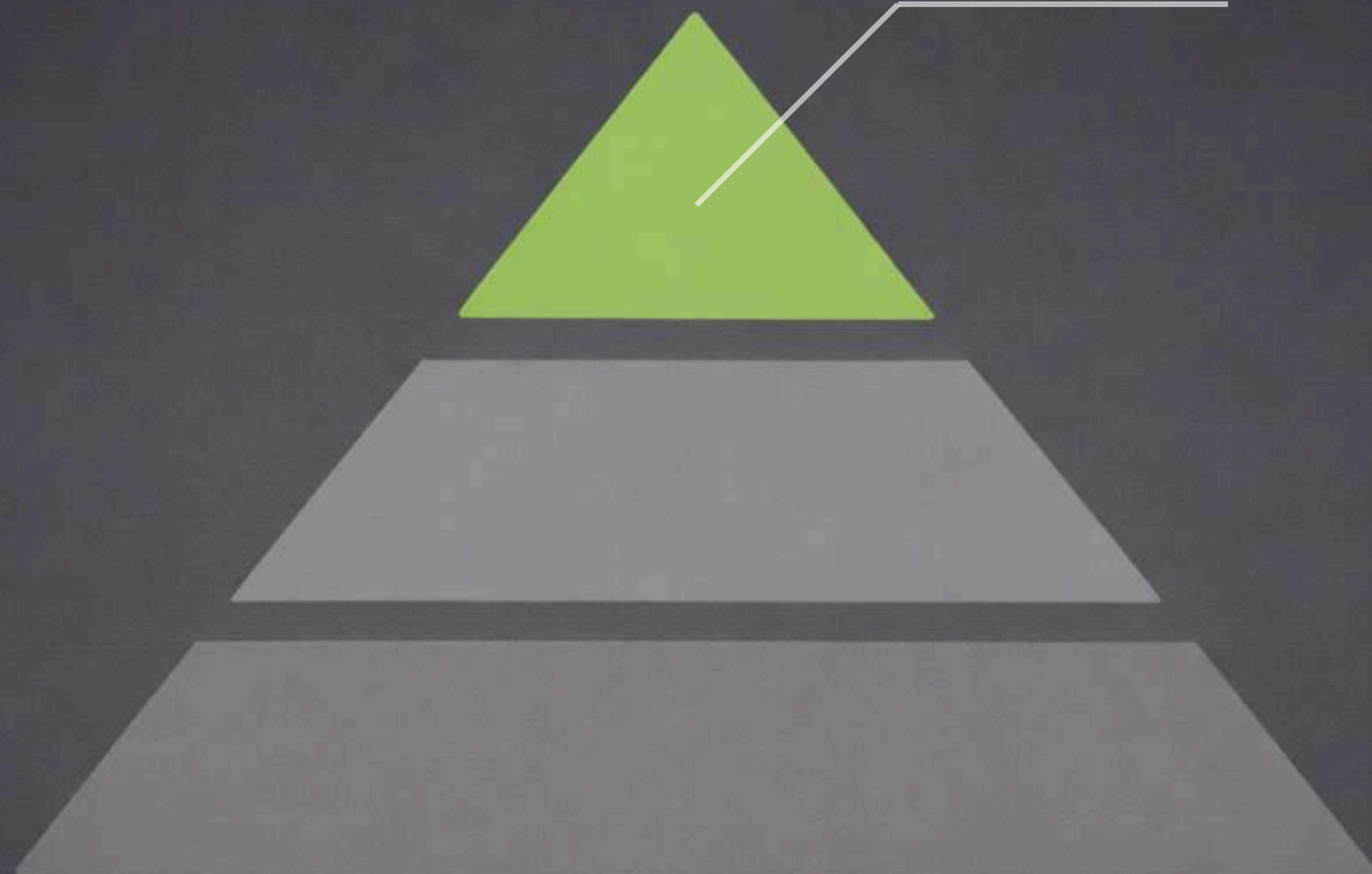
Like · 65

Reply · 7 replies

Pirâmide do **Conhecimento**

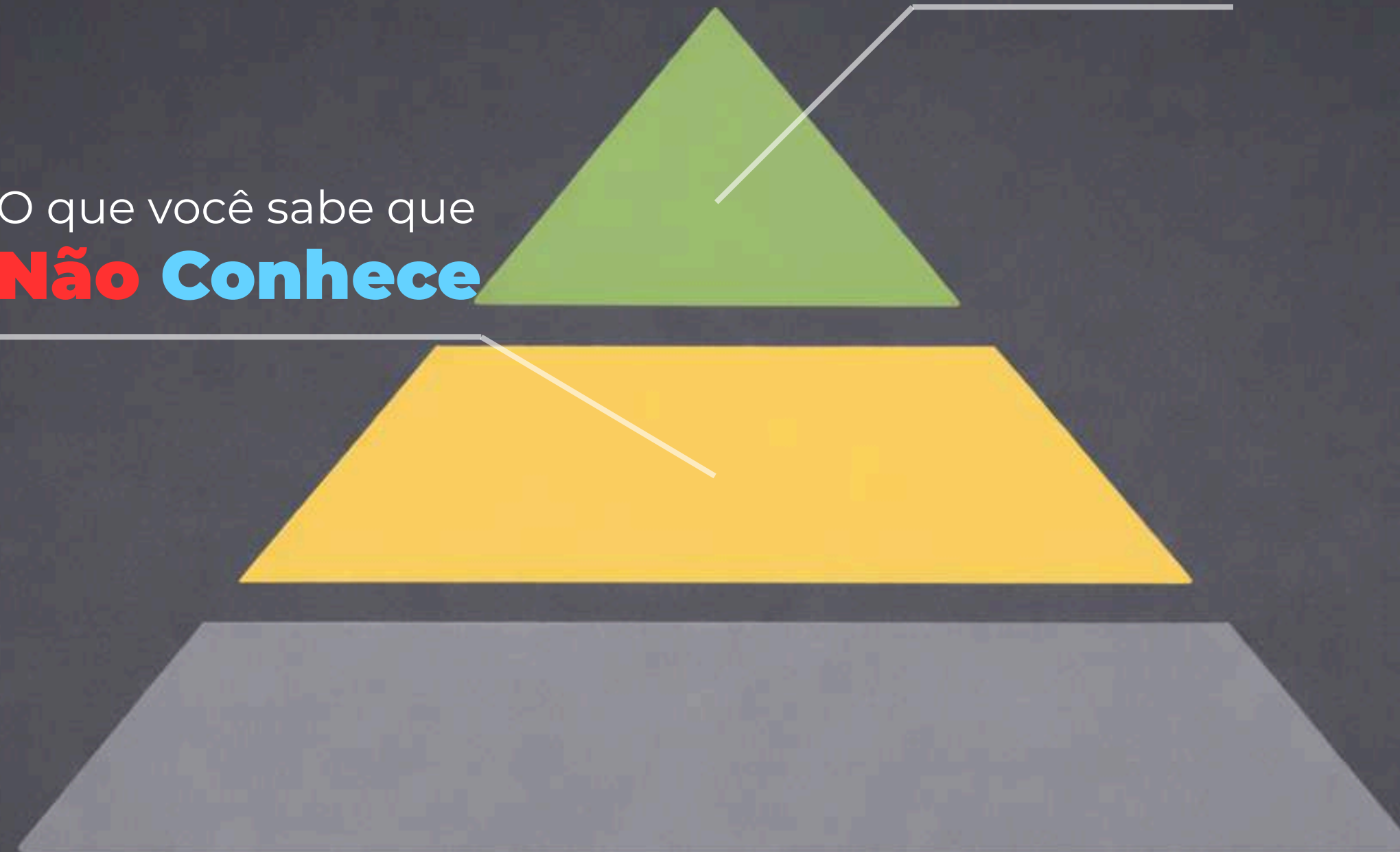


O que você
Conhece



O que você
Conhece

O que você sabe que
Não Conhece



O que você
Conhece

O que você sabe que
Não Conhece

O que você **NÃO** sabe que
Não Conhece



Noventa e cinco por cento das palavras
[sobre arquitetura de software] são usadas
enaltecendo os benefícios da
"modularidade" e poucas, se algumas,
tratam de como alcançá-la.

Glenford J. Myers

Foco em Apps

A Mentoria 360 tem como foco o desenvolvimento de aplicações reais, usando projetos como eixo central para ensinar, na prática, paradigmas de programação, arquitetura de software, modelagem de sistemas e o uso de ferramentas e frameworks alinhados ao mercado.



Exemplos de Aplicação



ERP



GESTÃO
FINANCEIRA



E-COMMERCE



EAD

Dinâmica das Mentorias



CONTEÚDO AO VIVO

Explorar conceitos, princípios e padrões aplicados diretamente em projetos e cenários reais de mercado.



PERGUNTAS E RESPOSTAS

Esclarecer dúvidas, discutir decisões técnicas e aprofundar os temas apresentados na mentoria.



MATERIAIS EXTRAS

Disponibilizar conteúdos de apoio para reforçar o aprendizado e facilitar a aplicação prática.

Últimos Tópicos

Aulas ficarão
gravadas

Calendário da
Plataforma

Sobre os
pré-requisitos

SEJA MUITO
BEM-VINDO



MENTORIA 360

PHARMA CORE

ERP



SLIDES FEITO COM



POR UMA MÃO SÓ

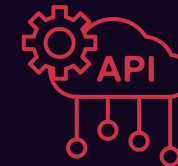


Nossos Objetivos

Propomos o desenvolvimento do PharmaCore, um ERP educacional criado para ensinar, na prática, paradigmas de programação, arquitetura de software e modelagem de sistemas reais.



PARADIGMAS EM APPS
DE MERCADO - POO & PF



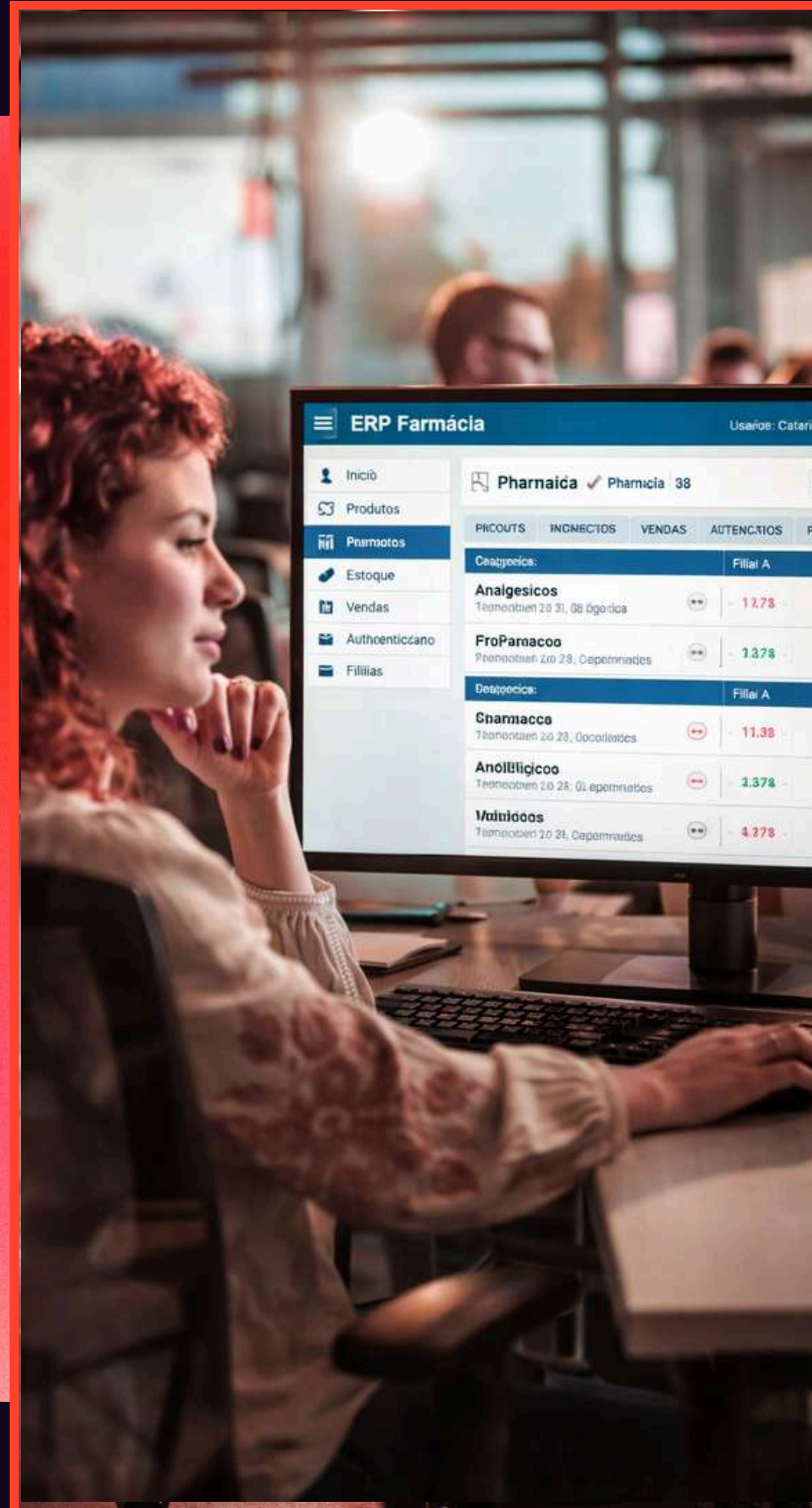
MODELAGEM DE DOMÍNIO



CASOS DE USOS & SAGAS



MODULARIZAÇÃO REAL
DA SOLUÇÃO



O que é ERP?

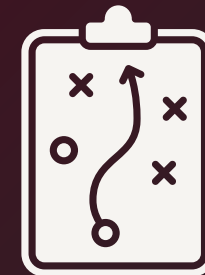
Um ERP (Enterprise Resource Planning) é um sistema de gestão empresarial que integra os principais processos de uma organização em uma única plataforma, centralizando dados, automatizando operações e garantindo consistência entre áreas como produtos, estoque, vendas e gestão operacional.

Roadmap do Desenvolvimento



ESTRATÉGICO

Compreender o domínio do negócio e organizar a aplicação em contextos delimitados e módulos bem definidos.



TÁTICO

Aplicar padrões de modelagem e arquitetura para transformar o domínio em soluções consistentes e sustentáveis.



OPERACIONAL

Implementar o sistema em código, integrando frameworks e bibliotecas de mercado sem comprometer as regras de negócio.

Roadmap Estratégico

Compreender o domínio do negócio e organizar a aplicação em contextos delimitados e módulos bem definidos.



Arquitetura começa
pelo domínio



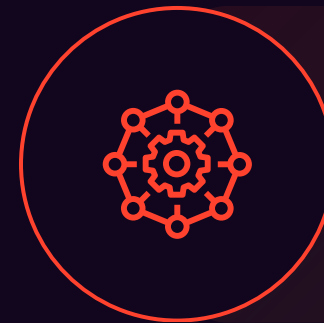
"O domínio de uma aplicação é o conjunto de conhecimentos, regras, conceitos e processos do negócio que o sistema precisa representar e sustentar."

—— O que é o domínio de uma aplicação?

Domínio de um **Negócio**



Representa o negócio real da empresa - não é o software



Define regras e limites do negócio



Pode ser dividido em subdomínios específicos



Orienta a estrutura e organização da aplicação

Compreender profundamente o domínio do negócio é o primeiro passo para construir aplicações bem estruturadas, onde arquitetura e código refletem as reais necessidades da empresa.

Subdomínios do **Negócio**

PHARMACORE



PRODUTOS



FILIAIS



ESTOQUE



VENDAS

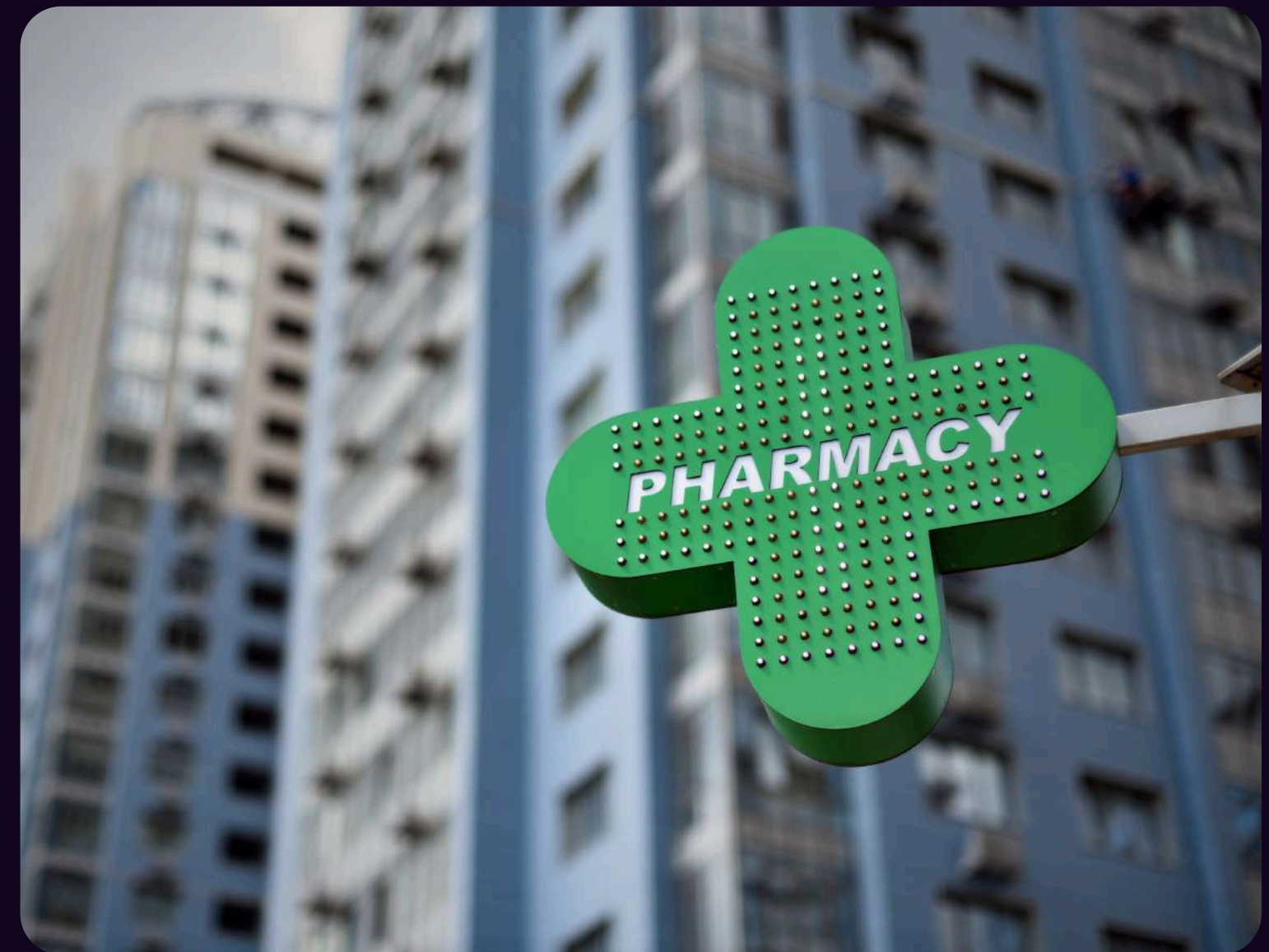
Subdomínio de **Produtos**

Responsável por organizar e gerenciar o catálogo de produtos da farmácia, incluindo medicamentos e itens de conveniência, estruturados em categorias e subcategorias para facilitar a busca, localização e identificação rápida dos produtos nas operações do dia a dia.



Subdomínio de **Filiais**

Responsável por gerenciar as filiais da farmácia, organizando informações essenciais de cada unidade, como identificação, localização e status operacional, servindo como base para o controle de estoque, vendas e demais operações realizadas em cada ponto de atendimento.



Subdomínio de **Estoque**

Responsável por controlar a quantidade de produtos disponível em cada filial da farmácia, acompanhando entradas, saídas e movimentações, garantindo visibilidade, organização e suporte às operações de venda e reposição.



Subdomínio de **Vendas**

Responsável por registrar e gerenciar as vendas realizadas na farmácia, organizando pedidos, itens vendidos e etapas do processo de venda, garantindo integração com estoque e suporte às operações comerciais do dia a dia.



Entender o negócio é o primeiro passo.
Agora, precisamos organizá-lo para virar **software**.

Ferramentas Estratégicas

Contexto
Delimitado

Mapa de
Contexto

Linguagem
Ubíqua

Contexto Delimitado

Segundo o Domain-Driven Design, um Contexto Delimitado é um limite explícito dentro do qual um modelo de domínio é definido, consistente e válido, incluindo seus conceitos, regras, linguagem e significados.



Big Ball of Mud



ERP



Não Existe Almoço **Gratís**

Modularizar de forma real traz benefícios claros, mas exige **custo**, **disciplina** e decisões conscientes ao abandonar um modelo único para toda a aplicação, a chamada Big Ball of Mud.



Contexto Delimitado



FILIAIS



PRODUTOS



ESTOQUE



VENDAS

ABRINDO PARÊNTESES...



Contexto Delimitado



FILIAIS



PRODUTOS



ESTOQUE



VENDAS

Contexto Delimitado

Não estou falando de **microserviços**

FILIAIS

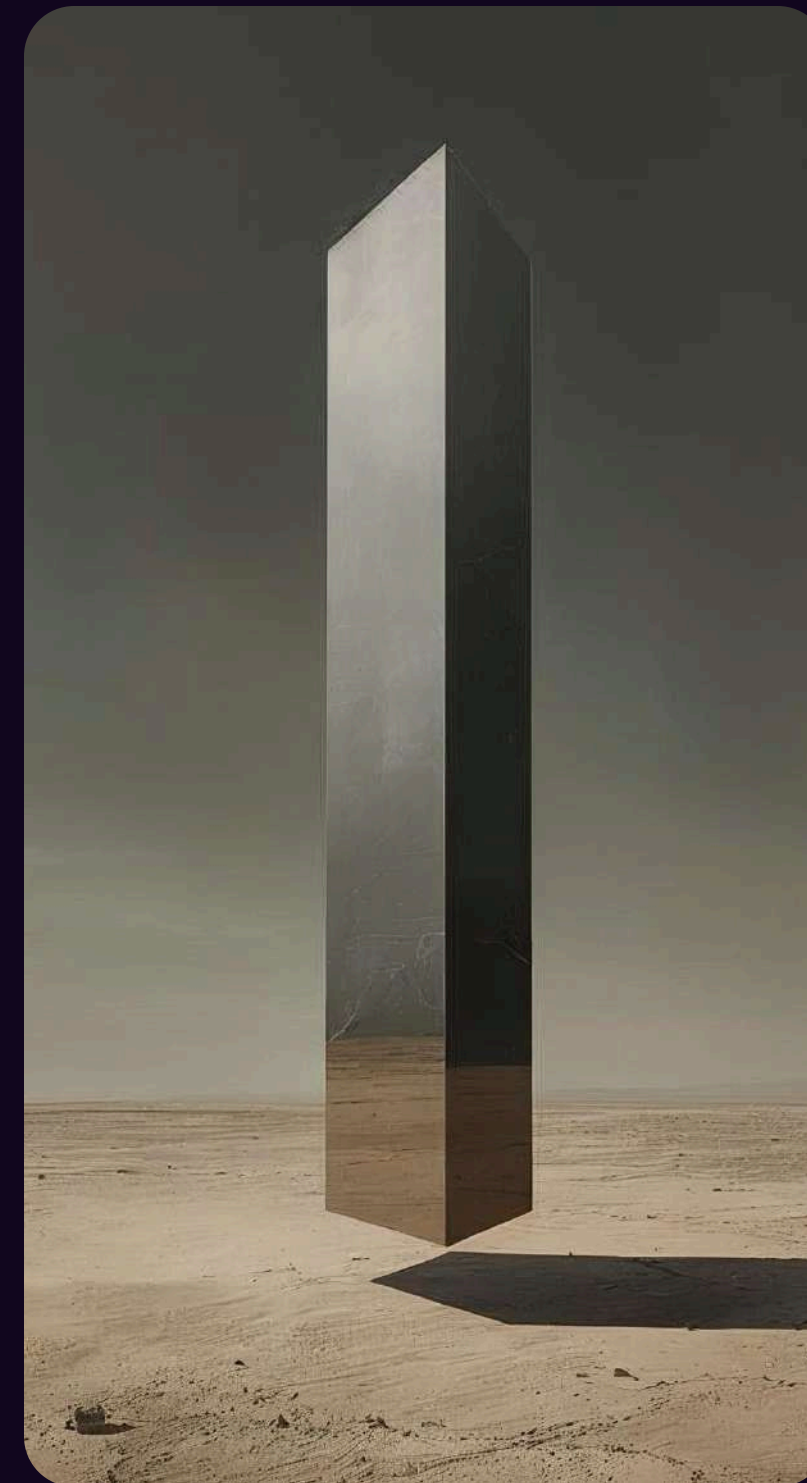
PRODUTOS

ESTOQUE

VENDAS

Detalhes de Infraestrutura

Monólito e microsserviços dizem respeito à forma como a aplicação é implantada e operada na infraestrutura, não à qualidade da modularização ou da modelagem do negócio.



Monólito vs
Microsserviços

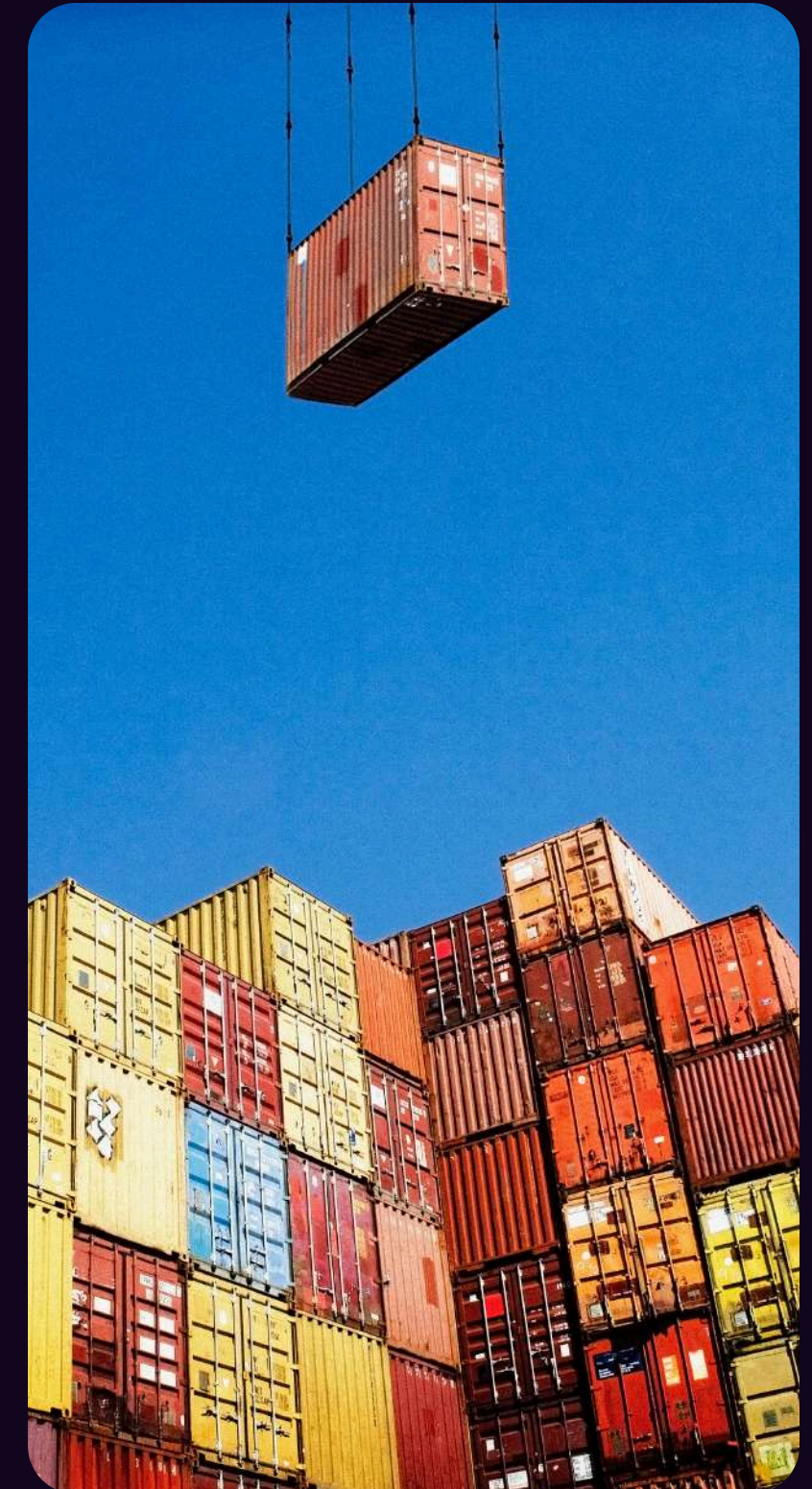


Modularização da Aplicação

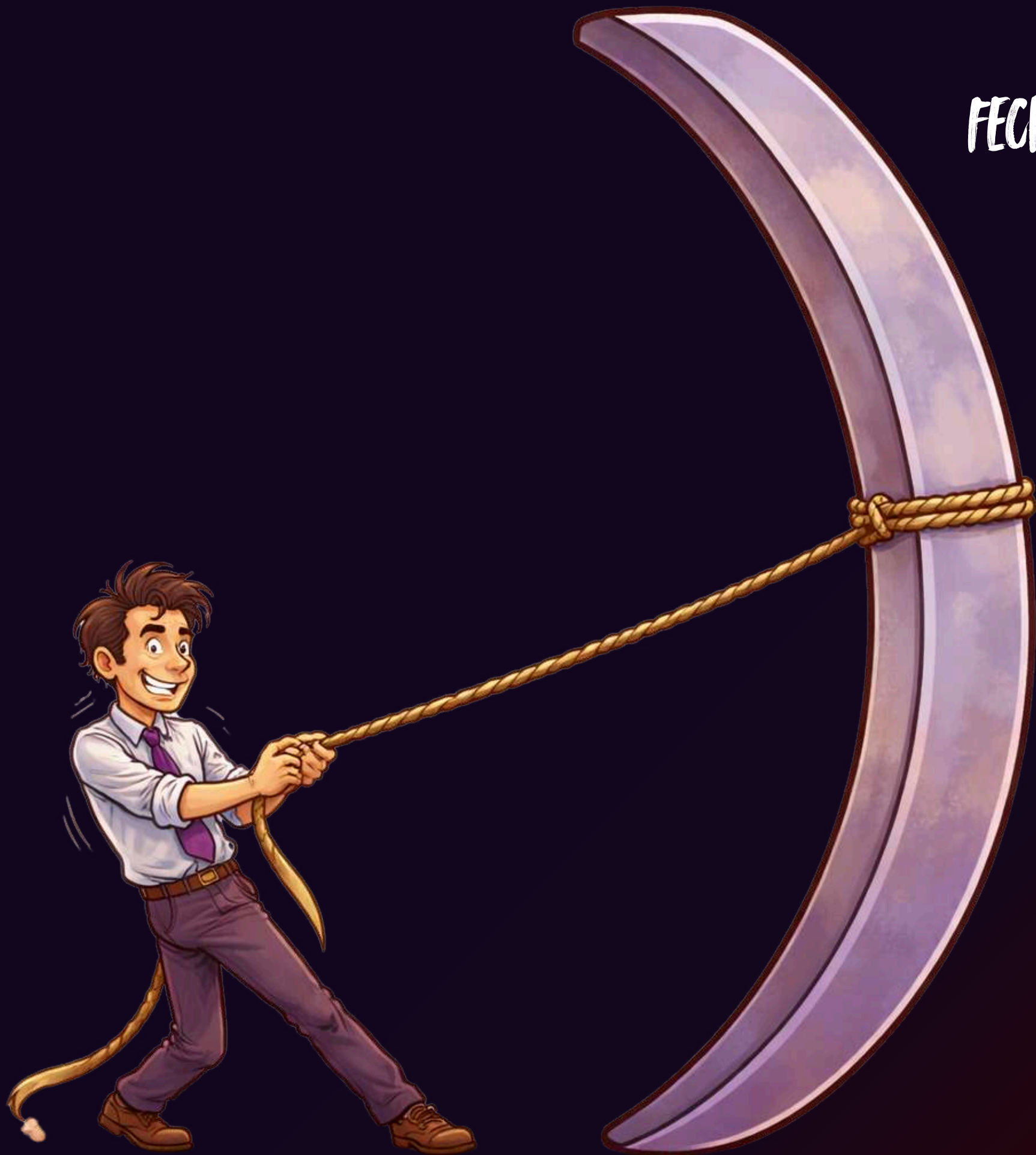
A diferença entre uma Big Ball of Mud e uma aplicação modularizada está na forma como o domínio do negócio é compreendido, organizado e refletido na estrutura do software.



Big Ball of Mud vs
Modularização



FECHANDO PARÊNTESES.

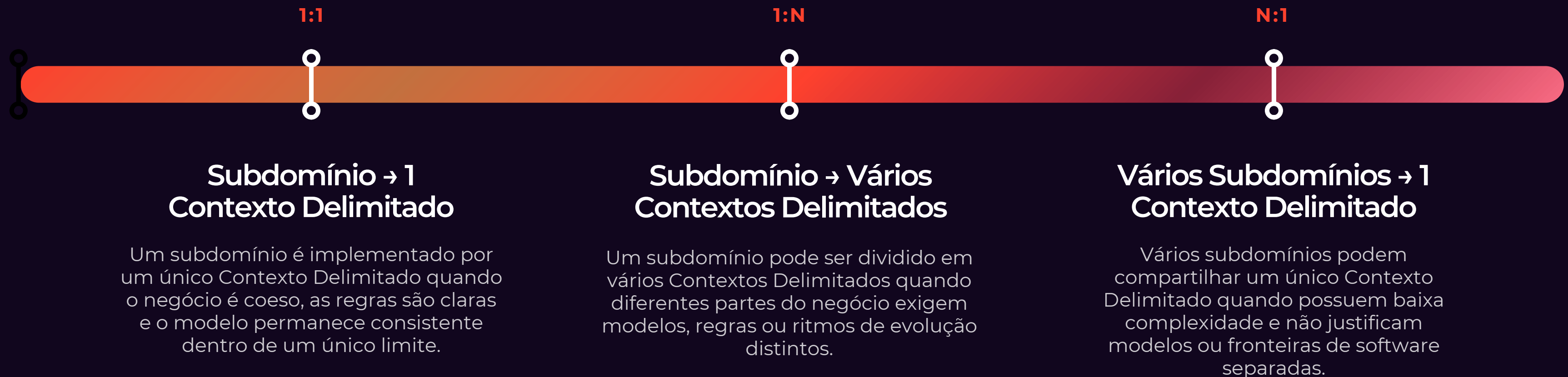


Subdomínio vs Ctx Delimitado

ASPECTO	SUBDOMÍNIO	CONTEXTO DELIMITADO
Natureza	Parte do negócio	Fronteira de software
Foco	O problema a ser resolvido	A solução em código
Origem	Surge da análise do negócio	Surge do design do sistema
Linguagem	Linguagem do negócio	Linguagem consistente do modelo
Papel	Define o que precisa existir	Define como isso será implementado

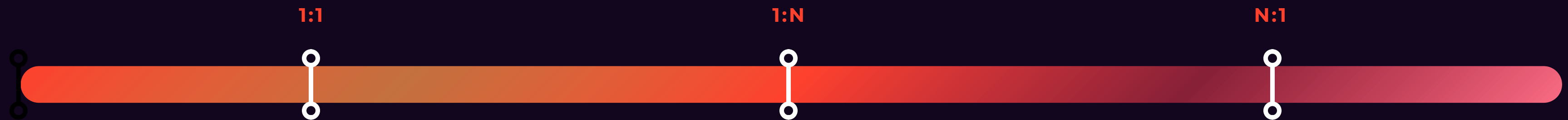
Subdomínio vs Ctx Delimitado

Não existe uma cardinalidade fixa entre Subdomínios e Contextos Delimitados, pois o negócio e o software evoluem de formas diferentes. Um subdomínio pode ser implementado por um ou vários contextos, dependendo da complexidade, das regras envolvidas e da necessidade de separação de modelos.



Subdomínio vs Ctx Delimitado

Não existe uma cardinalidade fixa entre Subdomínios e Contextos Delimitados, pois o negócio e o software evoluem de formas diferentes. Um subdomínio pode ser implementado por um ou vários contextos, dependendo da complexidade, das regras envolvidas e da necessidade de separação de modelos.



No subdomínio de **Produtos**, um único Contexto Delimitado de **Catálogo** pode atender bem quando as regras são simples, centralizando cadastro, categorização e consulta dos produtos da farmácia.

No subdomínio de **Vendas**, o negócio pode exigir Contextos Delimitados distintos para **Pedidos, Pagamentos e Entrega**, cada um lidando com regras próprias e evoluindo de forma independente.

Subdomínios menores, como **Configurações e Cadastros auxiliares**, podem ser agrupados em um único Contexto Delimitado quando não há conflitos de regras ou linguagem relevantes.

Ferramentas Estratégicas

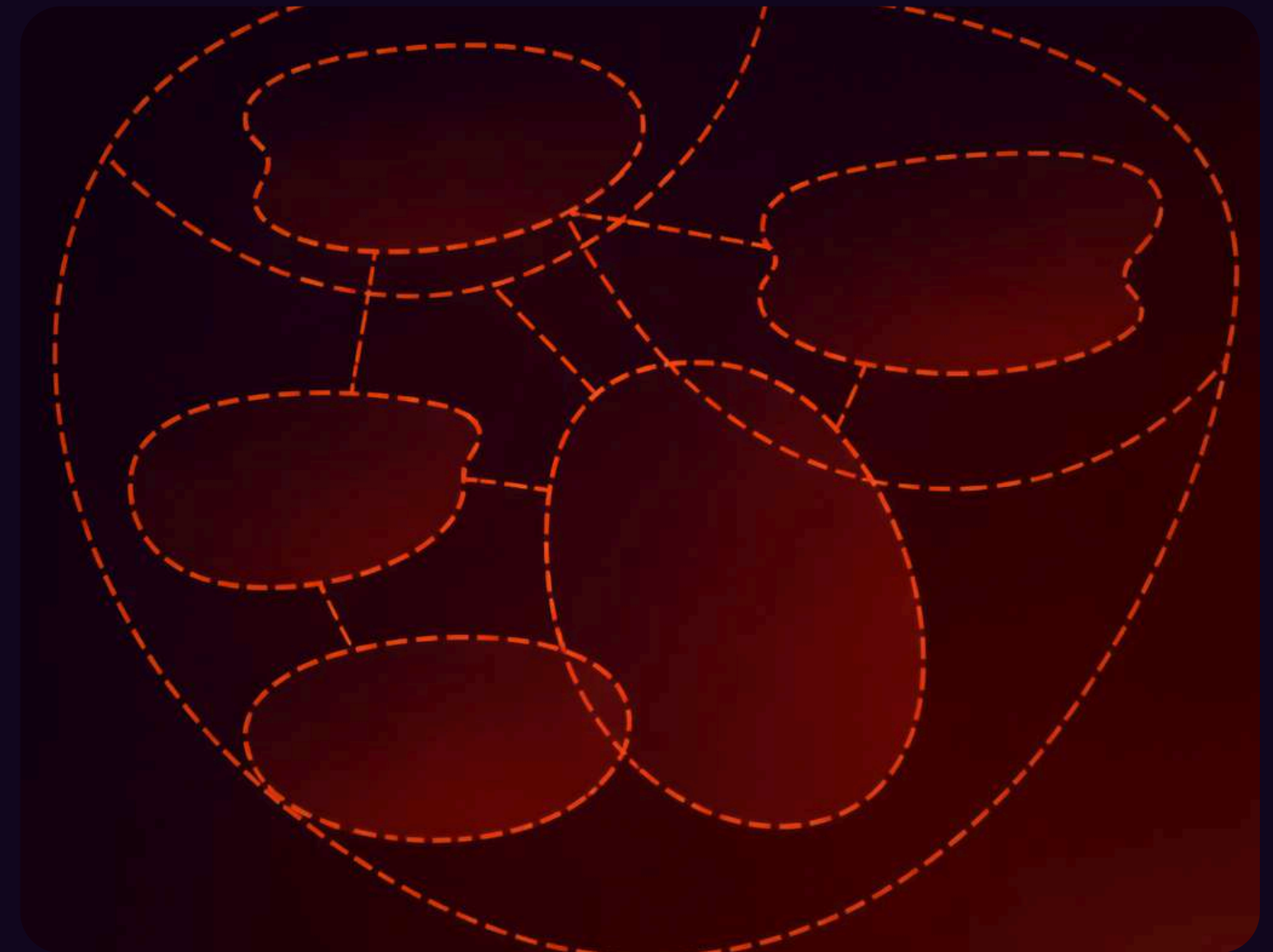
Contexto
Delimitado

Mapa de
Contexto

Linguagem
Ubíqua

Mapa de Contexto

Segundo o Domain-Driven Design, um Mapa de Contexto é uma representação que descreve como diferentes Contextos Delimitados se relacionam, se comunicam e dependem entre si, explicitando os tipos de integração, responsabilidades e limites entre esses contextos dentro de um sistema.



FUNDAMENTOS

PHARMACORE

FILIAIS



PRODUTOS



ESTOQUE



VENDAS



Mapa de Contexto

MENTORIA 360

FUNDAMENTOS

PHARMACORE

FILIAIS

PRODUTOS

COMO?

Mapa de Contexto

MENTORIA 360

ESTOQUE

VENDAS

Mapa de Contexto

FILIAIS



PRODUTOS



ESTOQUE

VENDAS

U

D

U

D

U

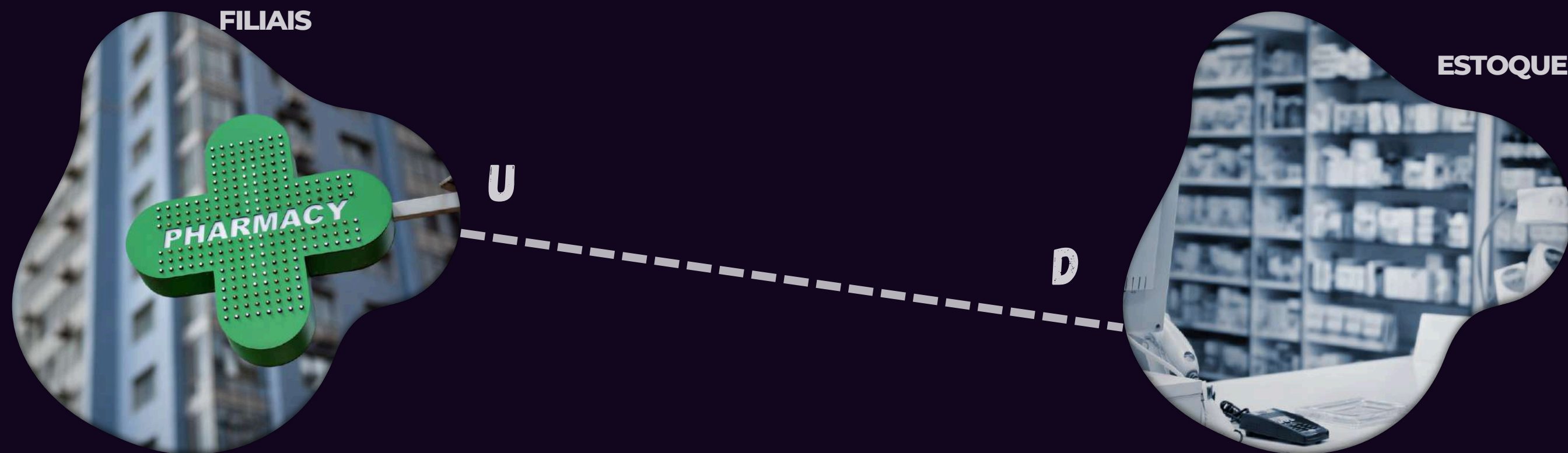
D

U

D

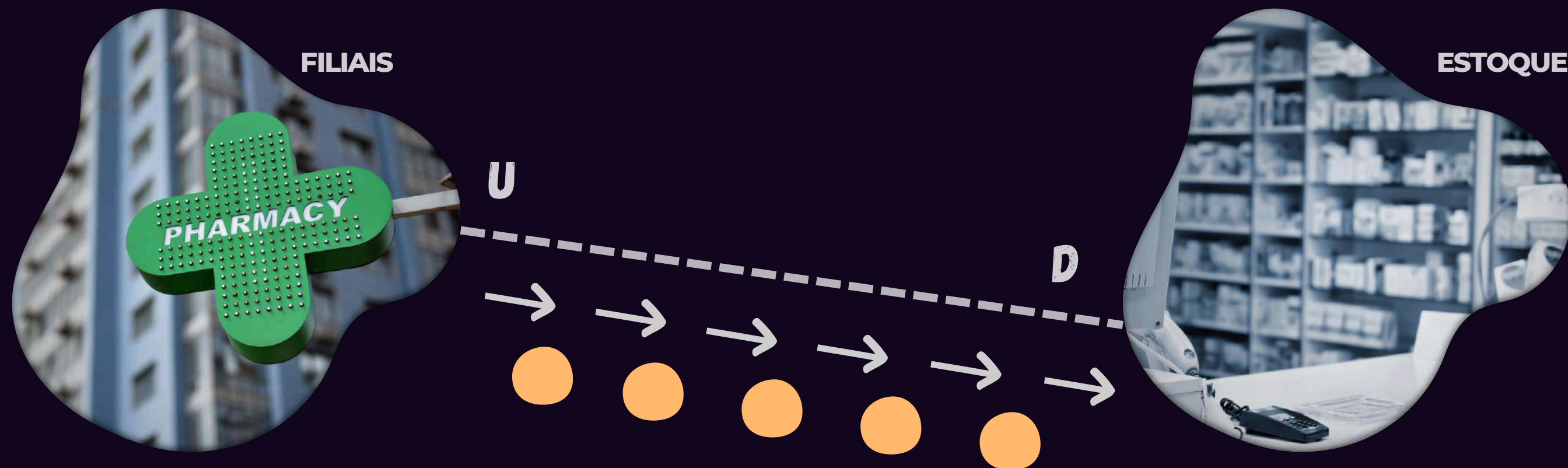
Mapa de Contexto

PHARMACORE



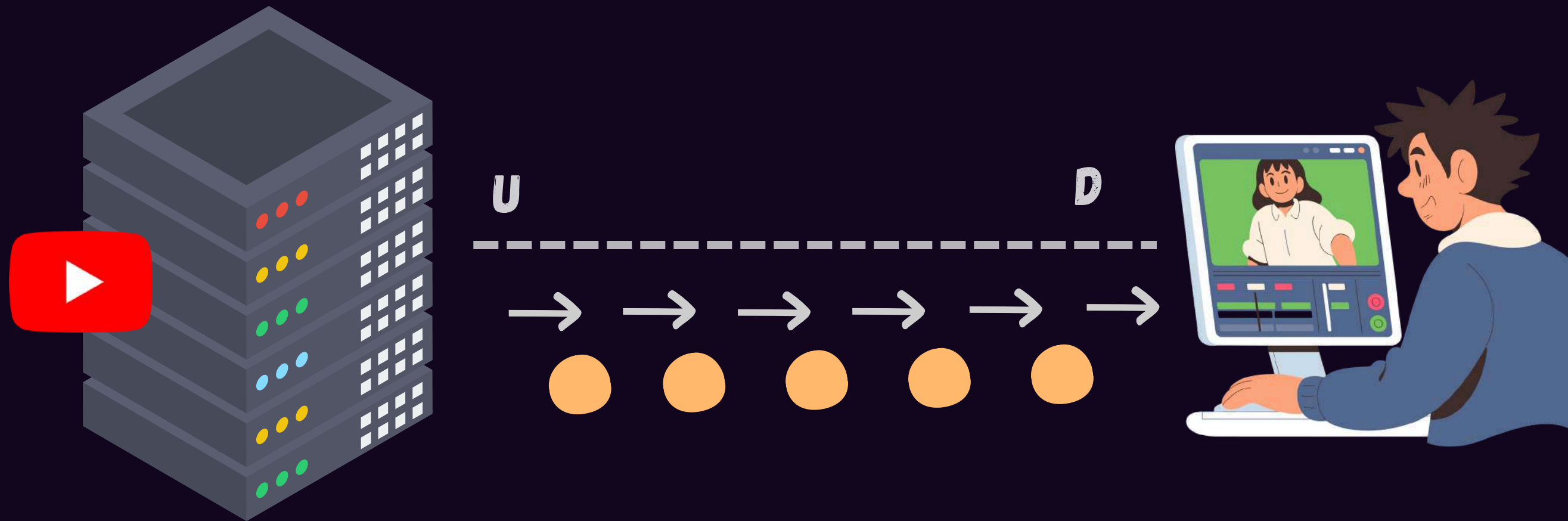
Mapa de Contexto

PHARMACORE

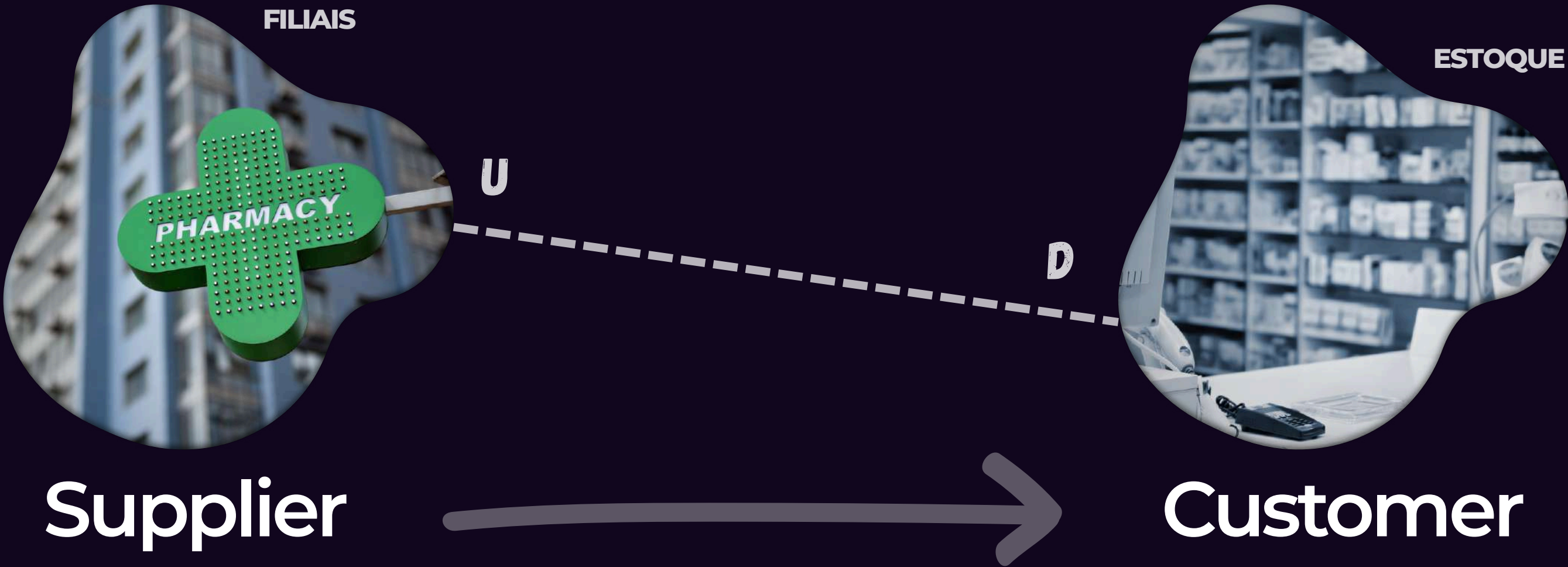


Mapa de Contexto

PHARMACORE



Mapa de Contexto



Supplier



filiais

U

Customer



estoque

D



```
{
  "name": "@pharmacore/estoque",
  "version": "1.0.0",
  "private": true,
  "description": "Contexto delimitado de Estoque",
  "main": "dist/index.js",
  "types": "dist/index.d.ts",
  "scripts": {
    "build": "tsc",
    "dev": "ts-node src/index.ts"
  },
  "dependencies": {
    "@pharmacore/filiais": "*"
  }
}
```

Supplier



filiais

U

Customer



estoque

D

```
{
  "name": "@pharmcore/estoque",
  "version": "1.0.0",
  "private": true,
  "description": "Software para gerenciamento de Estoque",
  "main": "dist/index.js",
  "types": "dist/index.d.ts",
  "scripts": {
    "build": "tsc",
    "dev": "ts-node src/index.ts",
  },
  "dependencies": {
    "@pharmcore/filiais": "*"
  }
}
```



Published Language (PL)

- DTOs

- BranchSnapshot
- BranchListItem

- Interfaces (Queries)

- BranchByIdQuery
- ActiveBranchesQuery
- BranchSearchQuery



```
{
  "name": "@pharmacore/estoque",
  "version": "1.0.0",
  "private": true,
  "description": "Contexto delimitado de Estoque",
  "main": "dist/index.js",
  "types": "dist/index.d.ts",
  "scripts": {
    "build": "tsc",
    "dev": "ts-node src/index.ts"
  },
  "dependencies": {
    "@pharmacore/filiais-pl": "*"
  }
}
```

Ferramentas Estratégicas

Contexto
Delimitado

Mapa de
Contexto

Linguagem
Ubíqua

Linguagem Ubíqua

É uma linguagem comum e compartilhada entre especialistas do domínio e desenvolvedores, usada de forma consistente em conversas, documentação e no código, garantindo que os conceitos, regras e significados do domínio sejam compreendidos da mesma forma dentro de um **Contexto Delimitado**.



O que é um produto?



Produto



PRODUTOS



ESTOQUE



PRODUTOS

← Produto →



ESTOQUE

```
Produto (Catalog)
- catalogProductId: "CAT-000231"
- sku: "SKU-231"
- gtin: "7896212401783"
- nomeComercial: "Dipirona Monoidratada 500mg (20 comprimidos)"
- marca: "GenPharma"
- categoria: "Medicamentos"
- subcategoria: "Analgésicos e Antitérmicos"
- descricaoCurta: "Alívio de dor e febre"
- formaApresentacao: "Comprimido"
- precoVenda: 14.90
- statusCatalogo: "ATIVO"
- tagsBusca: ["dipirona", "dor", "febre", "analgésico"]
```

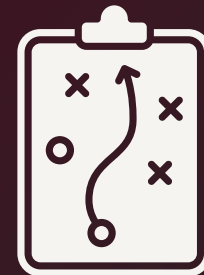
```
Produto (Inventory)
- stockProductId: "STK-009812"
- sku: "SKU-231" # referência ao item vendável
- gtin: "7896212401783"
- nomeOperacional: "DIPIRONA 500MG CPD 20UN"
- unidadeMovimentacao: "CAIXA"
- controlado: false
- exigeLote: true
- exigeValidade: true
- localArmazenamentoPadrao: "A2-PRAT-03"
- politicaReposicao:
  minimo: 10
  maximo: 80
- statusEstoque: "ESTOCAVEL"
```

Roadmap do Desenvolvimento



ESTRATÉGICO

Compreender o domínio do negócio e organizar a aplicação em contextos delimitados e módulos bem definidos.



TÁTICO

Aplicar padrões de modelagem e arquitetura para transformar o domínio em soluções consistentes e sustentáveis.

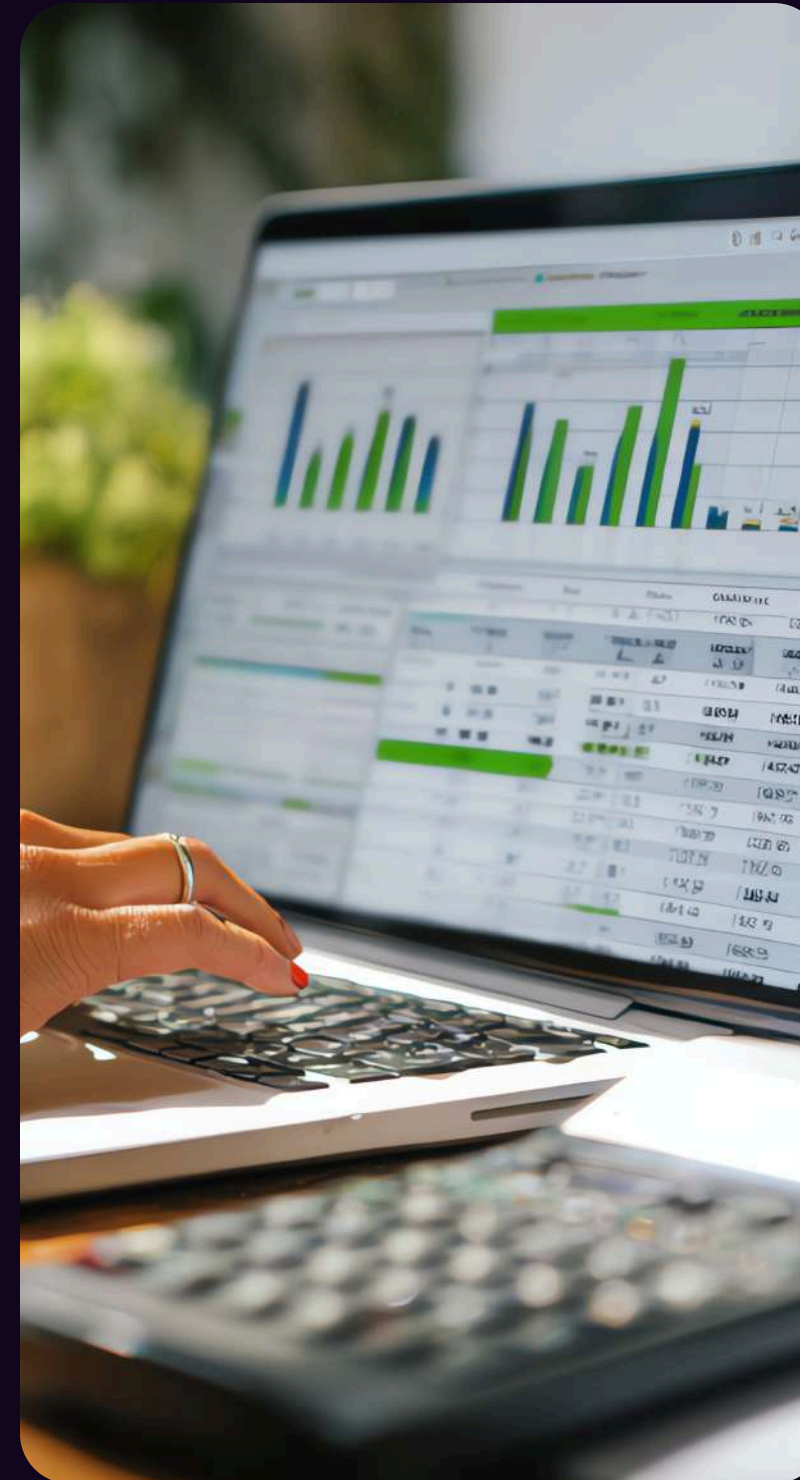


OPERACIONAL

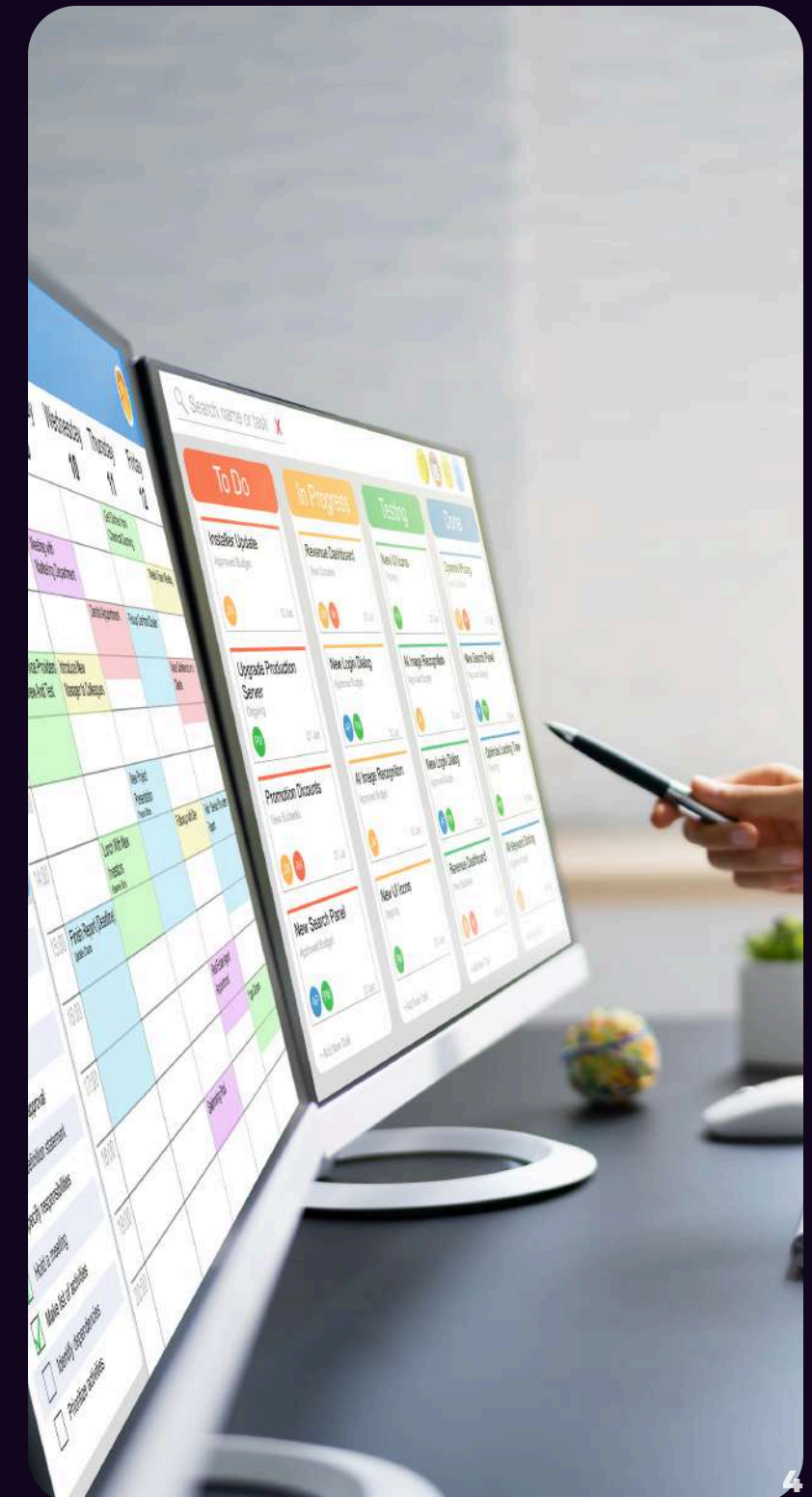
Implementar o sistema em código, integrando frameworks e bibliotecas de mercado sem comprometer as regras de negócio.

Roadmap Tático

Aplicar padrões de modelagem e princípios de arquitetura limpa para transformar o domínio em soluções coerentes, sustentáveis e bem estruturadas.

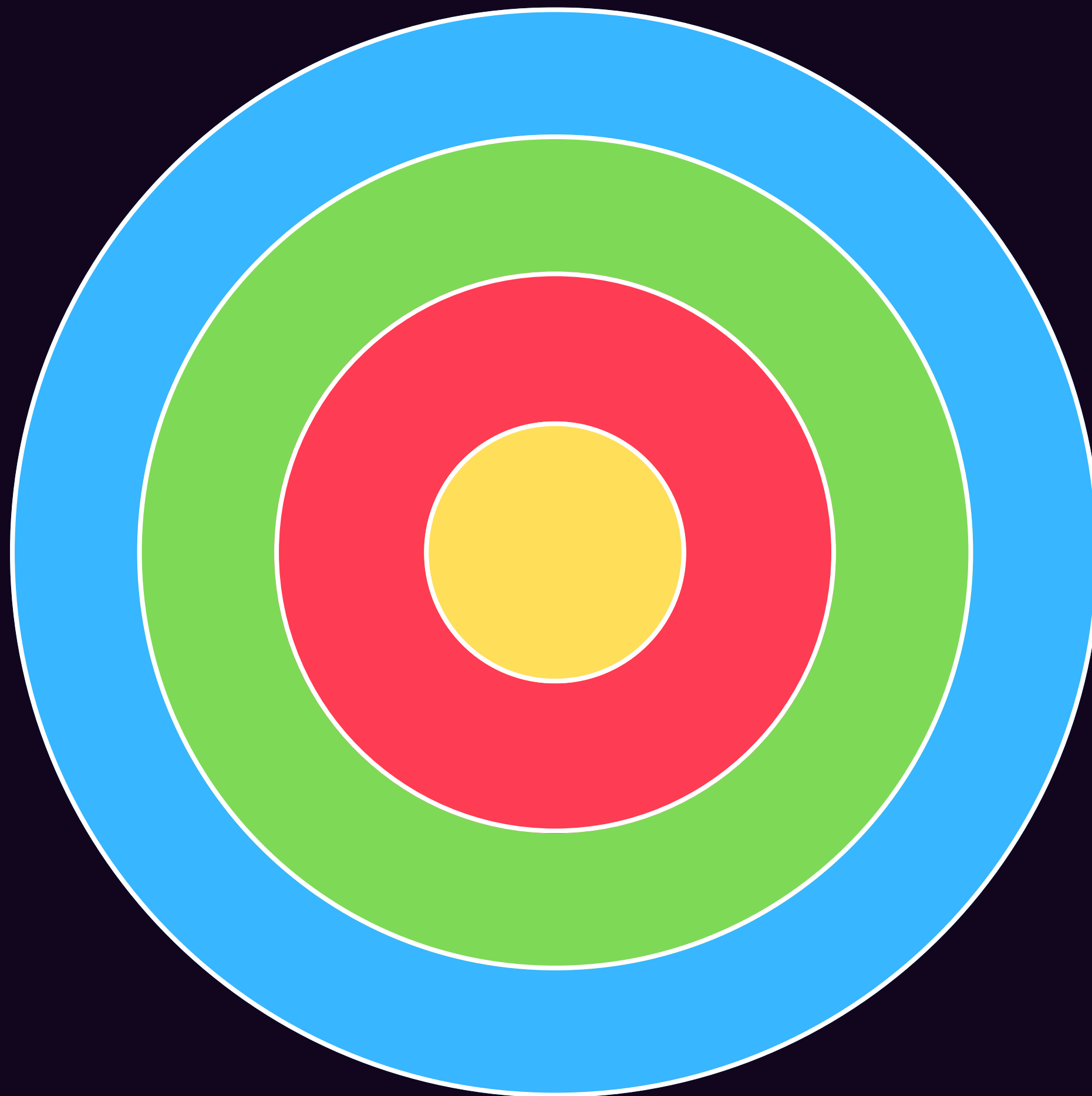
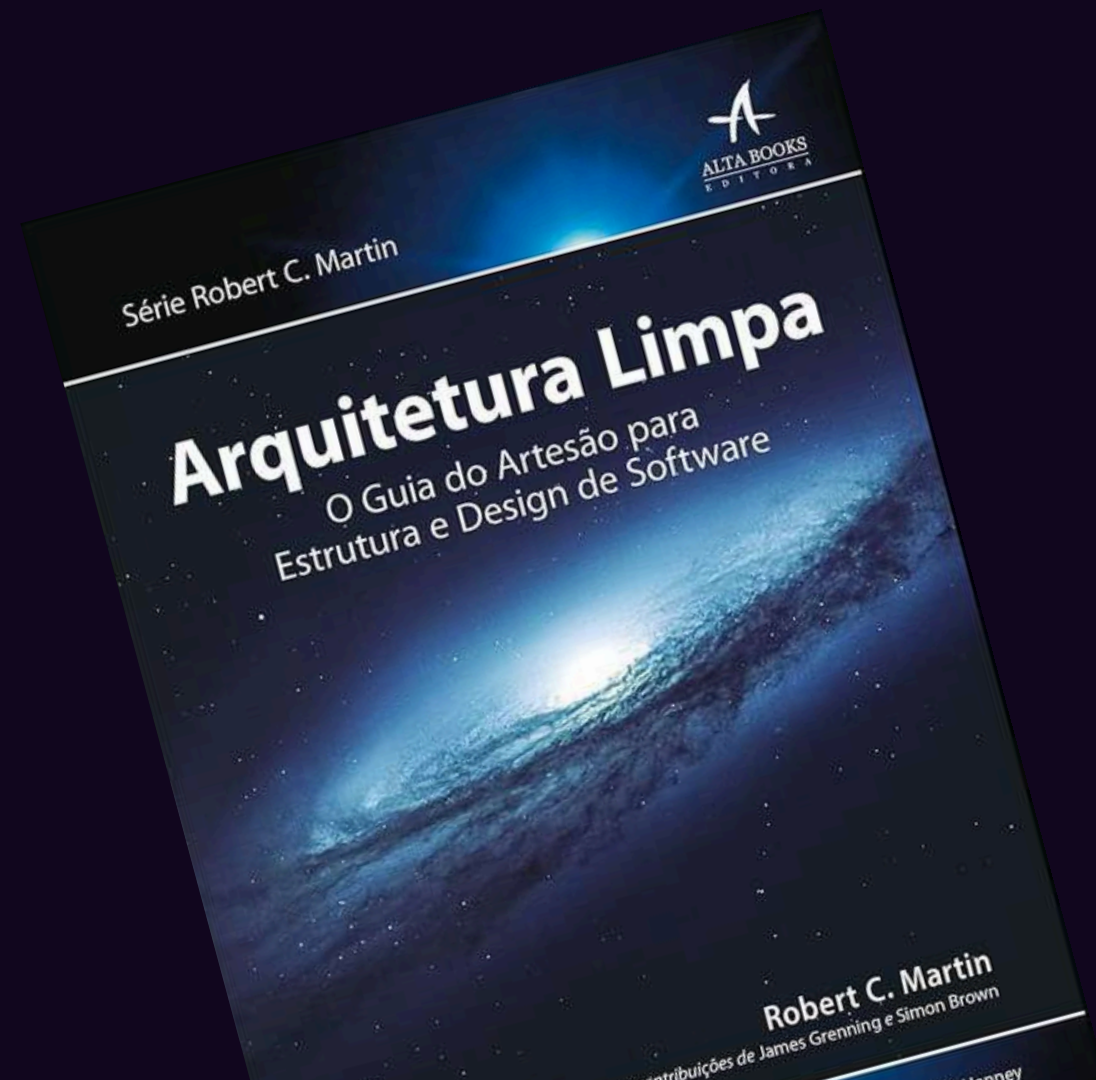


Arquitetura se
consolida nos padrões



Arquitetura Limpa

Uma abordagem para organizar a aplicação em camadas, separando responsabilidades e mantendo as regras de negócio independentes de detalhes tecnológicos.

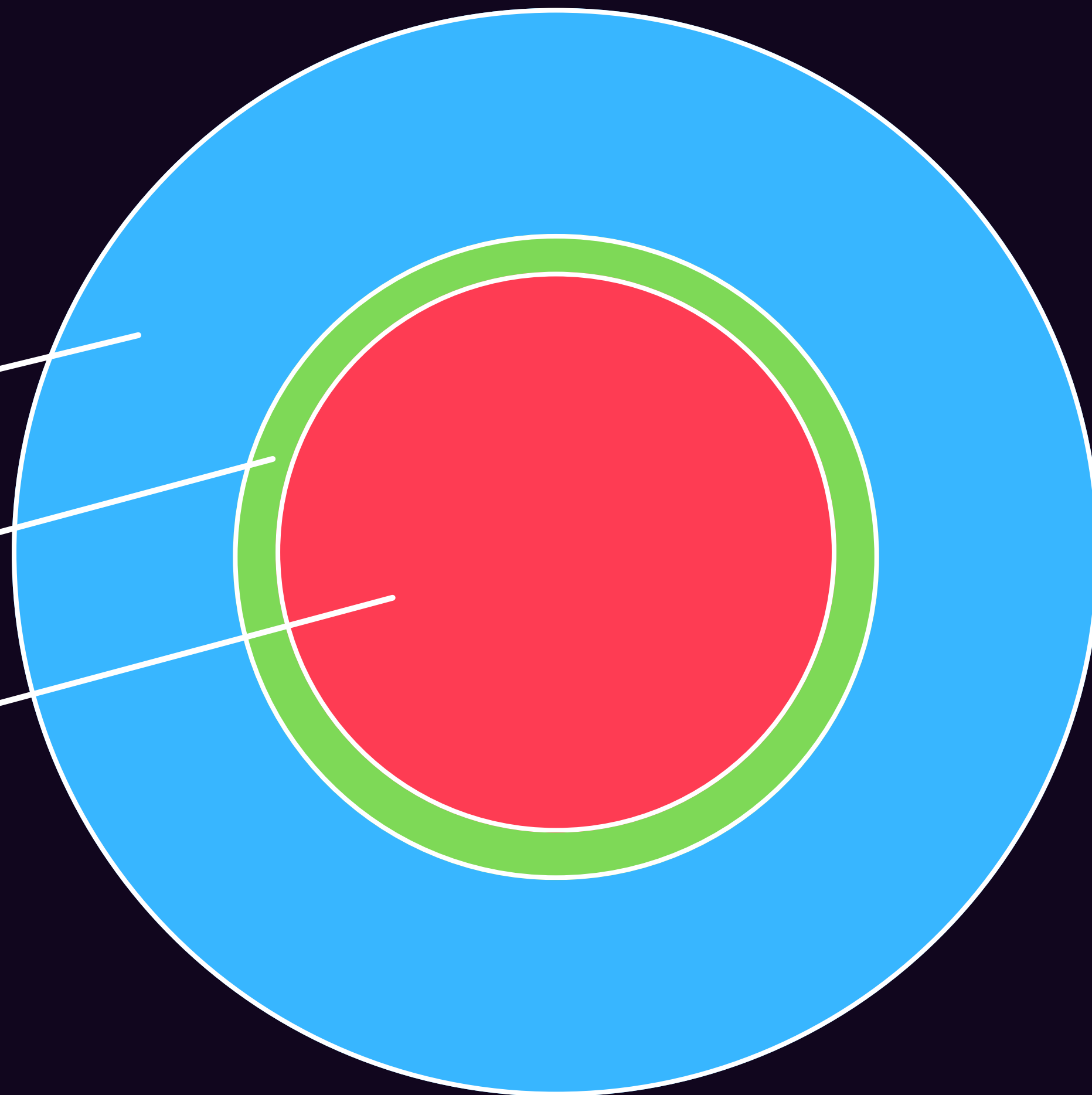


Separação Importante

TECNOLOGIA

FRONTEIRA

NEGÓCIO



ERP

Big Ball of Mud

Uma das possibilidades de organização é aplicar a Arquitetura Limpa como uma estrutura única de camadas, organizando toda a aplicação em um único modelo. Nessa abordagem, as regras de negócio, os casos de uso e os detalhes técnicos convivem sem limites claros.



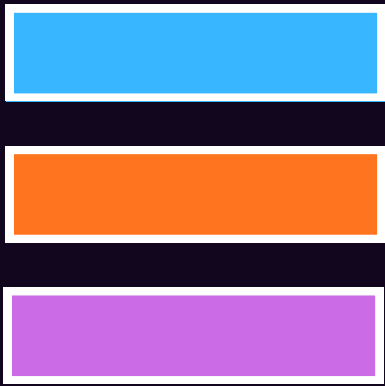
ERP Modularizado



FILIAIS



PRODUTOS

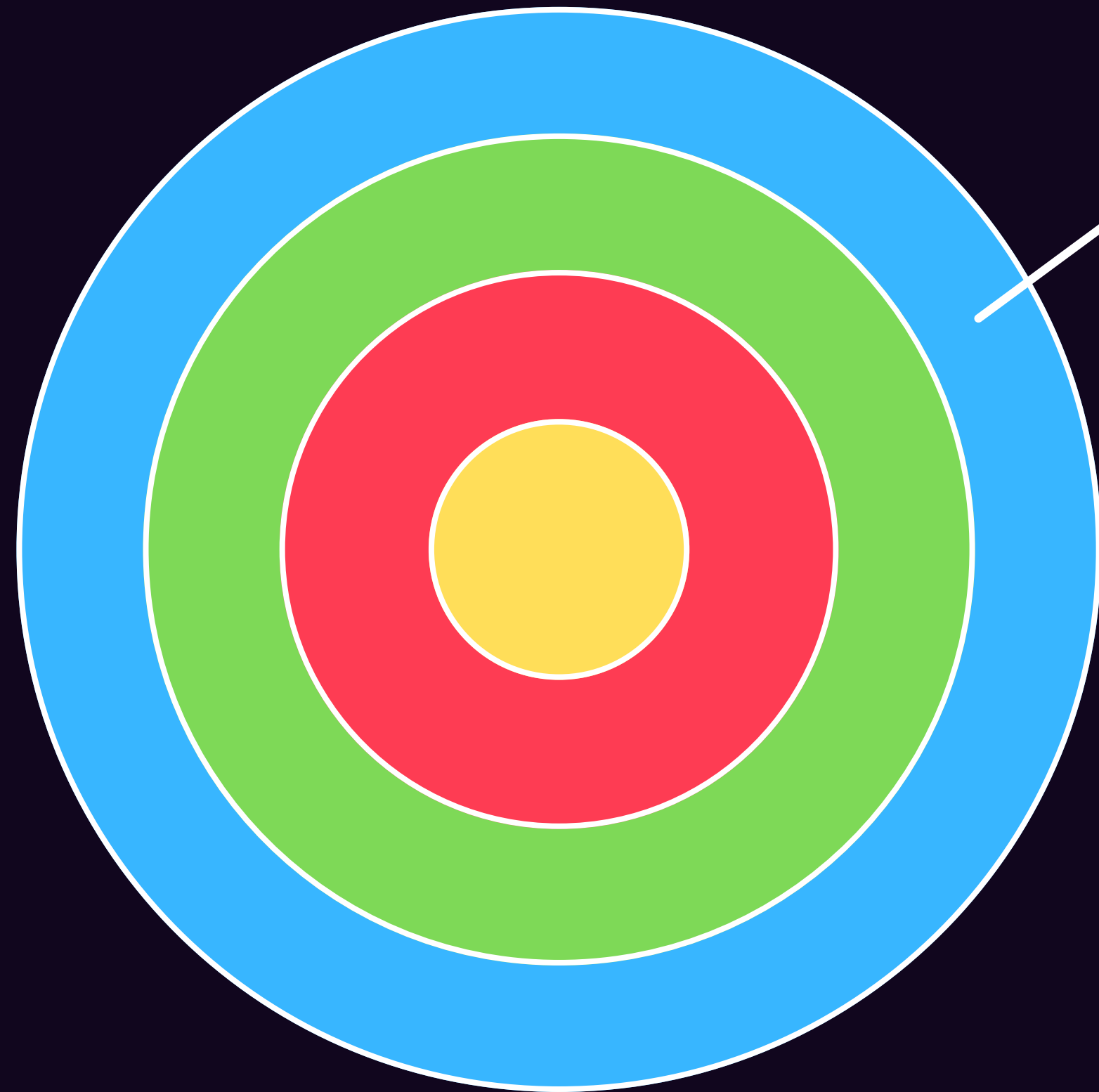


ESTOQUE

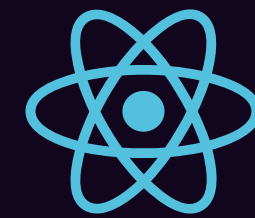


VENDAS





Frameworks & Drivers



— React



— NestJS



— Prisma



— Postgres



— NextJS

ERP Modularizado

FRONTEND

BACKEND



FILIAIS



PRODUTOS



ESTOQUE



VENDAS



Regras Protegidas



FILIAIS



PRODUTOS



ESTOQUE

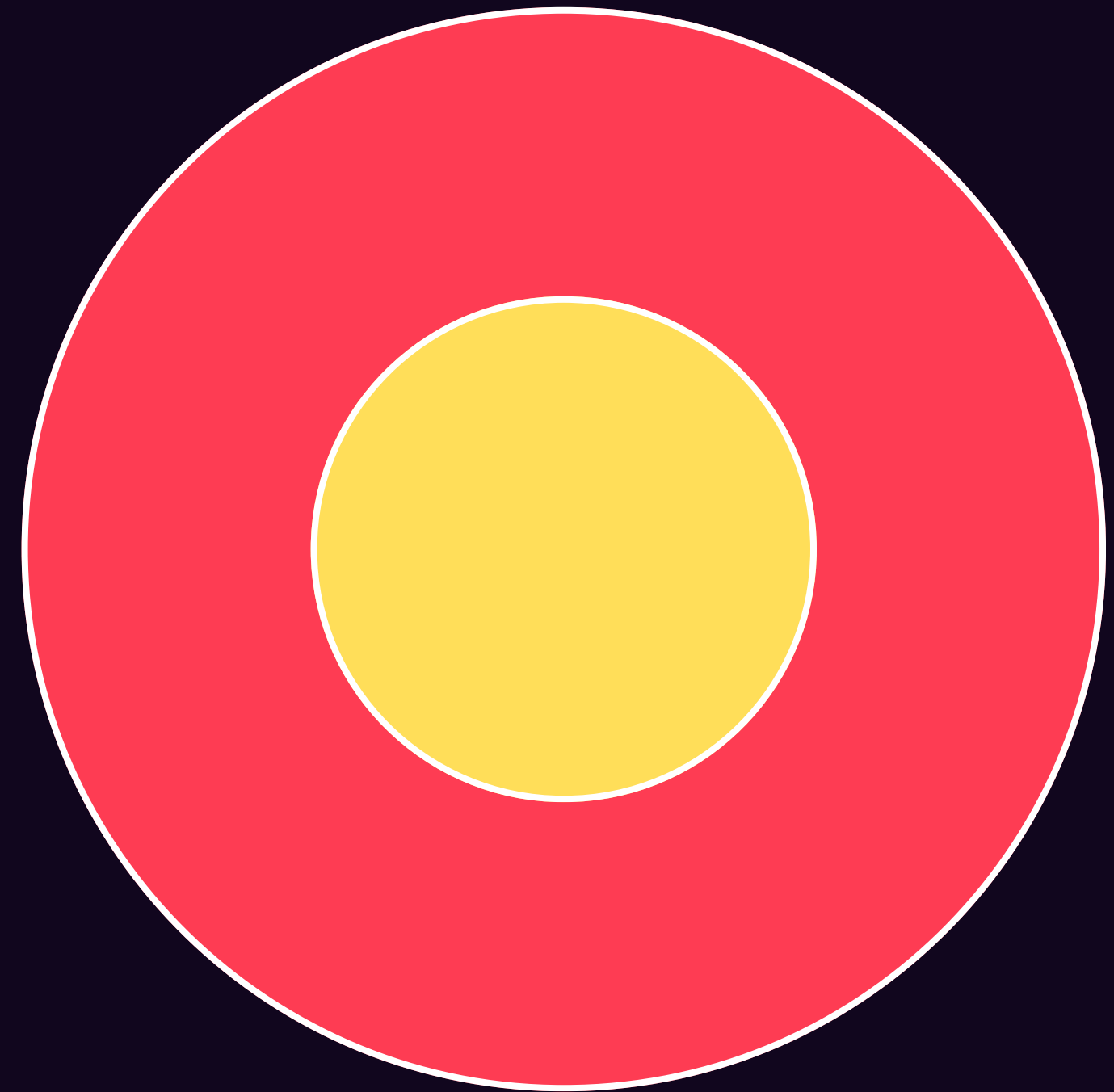


VENDAS

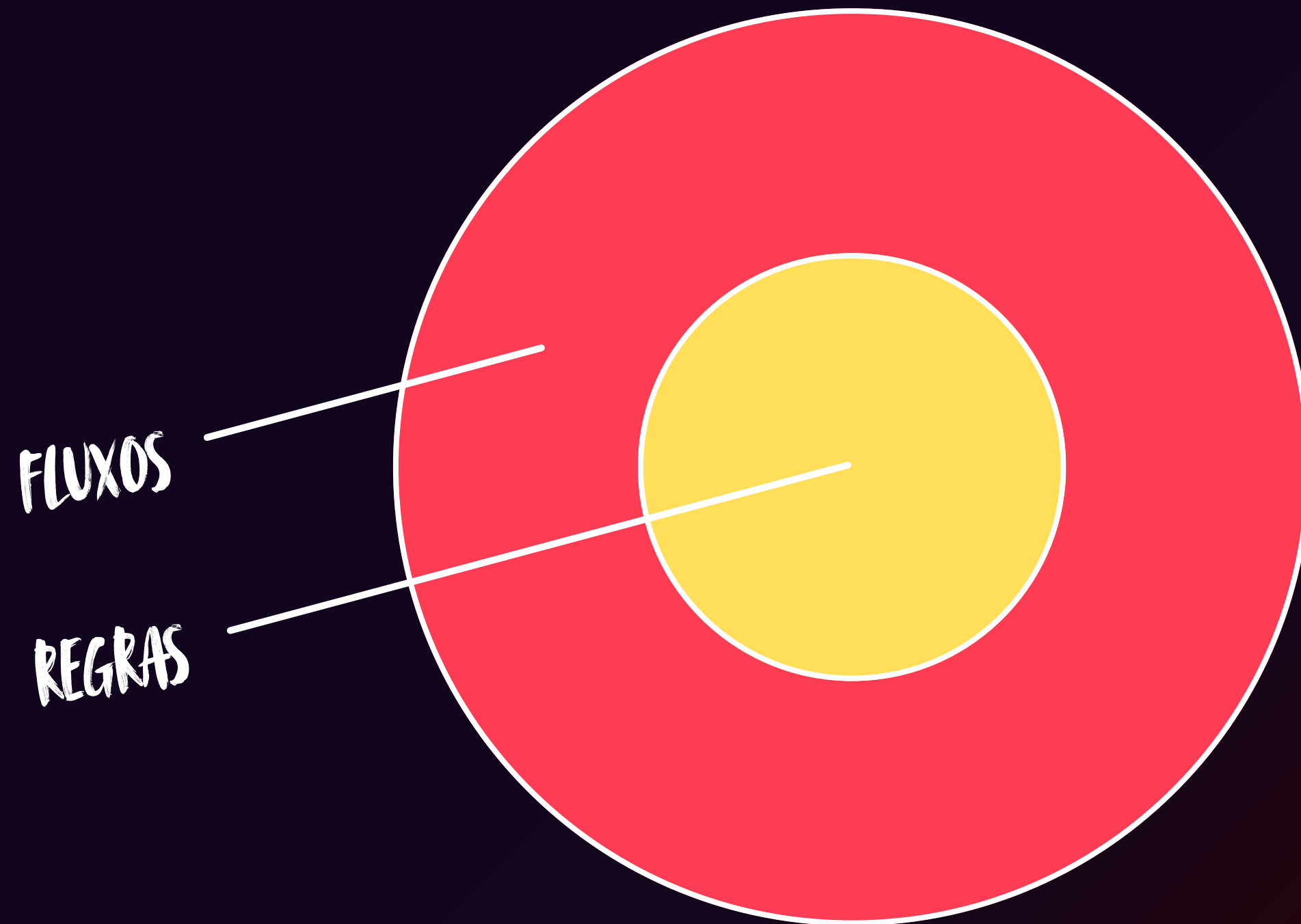


Modelando o **Negócio**

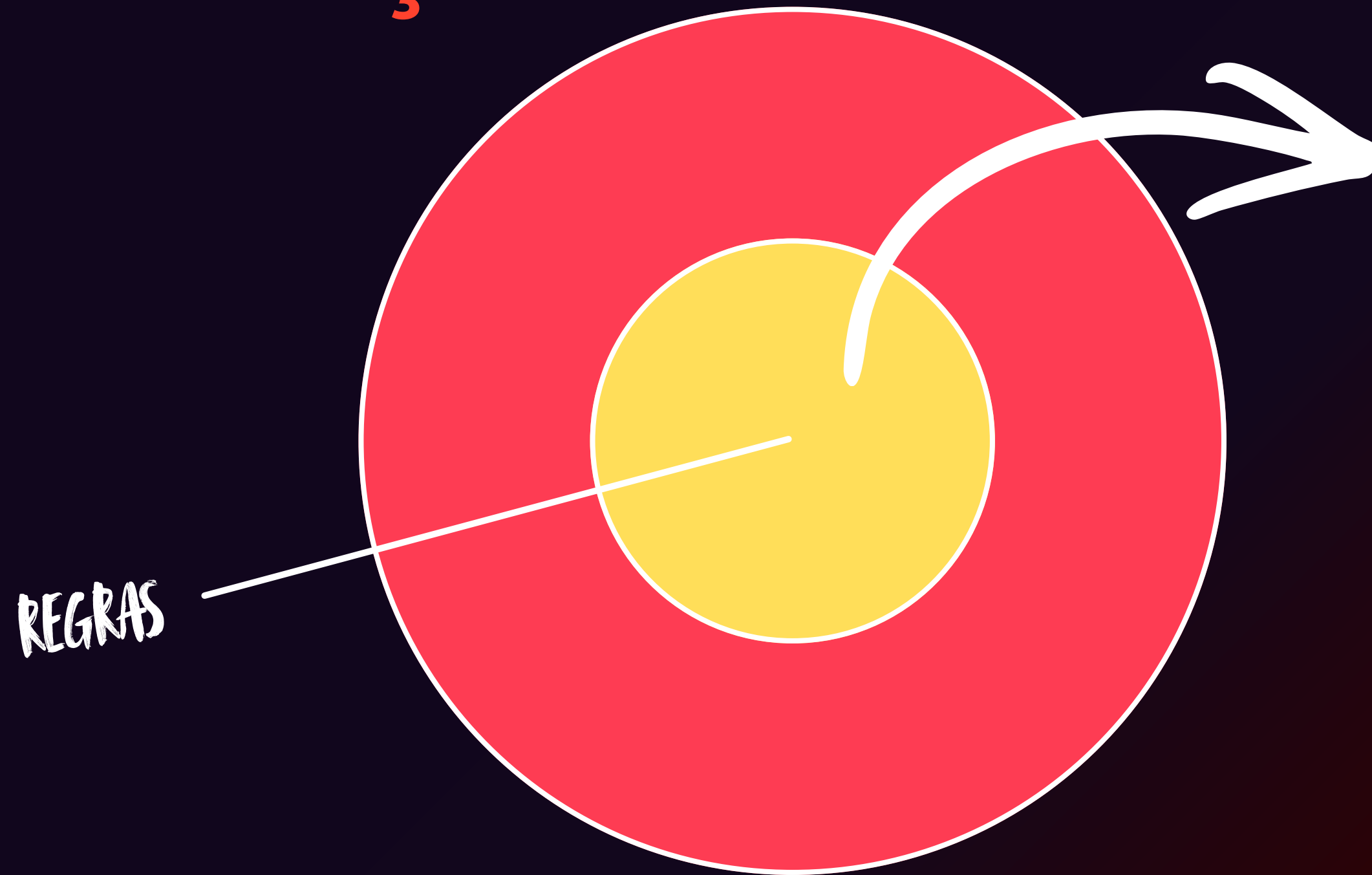
Foco nas regras de negócio e nos fluxos
que expressam como o domínio
realmente funciona, independentemente
de tecnologia.



Simplificando



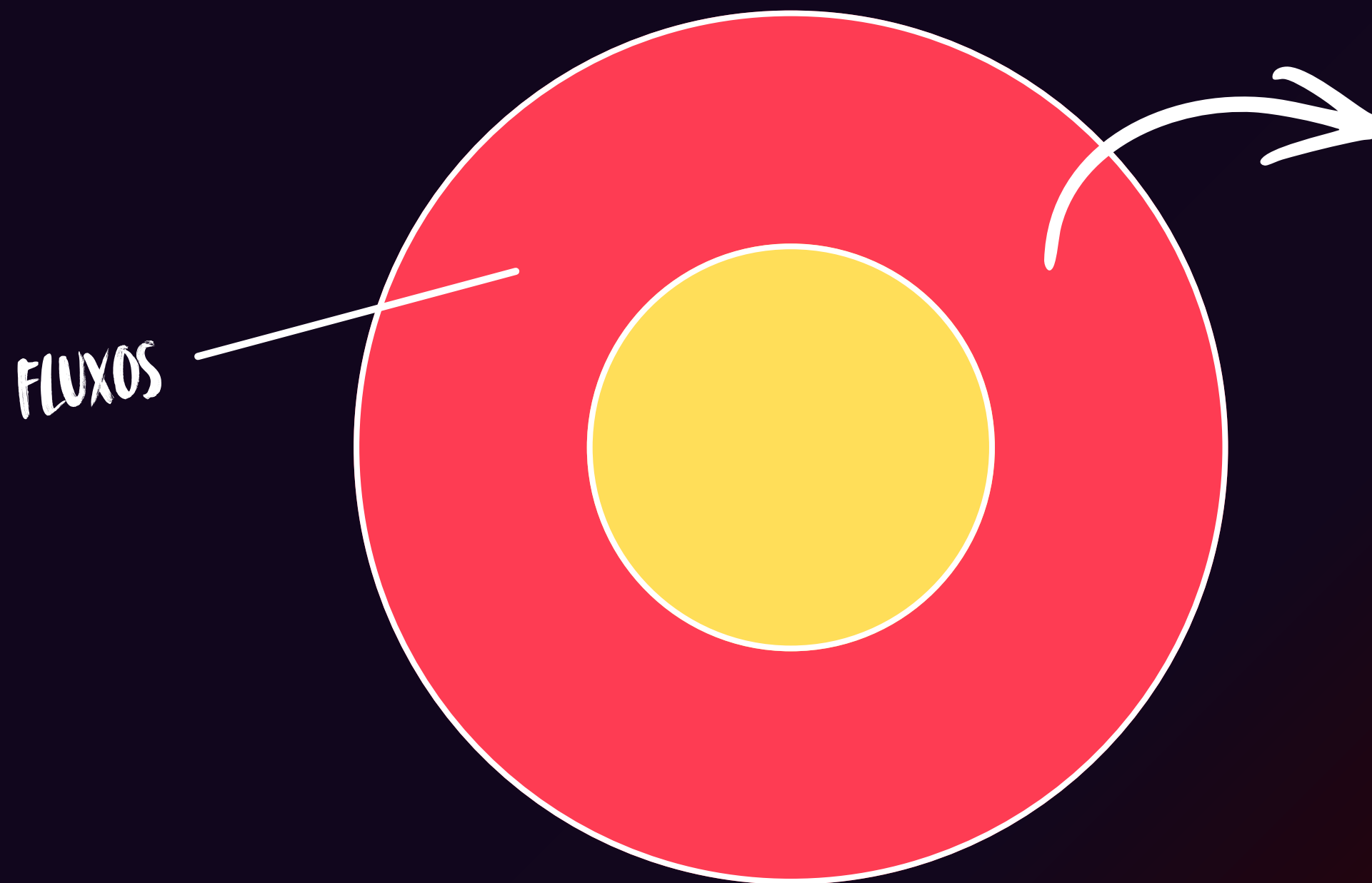
Contexto de Autenticação



Entidades

- User
- Password
- Role
- Permission

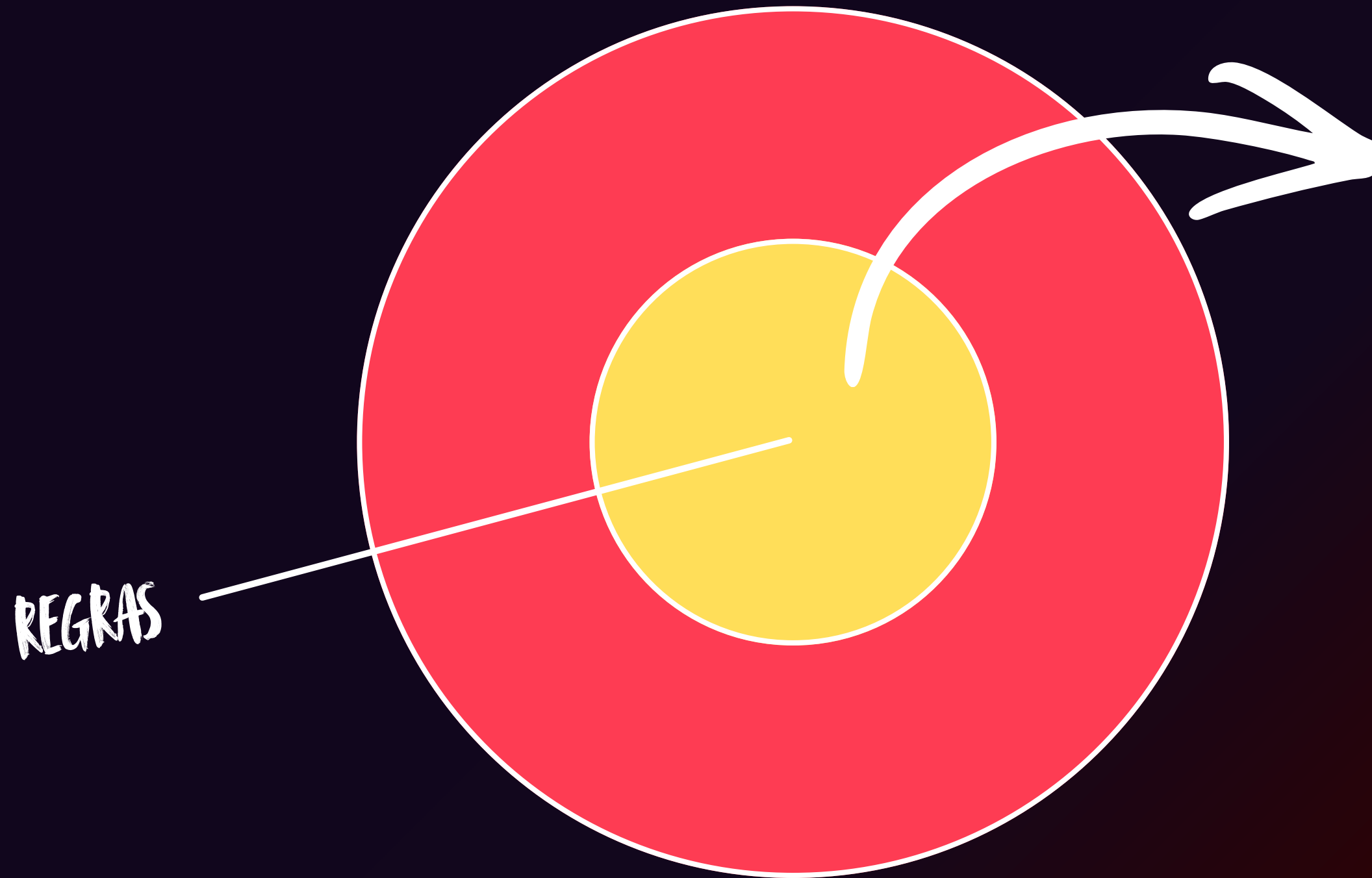
Contexto de Autenticação



Casos de Uso

- Register User
- Authenticate User
- Update User Profile
- Change Password
- Create Role
- Assign Role to User
- Remove Role from User
- Update Role Permissions

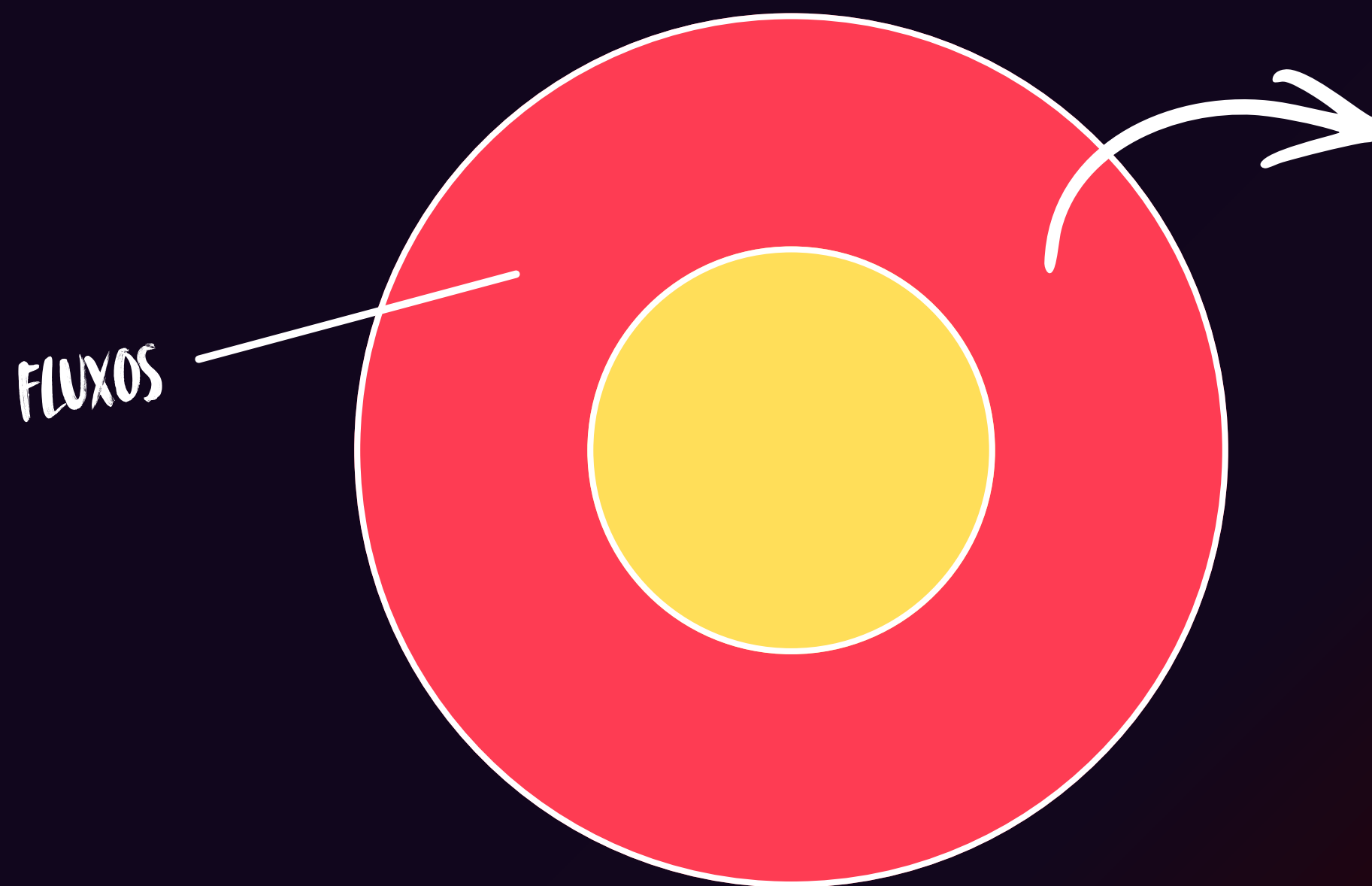
Contexto de **Produto**



Entidades

- Category
- Subcategory
- Product
- Brand
- Medication

Contexto de Produto



Casos de Uso

- Create Category
- Create Subcategory
- Create Brand
- Create Product
- Update Product
- Deactivate Product
- Register Medication Details
- Search Products

Caso de Uso

O caso de uso define o fluxo da aplicação, recebendo dados de entrada, orquestrando regras e infraestrutura, e produzindo uma saída consistente para o restante do sistema.



```
import { Result } from "../result";

export interface UseCase<IN, OUT> {
  execute(data: IN): Promise<Result<OUT>>;
}
```

Casos de Uso

```
export interface LoginIn {  
  email: string;  
  password: string;  
}
```

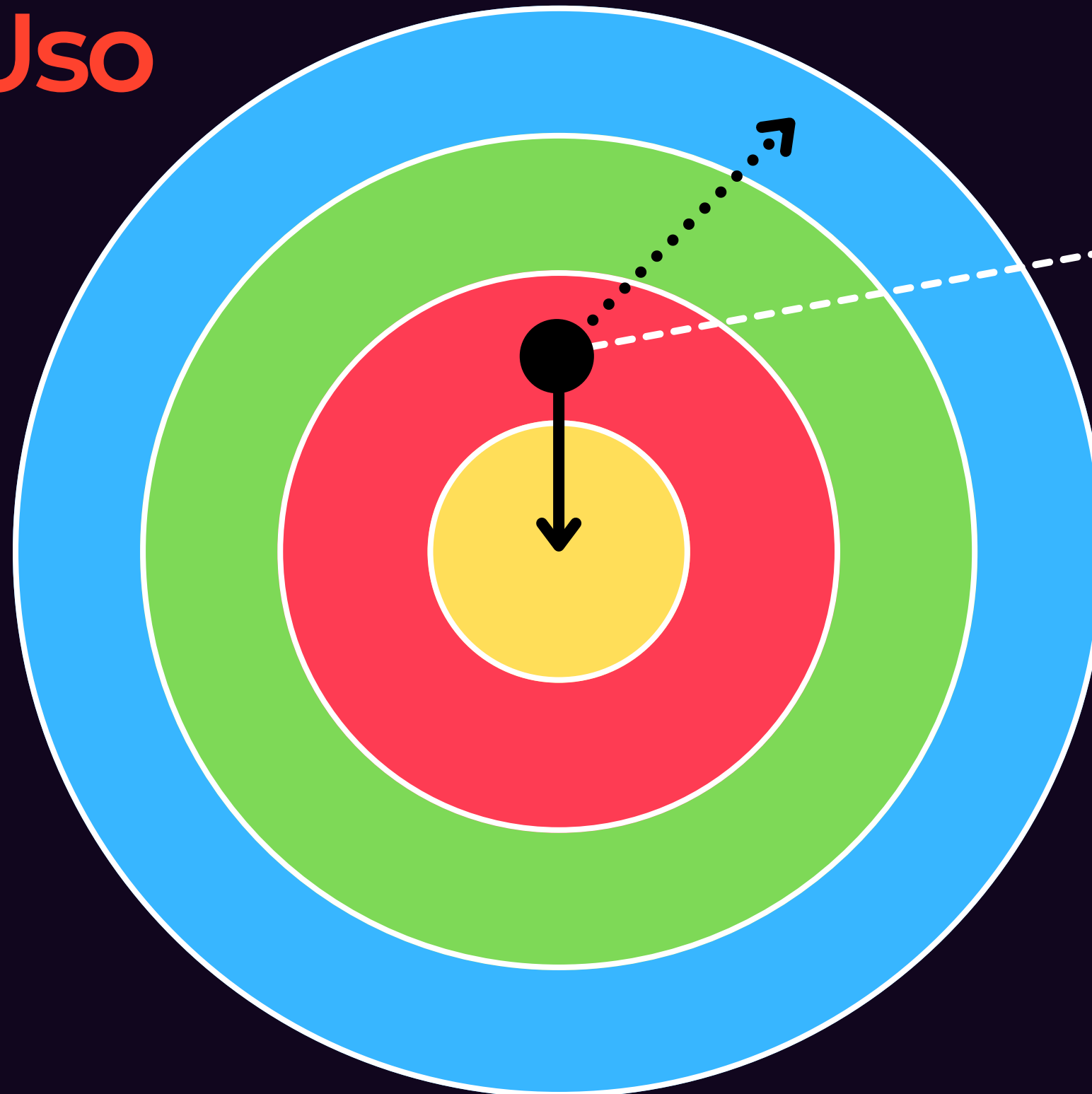
```
export interface LoginOut {  
  id: string;  
  name: string;  
  email: string;  
  roles: Role[];  
}
```

```
export class LoginUseCase implements UseCase<LoginIn, LoginOut> {  
  
  async execute(input: LoginIn): Promise<Result<LoginOut>> {  
    // ...  
  }  
}
```

Regras

Banco de Dados

Caso de **Uso**



Na Arquitetura Limpa, o caso de uso ocupa uma **posição central na aplicação**, sendo o ponto responsável por orquestrar o fluxo da lógica de negócio.

Ele **acessa diretamente as regras de negócio**, representadas no núcleo da aplicação, e coordena suas execuções de forma explícita e intencional.

Já o acesso a recursos externos, como banco de dados, serviços de mensageria ou APIs, **acontece de forma indireta**, por meio de portas e adaptadores, mantendo o domínio protegido de detalhes de infraestrutura.

Dessa forma, o caso de uso se torna o **principal elo entre o negócio e o mundo externo**, controlando o fluxo da aplicação sem acoplá-la a tecnologias específicas.

Ferramentas Táticas

A modelagem de domínio **representa o núcleo da aplicação**, onde as regras de negócio são expressas por meio de entidades, objetos de valor, serviços e eventos. É nessa camada que o sistema deixa de ser apenas dados e passa a **incorporar comportamentos** e decisões reais do negócio.



Objeto de Valor

Encapsula um valor do domínio e suas regras, garantindo consistência por meio da imutabilidade.



Entidade

Modela elementos do negócio com identidade própria e comportamento ao longo do tempo.



Serviço de Domínio

Encapsula regras de negócio que não pertencem naturalmente a uma única entidade.



Evento de Domínio

Representa fatos importantes que ocorreram no domínio e podem gerar reações no sistema.

Entidade Anêmica

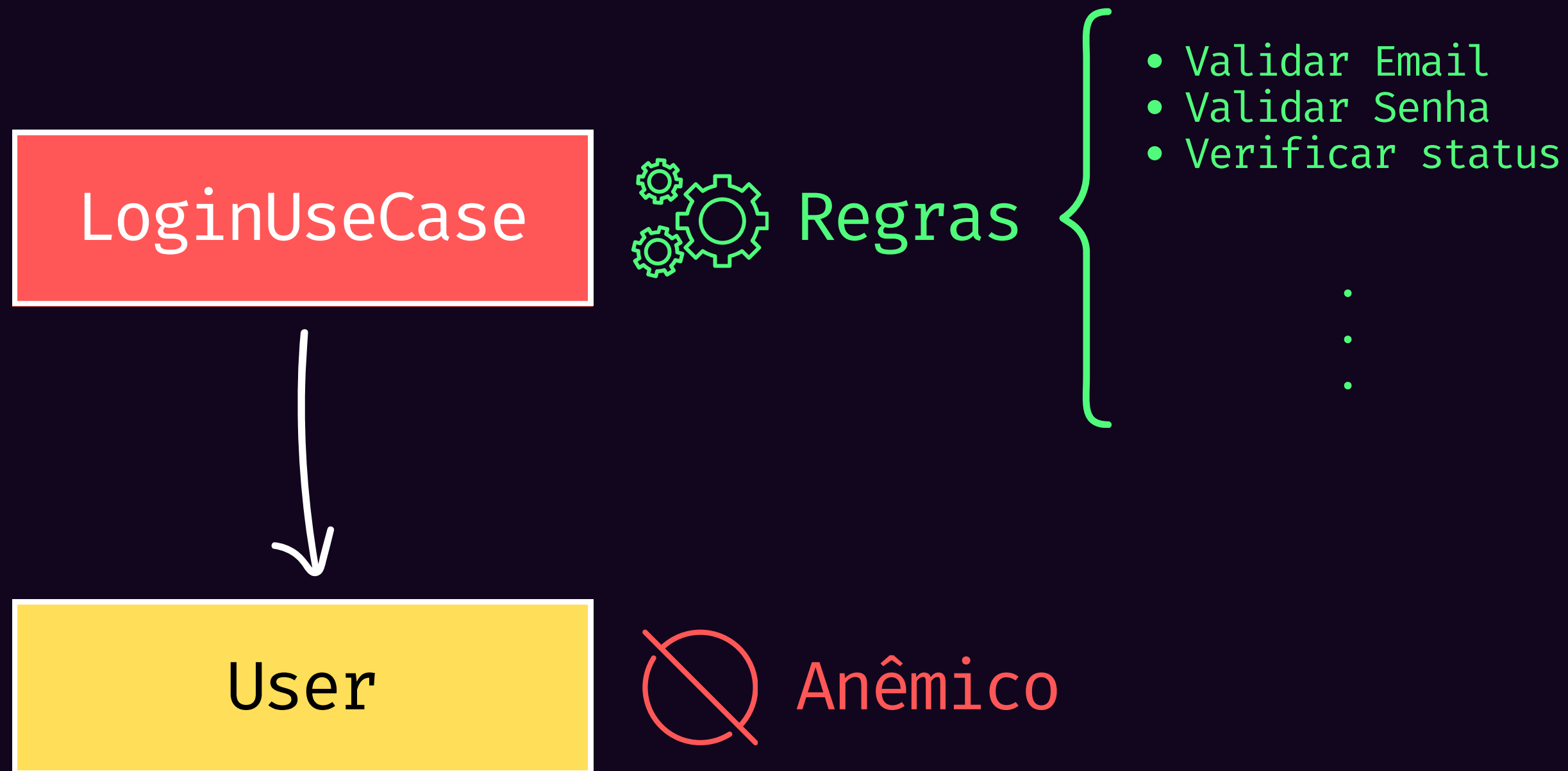


```
export interface User {  
  id: string;  
  name: string;  
  email: string;  
  roles: Role[];  
}
```

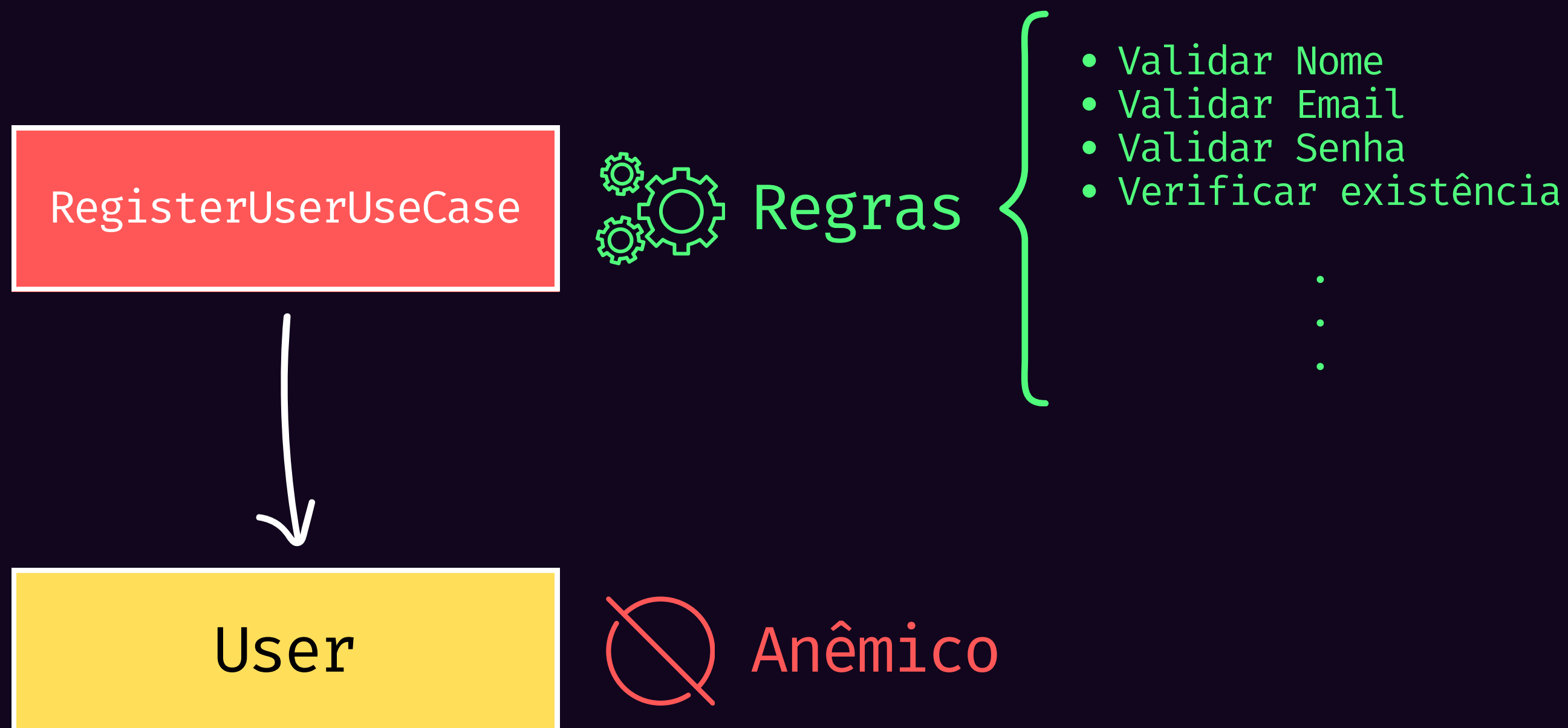
Entidade Anêmica

```
export class User {  
  constructor(  
    private id: string,  
    private name: string,  
    private email: string,  
    private roles: Role[]  
  ) {}  
  
  getId(): string { return this.id; }  
  setId(id: string): void { this.id = id; }  
  getName(): string { return this.name; }  
  setName(name: string): void { this.name = name; }  
  getEmail(): string { return this.email; }  
  setEmail(email: string): void { this.email = email; }  
  getRoles(): Role[] { return this.roles; }  
  setRoles(roles: Role[]): void { this.roles = roles; }  
}
```

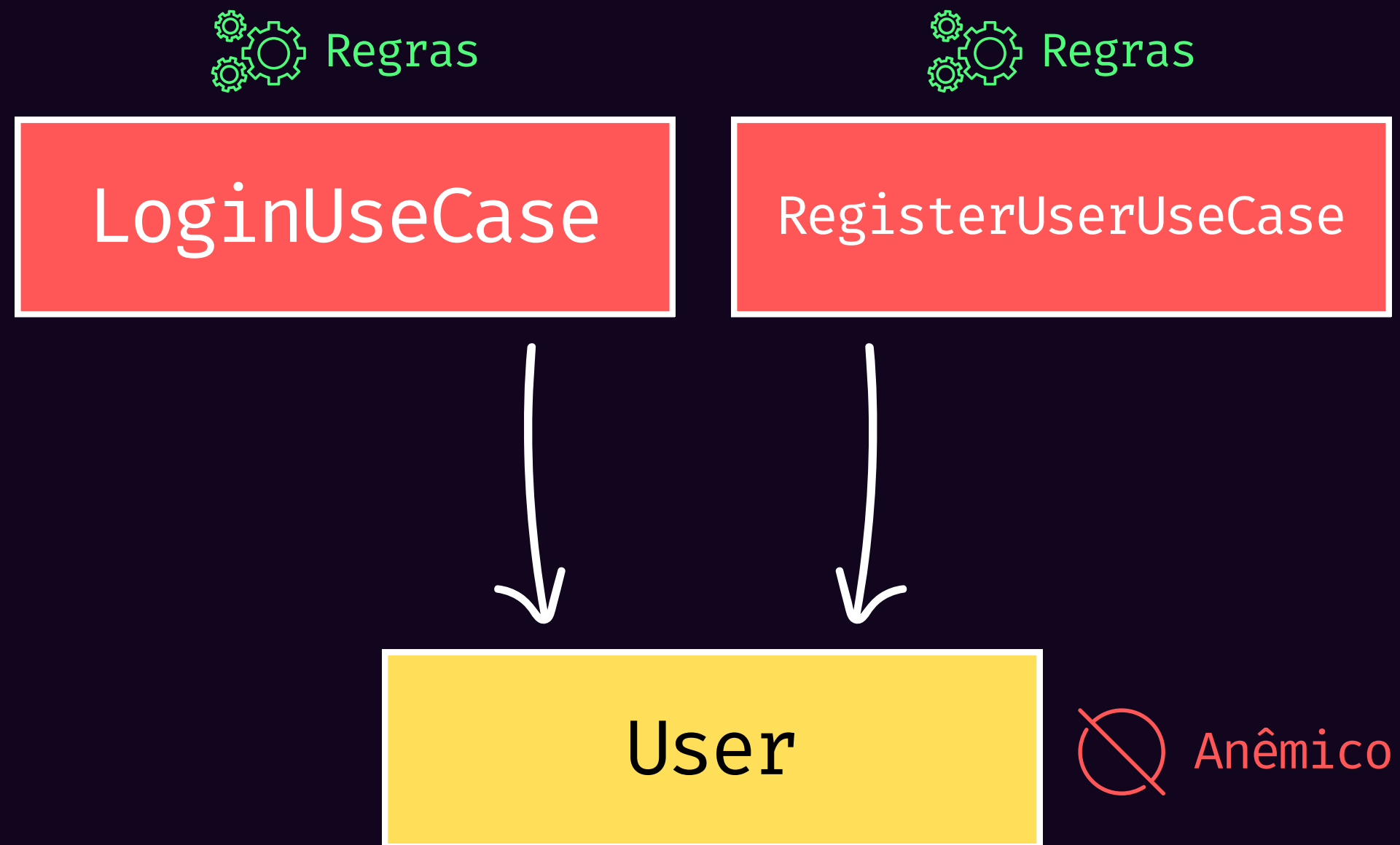
Onde ficam as Regras?



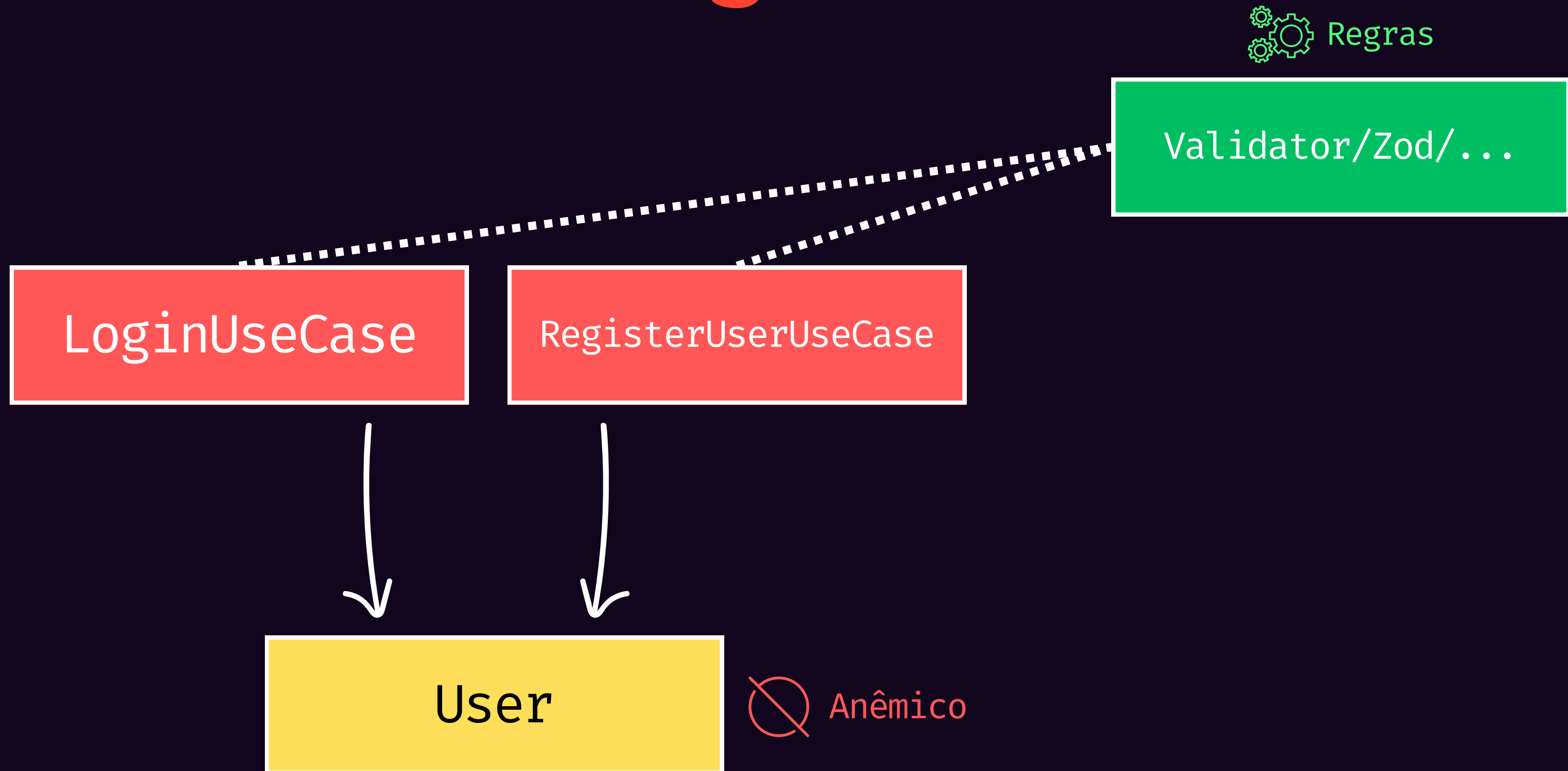
Onde ficam as Regras?



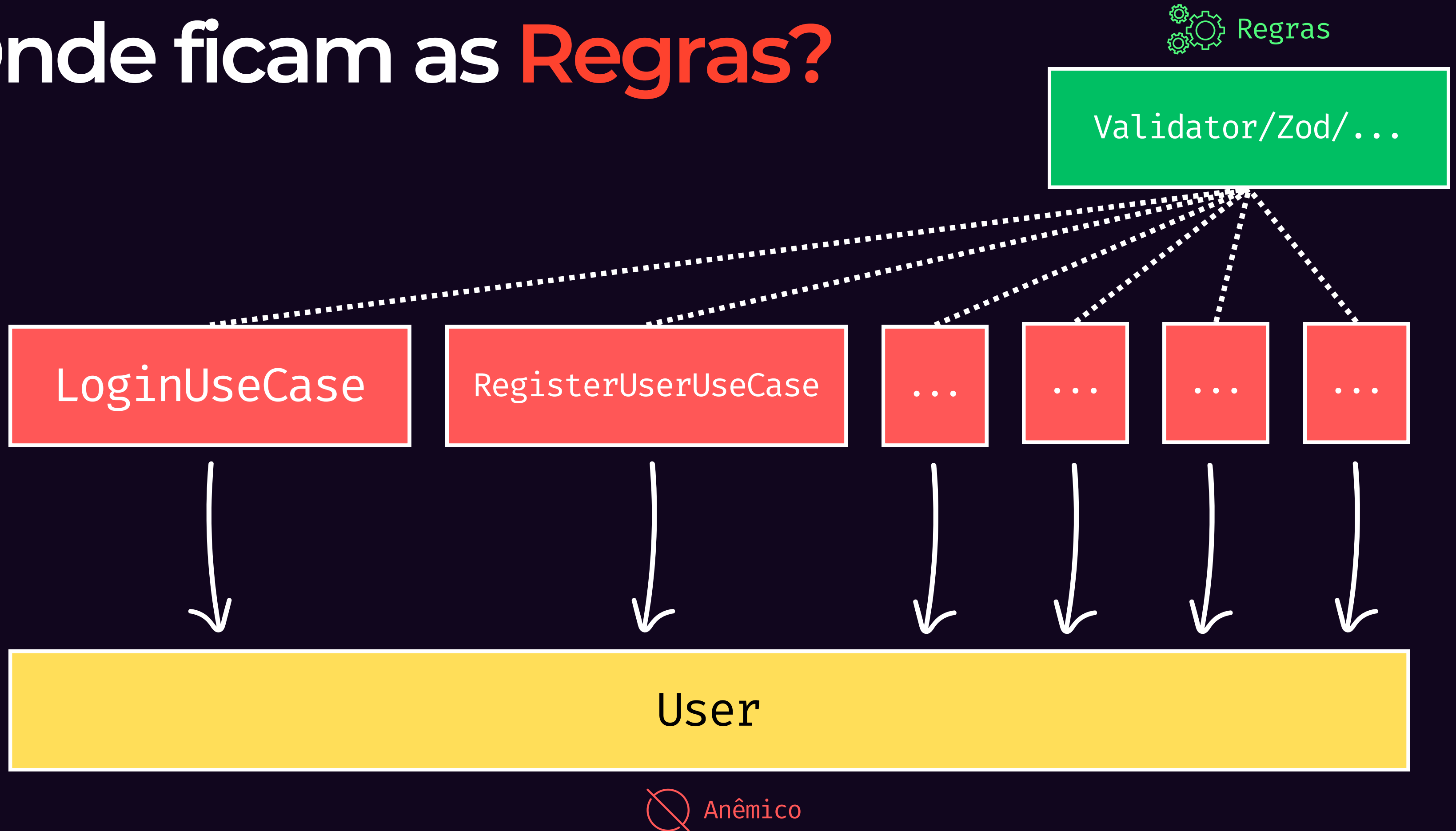
Onde ficam as **Regras**?



Onde ficam as **Regras**?



Onde ficam as **Regras**?



Violação da POO

 Regras

COMPORTAMENTOS

Validator/Zod/...

SEPARAÇÃO

 Anêmico

ATRIBUTOS

User

Ferramentas

Táticas

A modelagem de domínio **representa o núcleo da aplicação**, onde as regras de negócio são expressas por meio de entidades, objetos de valor, serviços e eventos. É nessa camada que o sistema deixa de ser apenas dados e passa a **incorporar comportamentos** e decisões reais do negócio.



Objeto de Valor

Encapsula um valor do domínio e suas regras, garantindo consistência por meio da imutabilidade.



Entidade

Modela elementos do negócio com identidade própria e comportamento ao longo do tempo.



Serviço de Domínio

Encapsula regras de negócio que não pertencem naturalmente a uma única entidade.



Evento de Domínio

Representa fatos importantes que ocorreram no domínio e podem gerar reações no sistema.

Transformando o Modelo



```
export interface User {  
  id: string;  
  name: string;  
  email: string;  
  roles: Role[];  
}
```



```
export interface User {  
  id: UserId;  
  name: PersonName;  
  email: Email;  
  roles: Roles;  
}
```

Transformando o Modelo



```
export interface User {  
  id: string;  
  name: string;  
  email: string;  
  roles: Role[];  
}
```



```
export interface User {  
  id: UserId;  
  name: PersonName;  
  email: Email;  
  roles: Roles;  
}
```

Objeto de Valor



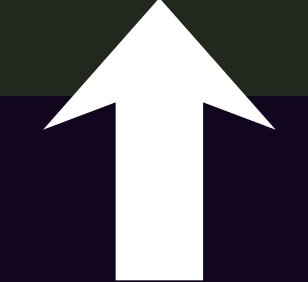
```
export class Email {  
  constructor(public readonly completo: string) {  
    if (!completo.includes("@")) throw new Error("E-mail inválido");  
  }  
  get dominio(): string { return this.completo.split("@")[1]; }  
  get local(): string { return this.completo.split("@")[0]; }  
}
```

Transformando o Modelo



```
const user: User = {  
  id: "1",  
  name: "João",  
  email: "joao.sem.arroba.com",  
  roles: [{name: "admin"}]  
};
```

Anêmico



```
const user: User = {  
  id: new UserId("1"),  
  name: new PersonName("João Silva"),  
  email: new Email("joao.sem.arroba.com"),  
  roles: [new Role("admin")]  
};
```

Rico



Transformando o Modelo

ERRO!!!

```
const user: User = {  
  id: "1",  
  name: "João",  
  email: "joao.sem.arroba.com",  
  roles: [{name: "admin"}]  
};
```

Anêmico

```
const user: User = {  
  id: new UserId("1"),  
  name: new PersonName("João Silva"),  
  email: new Email("joao.sem.arroba.com"),  
  roles: [new Role("admin")]  
};
```

Rico

Transformando o Modelo



```
export interface UserProps {  
  id: string;  
  name: string;  
  email: string;  
  roles: string[];  
}
```



```
export class User {  
  public readonly id: UserId;  
  public readonly name: PersonName;  
  public readonly email: Email;  
  public readonly roles: Roles;  
  
  constructor(public readonly props: UserProps) {  
    this.id = new UserId(props.id);  
    this.name = new PersonName(props.name);  
    this.email = new Email(props.email);  
    this.roles = new Roles(props.roles);  
  }  
}
```

Transformando o Modelo



```
const user = new User({  
  id: "1",  
  name: "João Silva",  
  email: "joao.silva@email.com",  
  roles: ["admin", "user"]  
});
```

Ferramentas Táticas

A modelagem de domínio **representa o núcleo da aplicação**, onde as regras de negócio são expressas por meio de entidades, objetos de valor, serviços e eventos. É nessa camada que o sistema deixa de ser apenas dados e passa a **incorporar comportamentos** e decisões reais do negócio.



Objeto de Valor

Encapsula um valor do domínio e suas regras, garantindo consistência por meio da imutabilidade.



Entidade

Modela elementos do negócio com identidade própria e comportamento ao longo do tempo.



Serviço de Domínio

Encapsula regras de negócio que não pertencem naturalmente a uma única entidade.



Evento de Domínio

Representa fatos importantes que ocorreram no domínio e podem gerar reações no sistema.

Entidades Ricas



```
export class User {
  public readonly id: UserId;
  public readonly name: PersonName;
  public readonly email: Email;
  public readonly roles: Roles;

  constructor(props: UserProps) {
    this.id = new UserId(props.id);
    this.name = new PersonName(props.name);
    this.email = new Email(props.email);
    this.roles = new Roles(props.roles);
  }

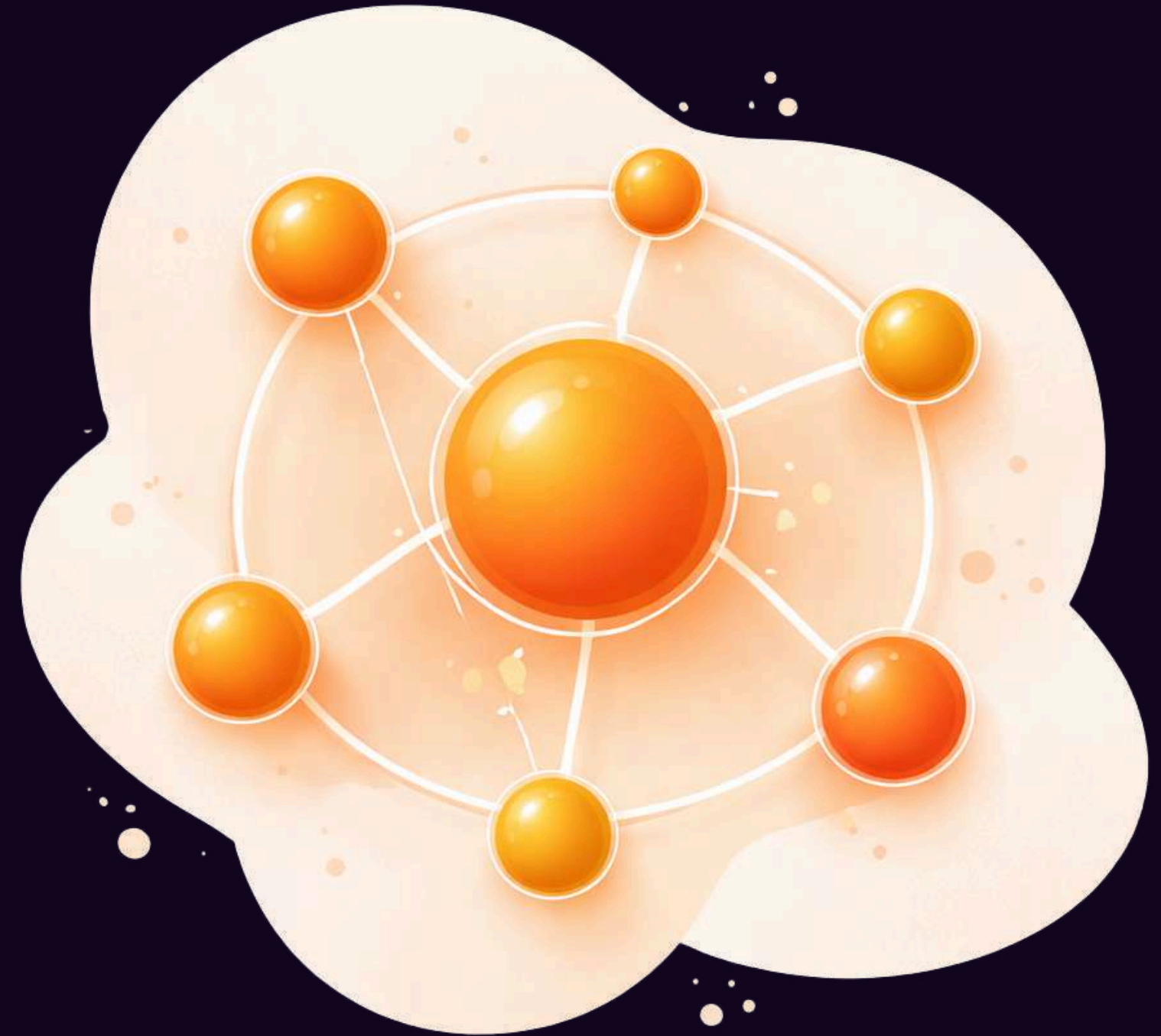
  hasPermissions(...permissions: string[]): boolean {
    return this.roles.hasPermissions(...permissions);
  }
}
```

ABRINDO PARÊNTESES...



Conceito de **Agregado**

Um agregado define um conjunto de objetos do domínio que devem ser tratados como uma única unidade de consistência, protegendo regras e invariantes do negócio.

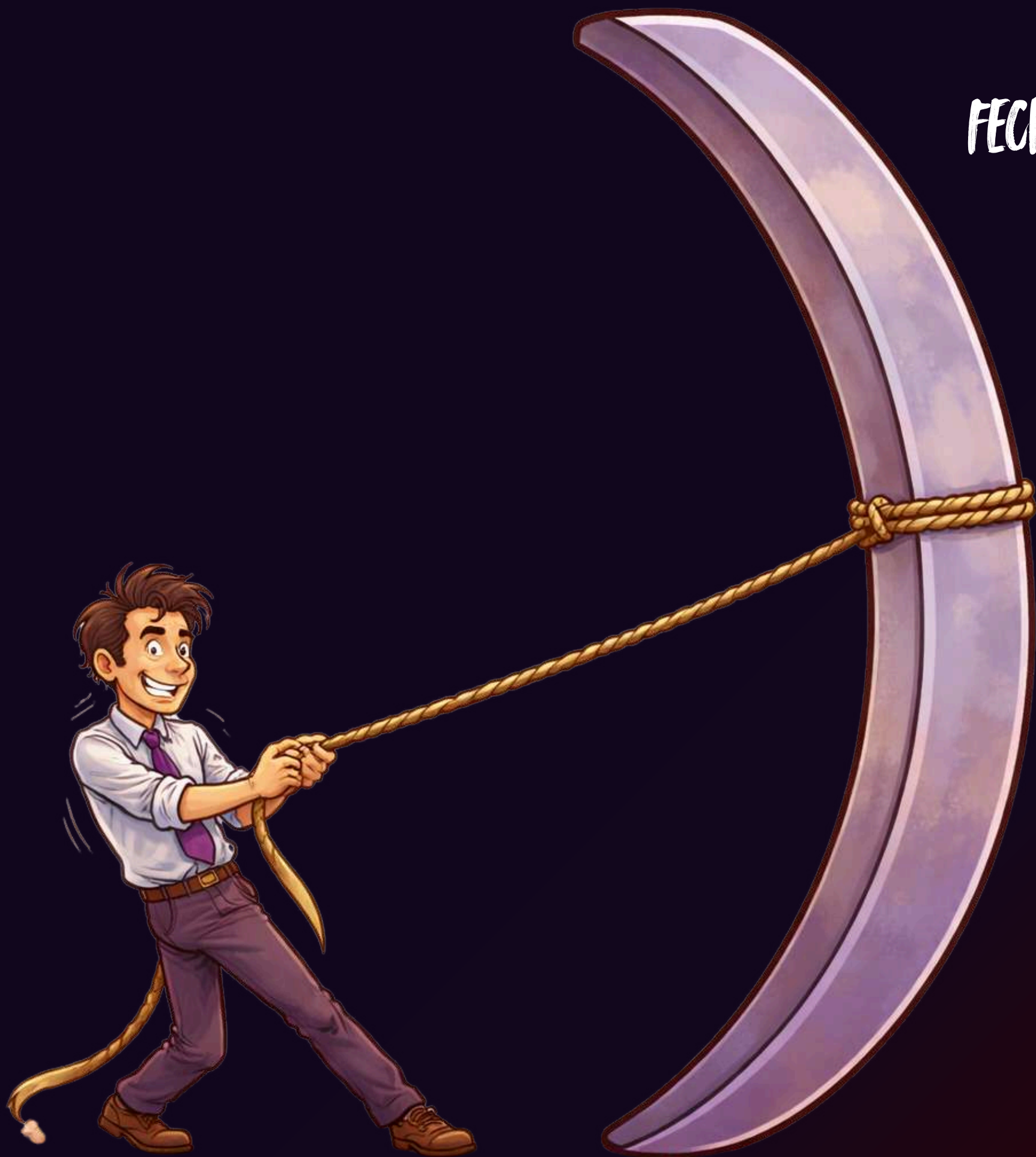


Conceito de Agregado



```
export class Pedido {  
  // ...  
  private readonly itens: ItemPedido[] = [];  
  
  constructor(public readonly props: PedidoProps) {}  
  
  // ...  
}
```

FECHANDO PARÊNTESES.



Ferramentas Táticas

A modelagem de domínio **representa o núcleo da aplicação**, onde as regras de negócio são expressas por meio de entidades, objetos de valor, serviços e eventos. É nessa camada que o sistema deixa de ser apenas dados e passa a **incorporar comportamentos** e decisões reais do negócio.



Objeto de Valor

Encapsula um valor do domínio e suas regras, garantindo consistência por meio da imutabilidade.



Entidade

Modela elementos do negócio com identidade própria e comportamento ao longo do tempo.



Serviço de Domínio

Encapsula regras de negócio que não pertencem naturalmente a uma única entidade.



Evento de Domínio

Representa fatos importantes que ocorreram no domínio e podem gerar reações no sistema.

Serviço de Domínio



```
export class PermissionSearchService {  
  filter(perms: Permission[], query: string): Permission[] {  
    const terms = query.toLowerCase().trim().split(/\s+/).filter(Boolean);  
    if (!terms.length) return perms;  
    return perms.filter(p => {  
      const hay = `${p.name} ${p.description}`.toLowerCase();  
      return terms.every(t => hay.includes(t));  
    });  
  }  
}
```

Ferramentas

Táticas

A modelagem de domínio **representa o núcleo da aplicação**, onde as regras de negócio são expressas por meio de entidades, objetos de valor, serviços e eventos. É nessa camada que o sistema deixa de ser apenas dados e passa a **incorporar comportamentos** e decisões reais do negócio.



Objeto de Valor

Encapsula um valor do domínio e suas regras, garantindo consistência por meio da imutabilidade.



Entidade

Modela elementos do negócio com identidade própria e comportamento ao longo do tempo.



Serviço de Domínio

Encapsula regras de negócio que não pertencem naturalmente a uma única entidade.



Evento de Domínio

Representa fatos importantes que ocorreram no domínio e podem gerar reações no sistema.

Evento de Domínio



```
export class RoleAssignedToUser implements DomainEvent {  
  readonly type = "RoleAssignedToUser";  
  readonly occurredAt = new Date();  
  
  constructor(  
    public readonly userId: string,  
    public readonly roleId: string,  
    public readonly assignedByUserId: string  
  ) {}  
}
```

Evento de Domínio



```
export abstract class AggregateRoot {  
  private events: DomainEvent[] = [];  
  
  protected addEvent(event: DomainEvent): void {  
    this.events.push(event);  
  }  
  
  pullEvents(): DomainEvent[] {  
    const e = this.events;  
    this.events = [];  
    return e;  
  }  
}
```

Evento de Domínio

```
export class User extends AggregateRoot {  
  constructor(  
    public readonly id: string,  
    private roleIds: Set<string> = new Set()  
  ) {  
    super();  
  }  
  
  assignRole(roleId: string, assignedByUserId: string): void {  
    if (this.roleIds.has(roleId)) return;  
    this.roleIds.add(roleId);  
    this.addEvent(new RoleAssignedToUser(  
      this.id,  
      roleId,  
      assignedByUserId  
    ));  
  }  
}
```

Padrão Outbox

Garante a publicação confiável de eventos ao persistir mudanças de estado e mensagens no mesmo contexto transacional.



Evento de Domínio

```
export class AssignRoleToUser {
  constructor(
    private readonly users: UserRepository,
    private readonly outbox: OutboxRepository,
    private readonly transaction: TransactionManager
  ) {}

  async execute(input: {
    userId: string;
    roleId: string;
    assignedByUserId: string;
  }): Promise<void> {

    await this.transaction.run(async () => {
      const user = await this.users.getById(input.userId);

      user.assignRole(input.roleId, input.assignedByUserId);

      await this.users.save(user);

      const events = user.pullEvents();
      await this.outbox.appendAll(events);
    });
  }
}
```

Evento de Domínio

```
@Controller("internal/outbox")
export class OutboxDispatcherController {

  constructor(
    private readonly outbox: OutboxRepository,
    private readonly kafka: KafkaService
  ) {}

  @Cron(CronExpression.EVERY_10_SECONDS)
  async dispatch(): Promise<void> {
    const batchSize = 50;

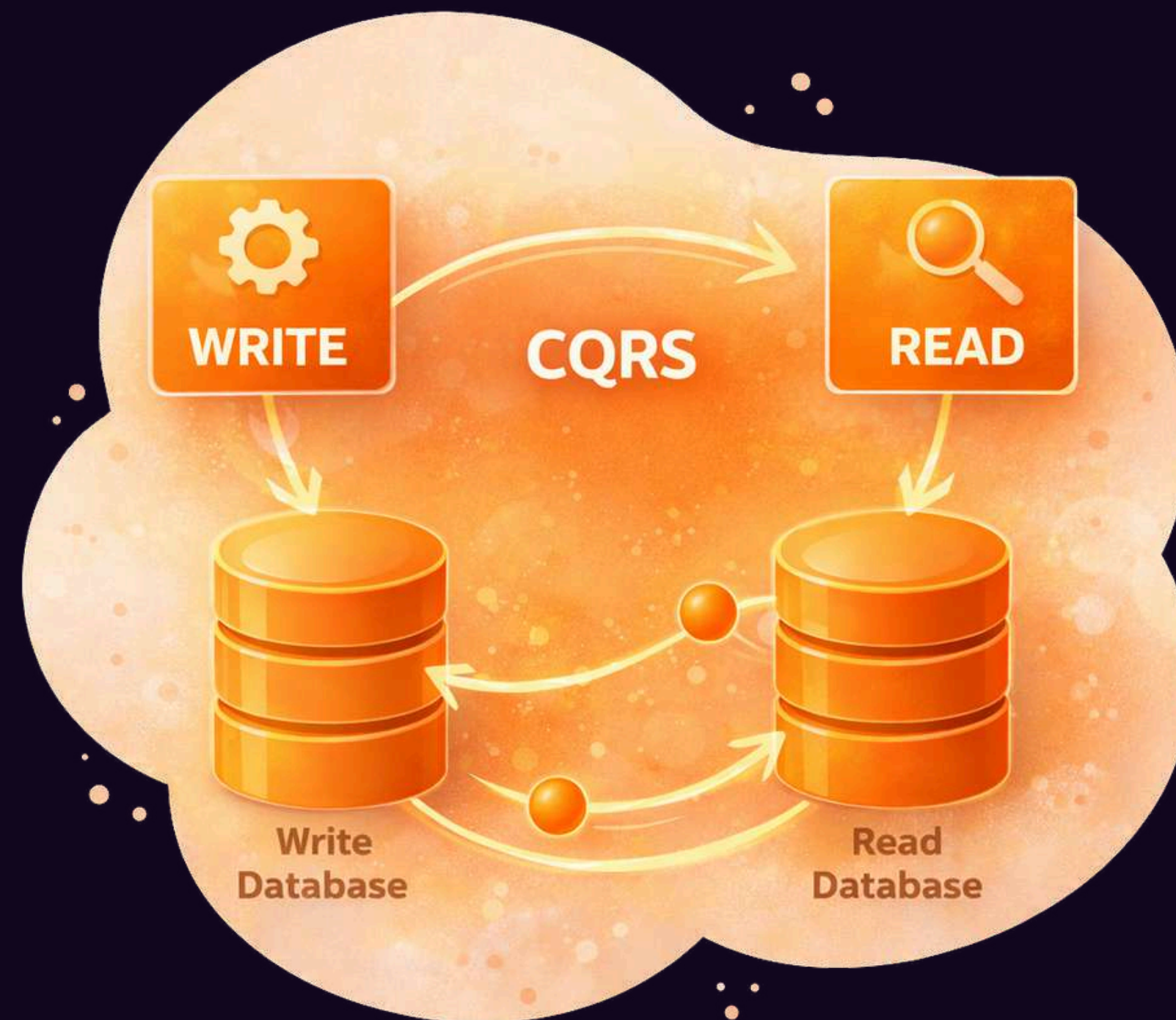
    const messages = await this.outbox.claimPending(batchSize);
    if (!messages.length) return;

    for (const msg of messages) {
      await this.kafka.publish(msg.topic, {
        key: msg.id,
        type: msg.eventType,
        payload: msg.payload
      });

      await this.outbox.markPublished(msg.id);
    }
  }
}
```

Padrão CQRS

Command Query Responsibility
Segregation divide comandos e consultas,
separando mudanças de estado da leitura
de dados.

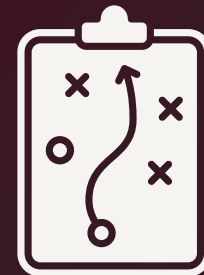


Roadmap do Desenvolvimento



ESTRATÉGICO

Compreender o domínio do negócio e organizar a aplicação em contextos delimitados e módulos bem definidos.



TÁTICO

Aplicar padrões de modelagem e arquitetura para transformar o domínio em soluções consistentes e sustentáveis.



OPERACIONAL

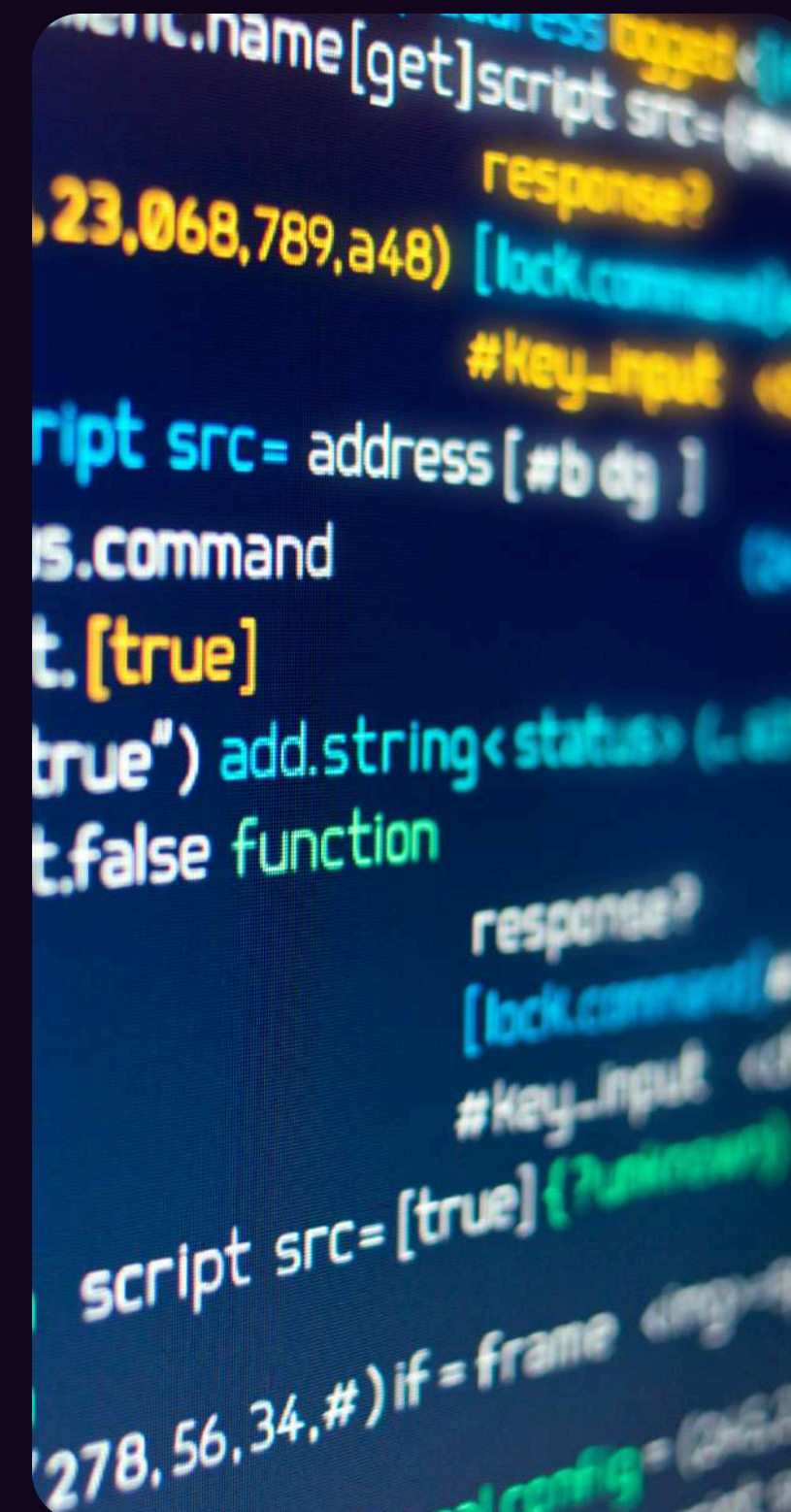
Implementar o sistema em código, integrando frameworks e bibliotecas de mercado sem comprometer as regras de negócio.

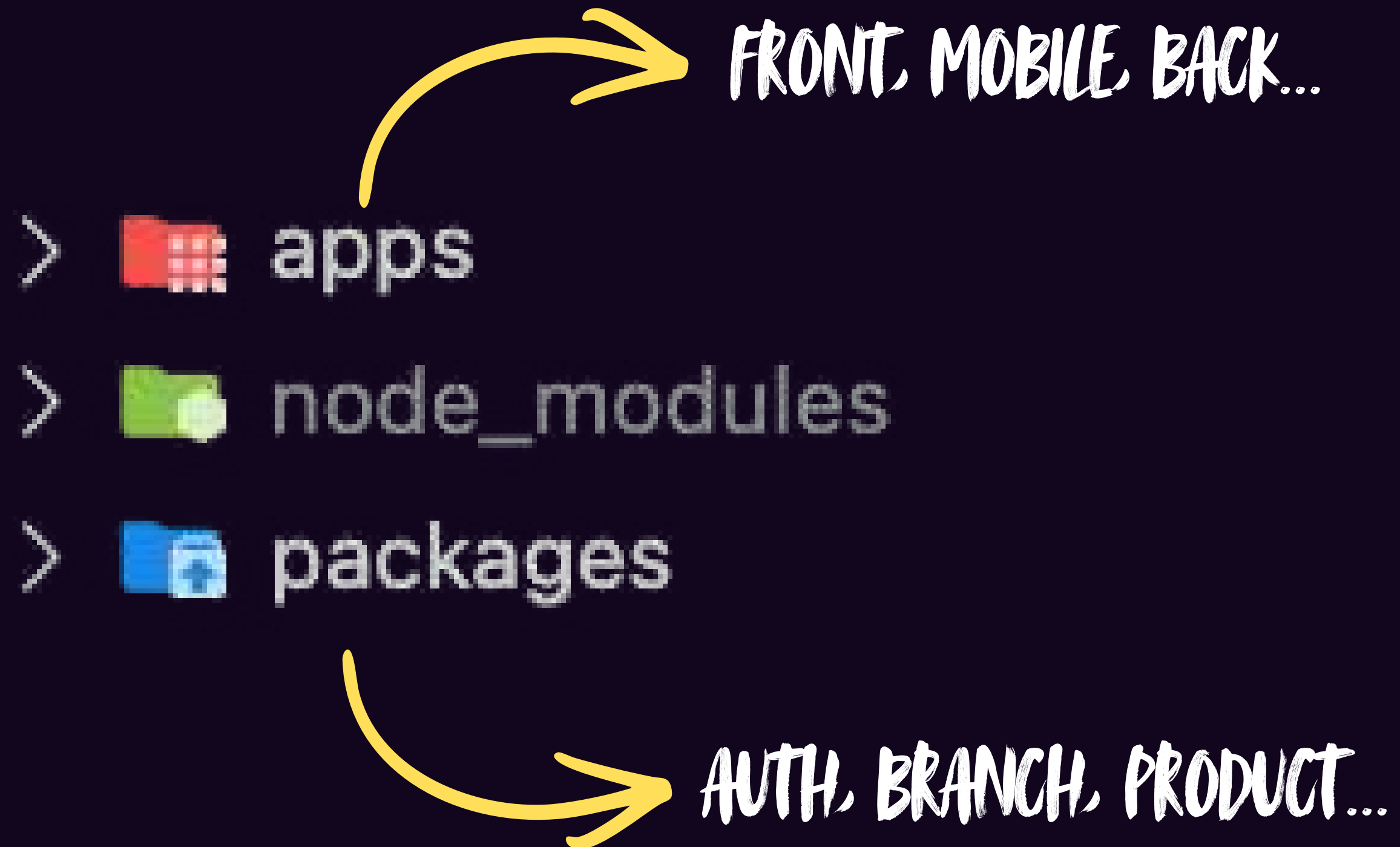
"Módulo" Shared

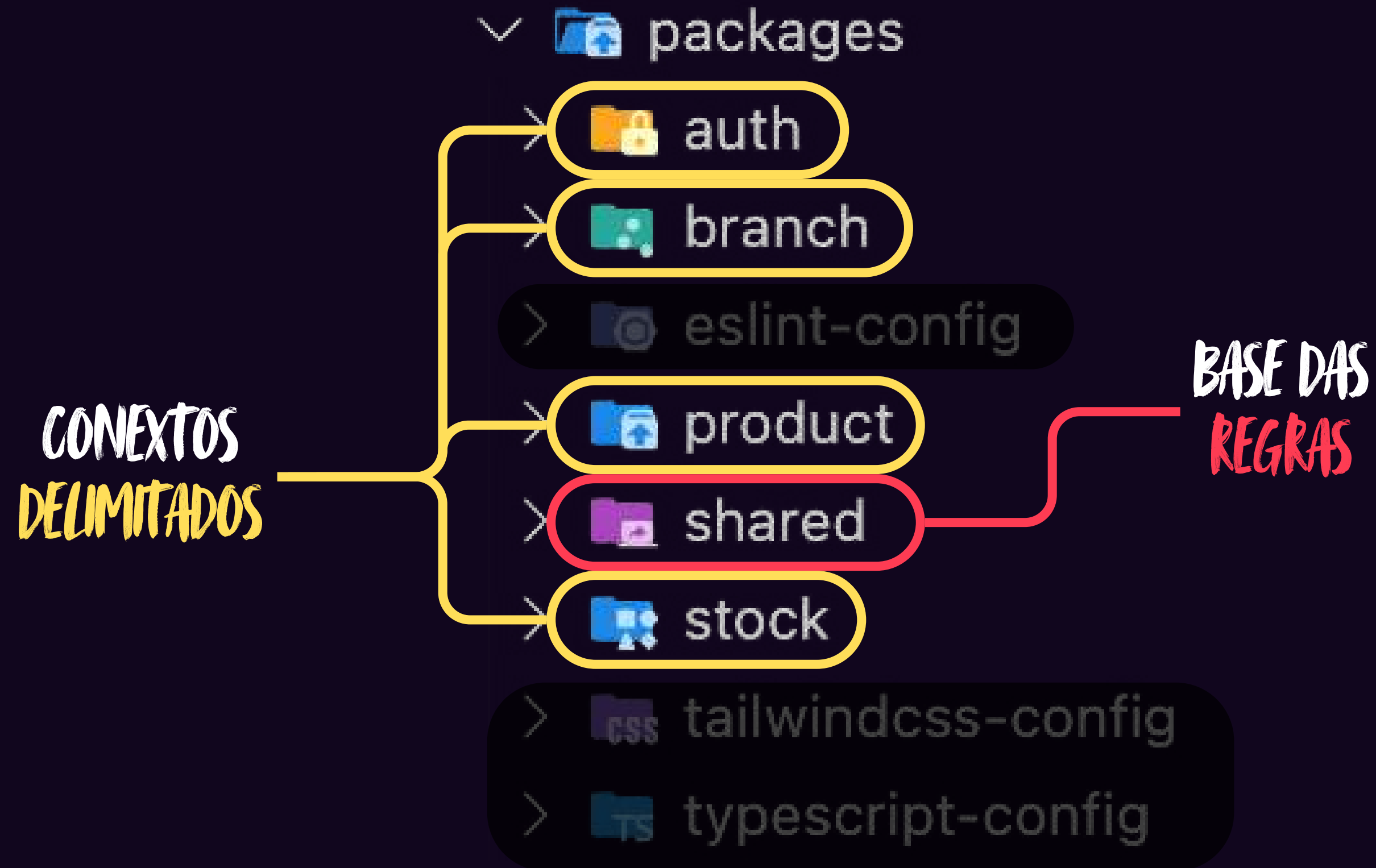
Centralizar e padronizar os elementos fundamentais da aplicação, como classes base, contratos, casos de uso e objetos de valor.

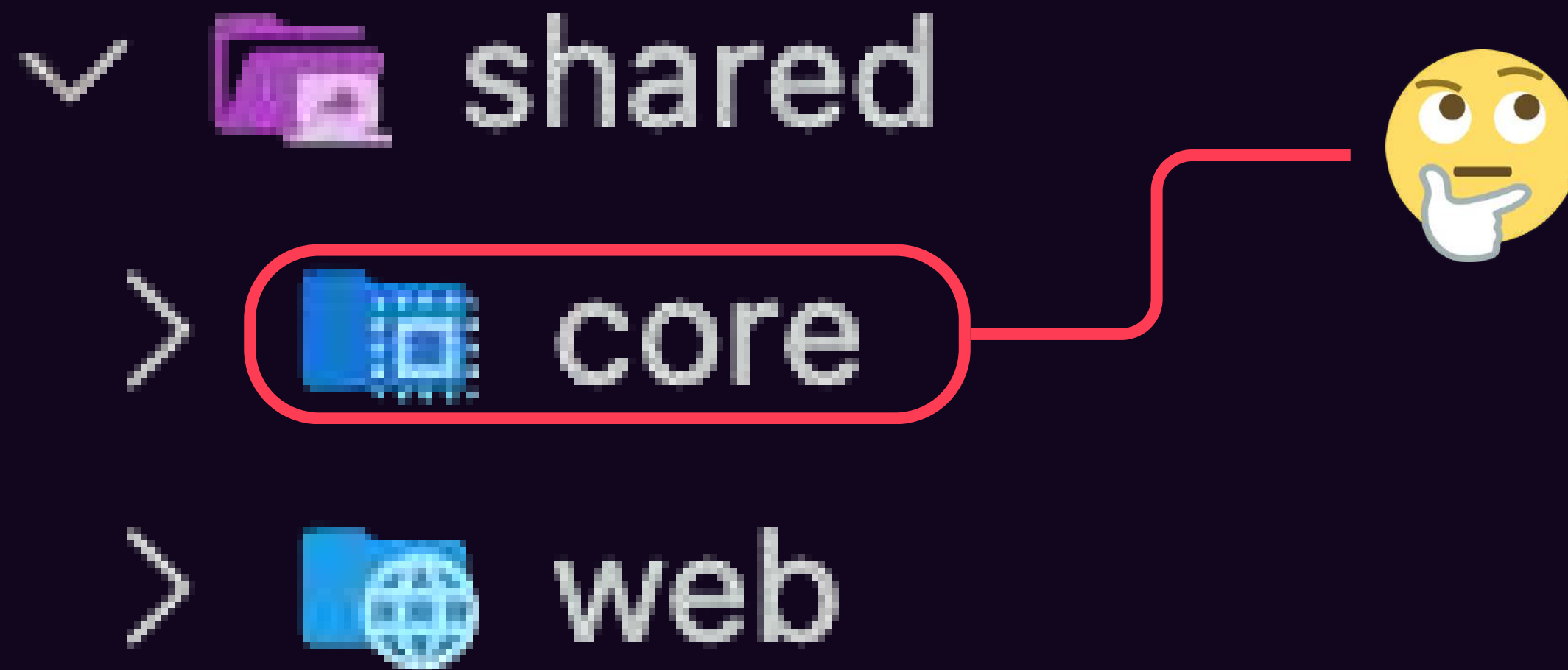


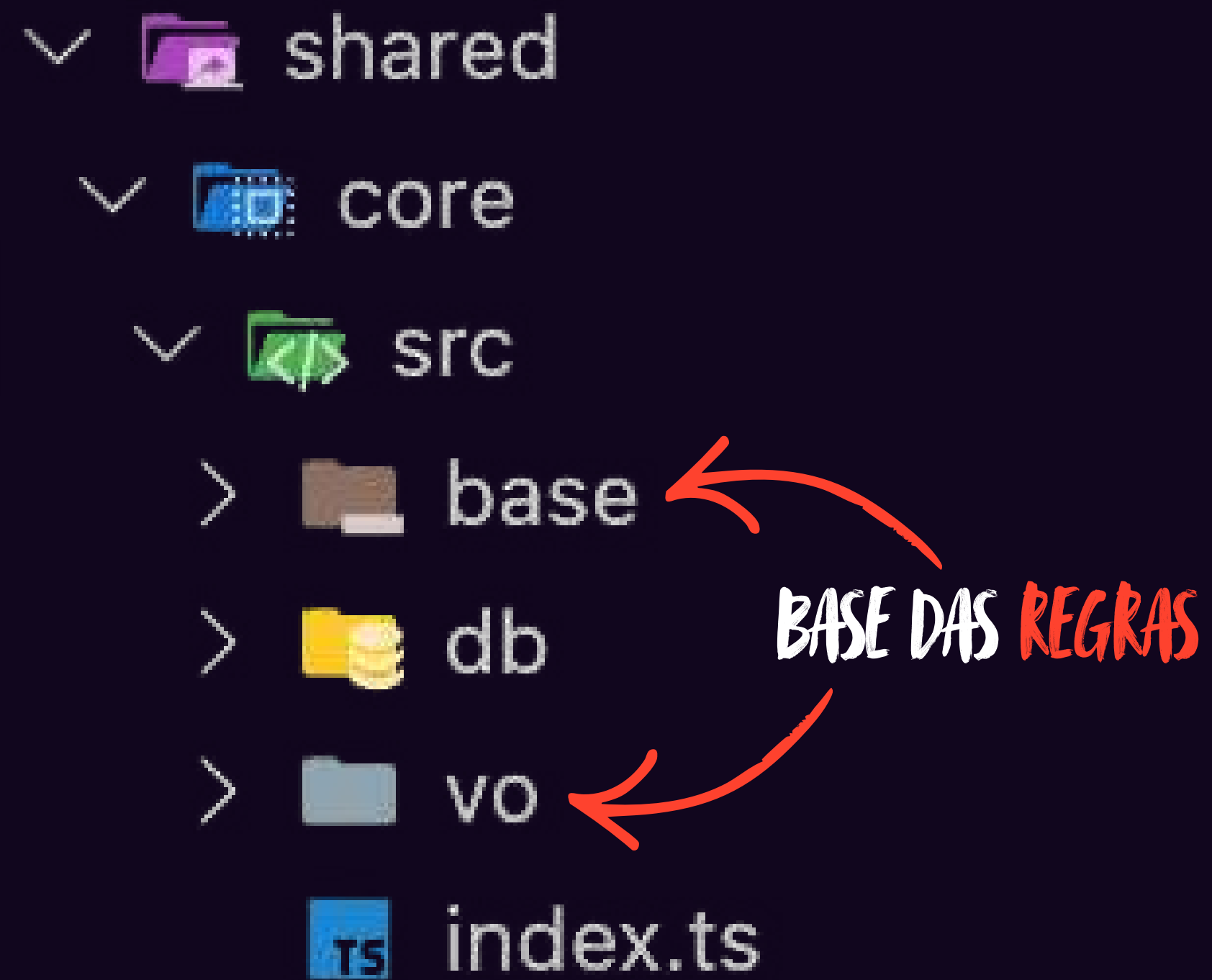
Entidades, Objetos de Valor e Contratos

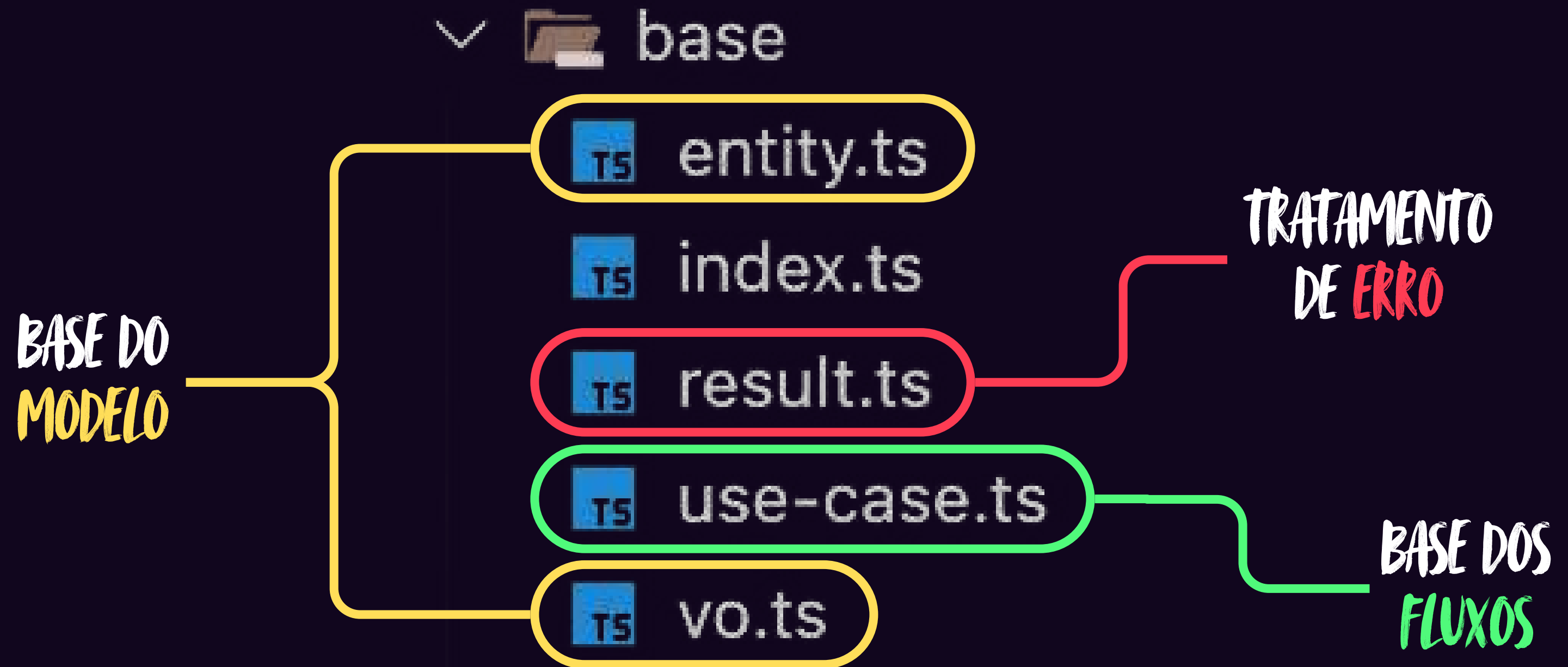


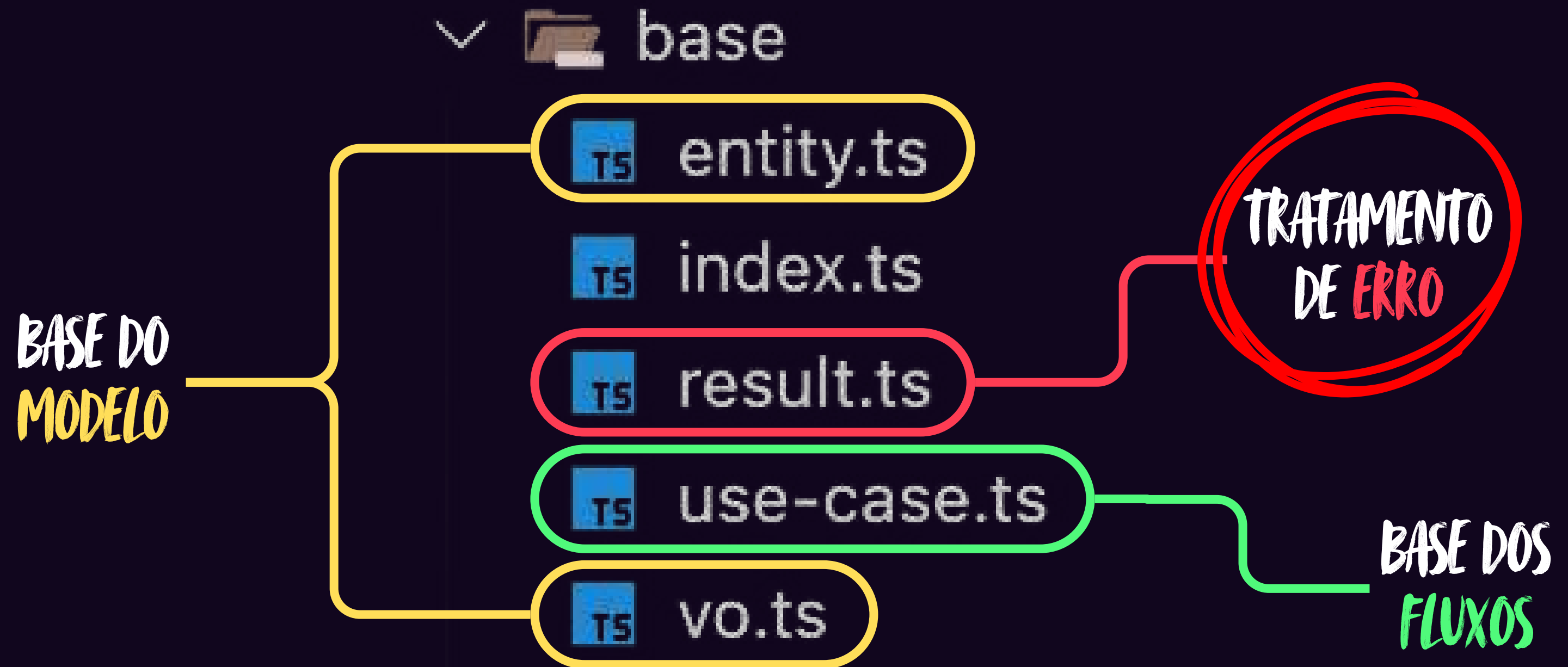












Tratamento de Erro

```
let email: string;
let password: string;

try {
  email = Email.create(input.email);
} catch (e) {
  return { ok: false, errors: ["EMAIL_INVALID"] };
}

try {
  password = Password.create(input.password);
} catch (e) {
  return { ok: false, errors: ["PASSWORD_WEAK"] };
}

return { ok: true, data: { email, password } };
```

Tratamento de Erro

CHATO

CHATO

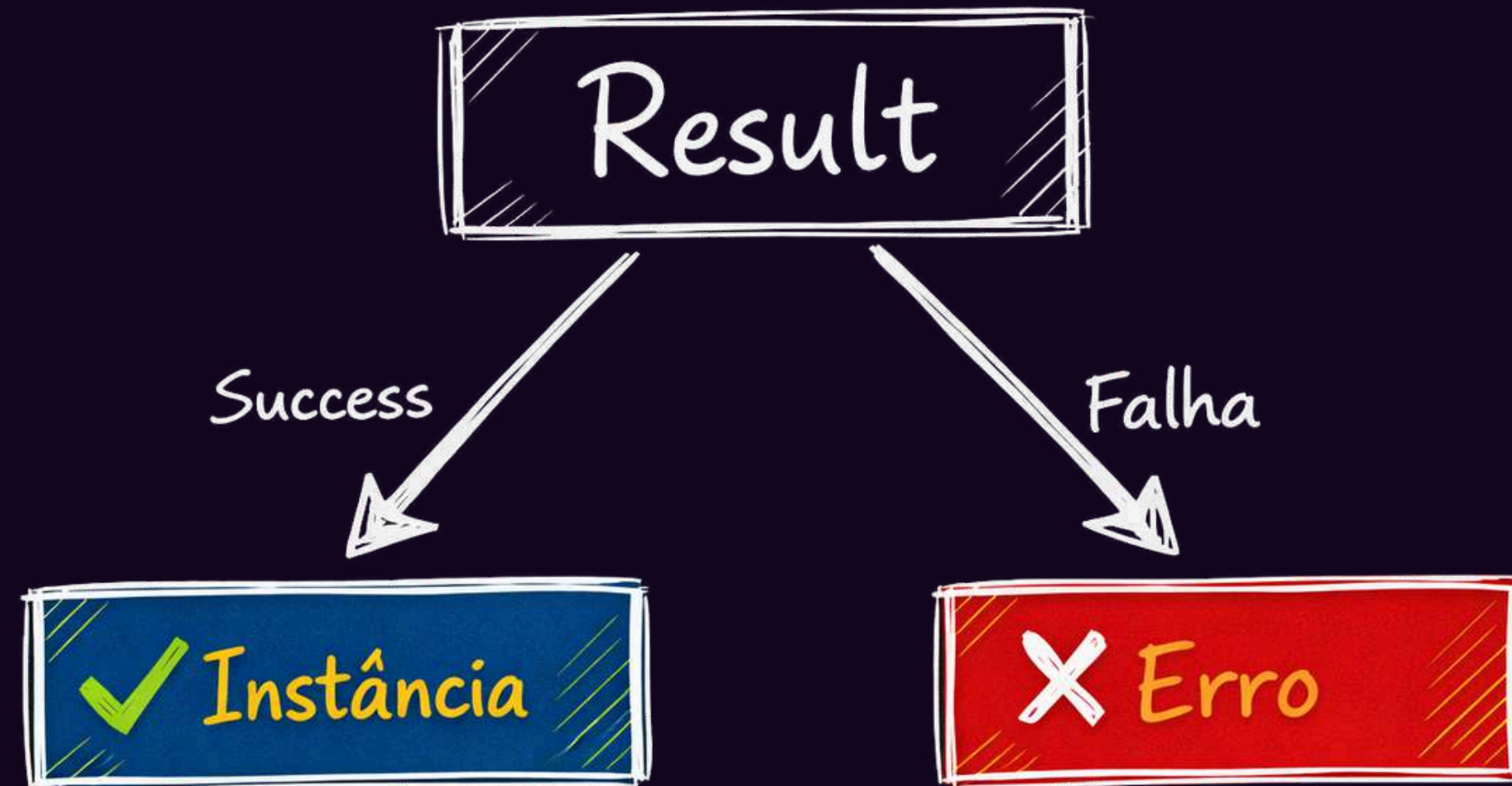
```
let email: string;
let password: string;

try {
  email = Email.create(input.email);
} catch (e) {
  return { ok: false, errors: ["EMAIL_INVALID"] };
}

try {
  password = Password.create(input.password);
} catch (e) {
  return { ok: false, errors: ["PASSWORD_WEAK"] };
}

return { ok: true, data: { email, password } };
```

Tratamento de Erro



Tratamento de Erro



```
const tryEmail = Email.tryCreate(input.email);  
const tryPass  = Password.tryCreate(input.password);  
  
const combined = Result.combine([tryEmail, tryPass]);  
if (combined.isFailure) return combined;  
  
const [email, password] = combined.instance;  
return Result.ok({ email, password });
```

Tratamento de Erro

PARA NO PRIMEIRO ERRO



```
try {  
    Email.create(input.email);  
    Password.create(input.password);  
    Name.create(input.name);  
    return { ok: true };  
} catch (e) {  
    return { ok: false, errors: [String(e)] };  
}
```

Tratamento de Erro

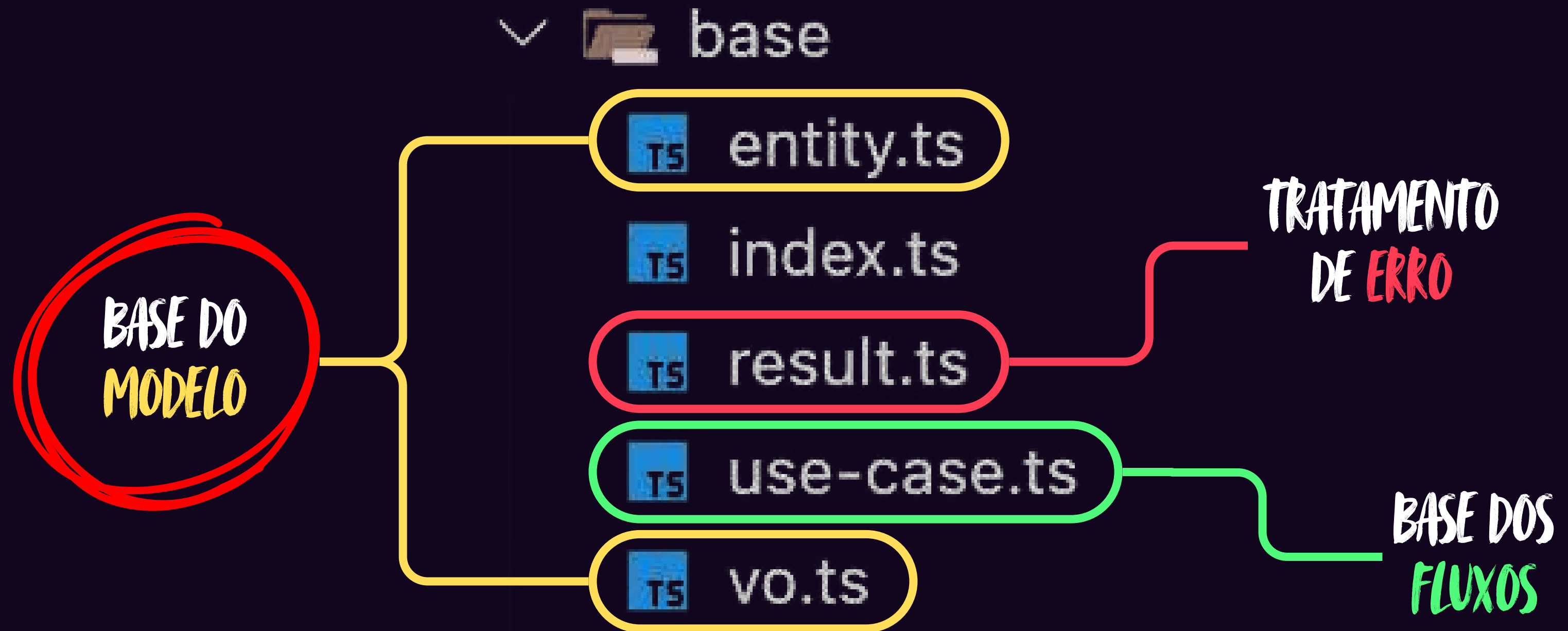


```
const r = Result.combine([
  Email.tryCreate(input.email),
  Password.tryCreate(input.password),
  Name.tryCreate(input.name),
]);

if (r.isFailure) return r;
return Result.ok();
```

Tratamento de Erro

```
export class Result {  
  
    static ok(instance?) {}  
    static fail(errors) {}  
  
    static combine(results) {}  
    static combineAsync(results) {}  
  
    get isOk() {}  
    get isFailure() {}  
  
    get instance() {}  
    get errors() {}  
  
    throwIfFailed() {}  
}
```



Objeto de Valor



```
export interface ValueObjectConfig {}

export abstract class ValueObject<T, Config> extends ValueObjectConfig {
    constructor(
        readonly value: T,
        readonly config?: Config,
    ) {}

    equals(vo: ValueObject<T, Config>): boolean {
        return this.value === vo.value;
    }

    notEquals(vo: ValueObject<T, Config>): boolean {
        return !this.equals(vo);
    }
}
```

Objeto de Valor



```
export class Email extends ValueObject {  
  static create(value, config?) {}  
  static tryCreate(value, config?) {}  
}
```



```
export class Name extends ValueObject {  
  static create(value, config?) {}  
  static tryCreate(value, config?) {}  
}
```



```
export class Cpf extends ValueObject {  
  static create(value, config?) {}  
  static tryCreate(value, config?) {}  
}
```



```
export class Cnpj extends ValueObject {  
  static create(value, config?) {}  
  static tryCreate(value, config?) {}  
}
```

Agents Skills

Skills



Codex



Search

Skills

Controller

Criar controllers Nest no padrão do projeto

pharmacore

DTO

Criar DTOs no padrão do projeto

pharmacore

Domain Service

Criar serviços de domínio em core

pharmacore

Entity

Criar entidades no padrão Pharmacore

pharmacore

Frontend Form Schema

Criar forms e schemas no padrão web

pharmacore

Prisma

Modelar e manter Prisma no padrão do projeto

pharmacore

Query CQRS

Criar queries de leitura no padrão CQRS

pharmacore

Repository

Criar repositórios no padrão Pharmacore

pharmacore

Skill Creator

Create or update a skill

System

Ask Codex anything, @ to add files, / for commands

+

GPT-5.3-Codex

▼

High

▼

Agents Skills

Skills



Codex



 Value Object
de DV

Criar um CNPJ com validações



Let's build
pharmacore ▾

Agents Skills

Skills



- ✓ .agents / skills
 - > ⚙ controller
 - > 📁 domain-service
 - > 📁 dto
 - > 📁 entity
 - > 📁 frontend-form-schema
 - > 📁 prisma
 - > 📁 query-cqrs
 - > 📁 repository
 - > 📁 use-case
 - > 📁 value-object

Entidade Base

POSSUI ID



```
export interface EntityProps {  
  id?: string;  
  createdAt?: Date | null;  
  updatedAt?: Date | null;  
  deletedAt?: Date | null;  
}  
  
export abstract class Entity<Type, Props extends EntityProps> {  
  readonly props: Props;  
  readonly id: string;  
  
  protected constructor(props: Props) {}  
  
  get createdAt() {}  
  get updatedAt() {}  
  get deletedAt() {}  
  
  equals(entity: Entity<Type, Props>): boolean {}  
  
  notEquals(entity: Entity<Type, Props>): boolean {}  
  
  public cloneWith(overrides: Partial<Props>): Result<Type> {}  
}
```

Casos de Uso



```
import { Result } from "../result";

export interface UseCase<IN, OUT> {
  execute(data: IN): Promise<Result<OUT>>;
}
```

Casos de Uso

O caso de uso depende apenas de **interfaces** (portas). As implementações reais ficam nas camadas externas, através de adaptadores.

```
export interface LoginIn { email: string; password: string }
export interface LoginOut extends UserProps {}

export class LoginUseCase implements UseCase<LoginIn, LoginOut> {
  constructor(
    private repo: UserRepository,
    private findPassHash: FindPasswordHashQuery,
    private passwordProvider: PasswordProvider,
  ) {}

  async execute(input: LoginIn): Promise<Result<LoginOut>> {
    // Implementação omitida
    return Result.ok(userResult.instance.props);
  }
}
```

Casos de Uso

```
export interface LoginIn {  
  email: string;  
  password: string;  
}
```

```
export interface LoginOut  
  extends UserProps {}
```

```
async execute(input: LoginIn): Promise<Result<LoginOut>> {  
  // Implementação omitida  
  return Result.ok(userResult.instance.props);  
}
```

Casos de Uso



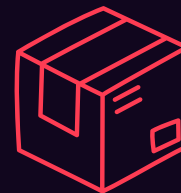
```
export class LoginUseCase implements UseCase<LoginIn, LoginOut> {  
  constructor(  
    private repo: UserRepository,  
    private findPassHash: FindPasswordHashQuery,  
    private passwordProvider: PasswordProvider,  
  ) {}  
  
  async execute(input: LoginIn): Promise<Result<LoginOut>> {  
    // Implementação omitida  
    return Result.ok(userResult.instance.baseProps);  
  }  
}
```

Persistência de Dados



Entidade

Modela elementos do negócio com identidade própria e comportamento ao longo do tempo.



Objeto de Valor

Encapsula um valor do domínio e suas regras, garantindo consistência por meio da imutabilidade.

Persistência de Dados

*** O modelo de domínio é **independente** da persistência. Entidades e VOs não são orientados a banco — o mapeamento para tabelas é responsabilidade dos adaptadores.

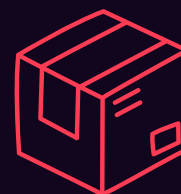
TABELA



Entidade

Modela elementos do negócio com identidade própria e comportamento ao longo do tempo.

COLUNA



Objeto de Valor

Encapsula um valor do domínio e suas regras, garantindo consistência por meio da imutabilidade.

Command

Responsibility

CQRS

Query

Segregation

Command

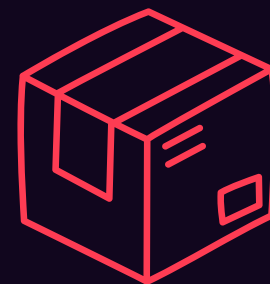
(ORM)



Agregados



Entidade



Objeto de Valor

Query

(SQL)



DTOs

Exemplo de Comando



```
const createOrUpdateUserCommand = {  
  user: {  
    id: "9a44d199-fe21-41bd-a507-b2d6c00461b3",  
    name: "Ana Lima",  
    email: "ana@acme.com",  
    status: "ACTIVE",  
    updatedAt: "2026-02-26T14:12:00Z",  
  
    // Relação com algo FORA do agregado -> só referência (IDs)  
    profileIds: ["admin", "sales"]  
  }  
};
```

Exemplo de Comando



```
const deleteUserCommand = {  
  userId: "9a44d199-fe21-41bd-a507-b2d6c00461b3"  
};
```



```
const addProfileToUserCommand = {  
  userId: "9a44d199-fe21-41bd-a507-b2d6c00461b3",  
  profileId: "support"  
};
```

Exemplo de Query



```
const userDetailsQueryResult = {
  id: "usr_123",
  name: "Ana Lima",
  email: "ana@acme.com",
  status: "ACTIVE",

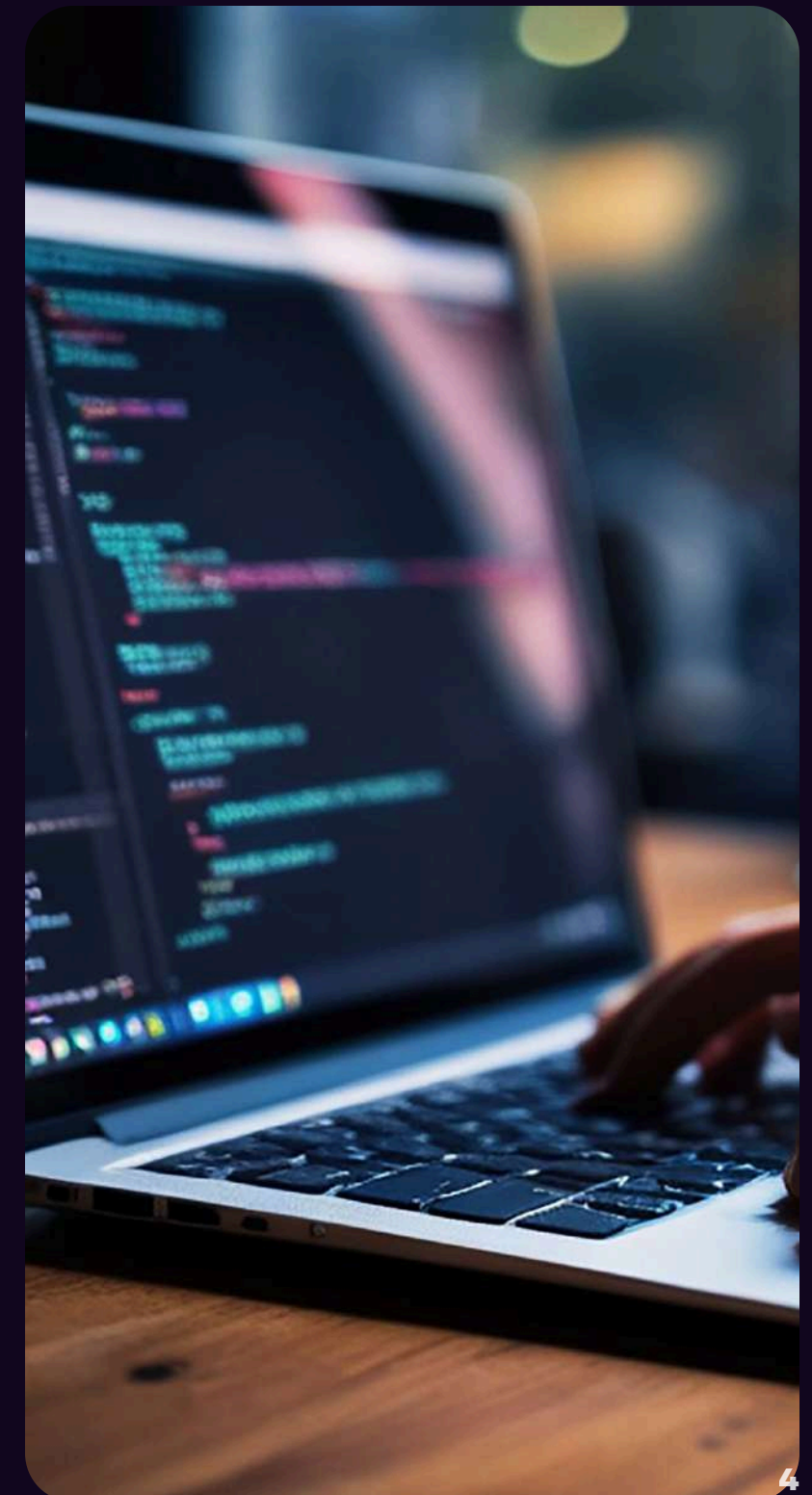
  profiles: [
    {
      id: "admin",
      name: "Admin",
      description: "Acesso total ao sistema",
      permissions: [
        { id: "perm_users_read", key: "users:read", label: "Ler usuários" },
        { id: "perm_users_write", key: "users:write", label: "Editar usuários" }
      ]
    }
  ]
};
```

Módulo de Autenticação

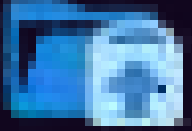
Gerenciar usuários, perfis e permissões, controlando autenticação, autorização e acesso seguro aos recursos da aplicação.



Usuários, Perfis e
Permissões

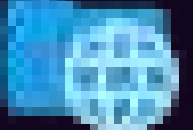


CONTEXTO DELIMITADO

✓  packages

✓  auth

>  core

>  web

CASOS DE USO É MODELO

COMPONENTES, HOOKS, PÁGINAS...

Modelo: Agregados

O módulo de autenticação é composto por quatro agregados — **usuário**, **senha**, **perfil** e **permissão** — cada um com uma única entidade. Usuários, senhas e perfis são gerenciados dinamicamente pela aplicação, enquanto as permissões formam um conjunto fixo que define as ações disponíveis no sistema.



User

Define a identidade do usuário, seus dados de acesso e os perfis que determinam suas permissões na aplicação.



Password

Gerencia a credencial do usuário de forma isolada, garantindo segurança, histórico e controle sobre alterações de senha.



Role

Define um perfil de acesso ao agrupar permissões que caracterizam responsabilidades dentro da aplicação.



Permission

Representa uma autorização específica que define ações permitidas dentro da aplicação.

Modelo: Objetos de Valor

Id

PersonName

Email

StrongPassword

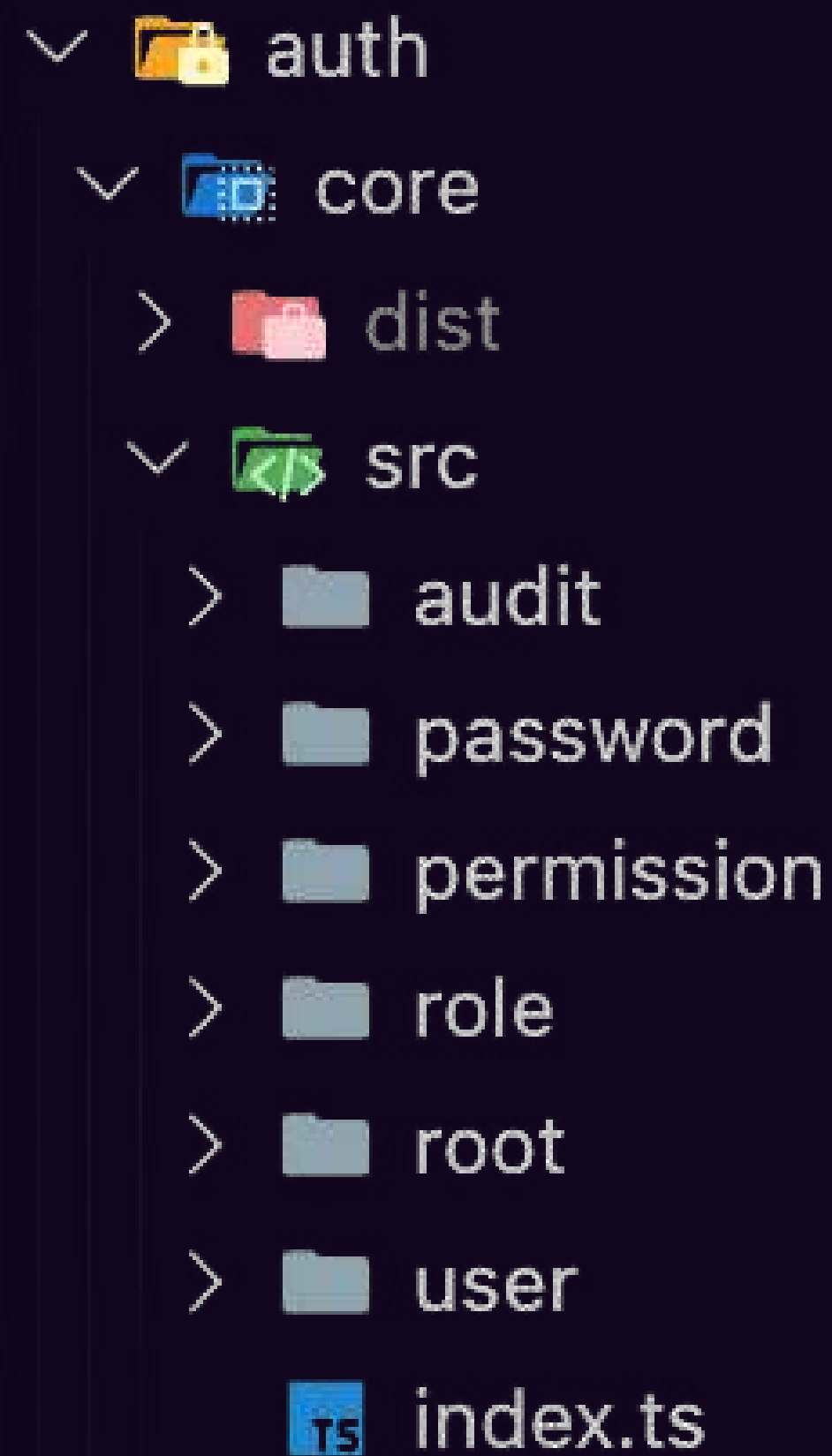
HashPassword

DotSeparatedName

Name

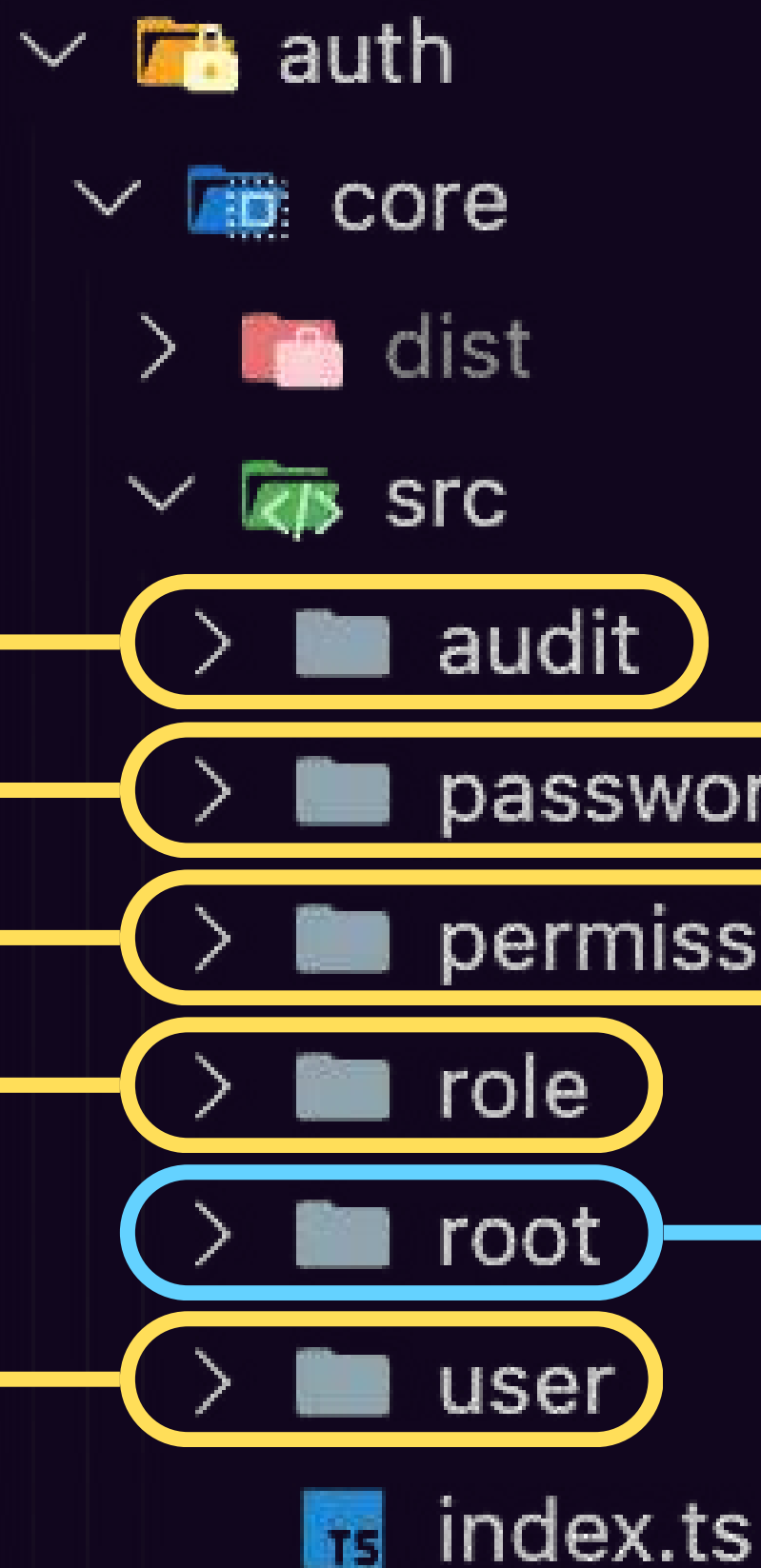
ShortDescription

Organização por Agregados



Organização por Agregados

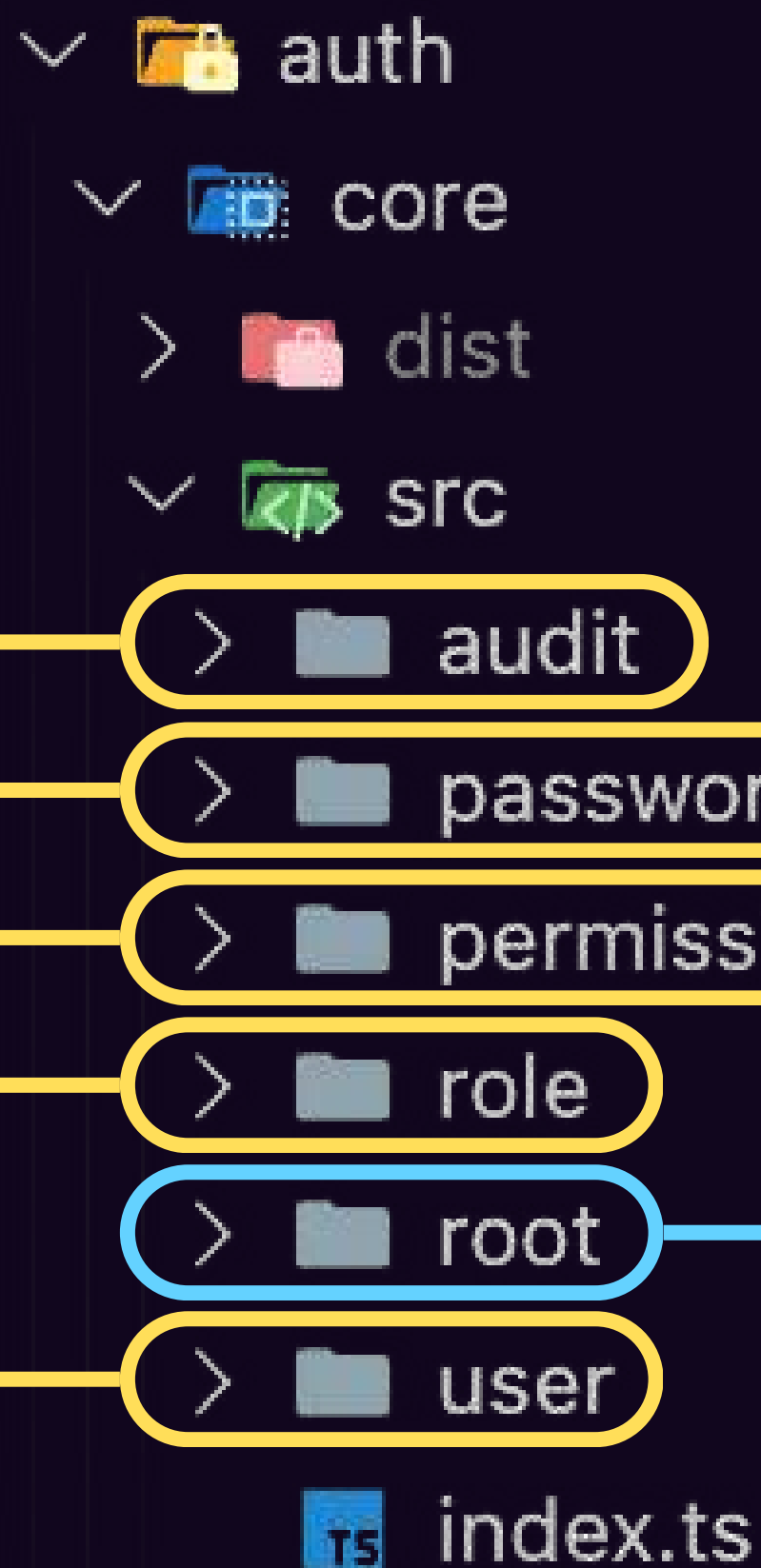
AGREGADOS



MULTI
AGREGADOS

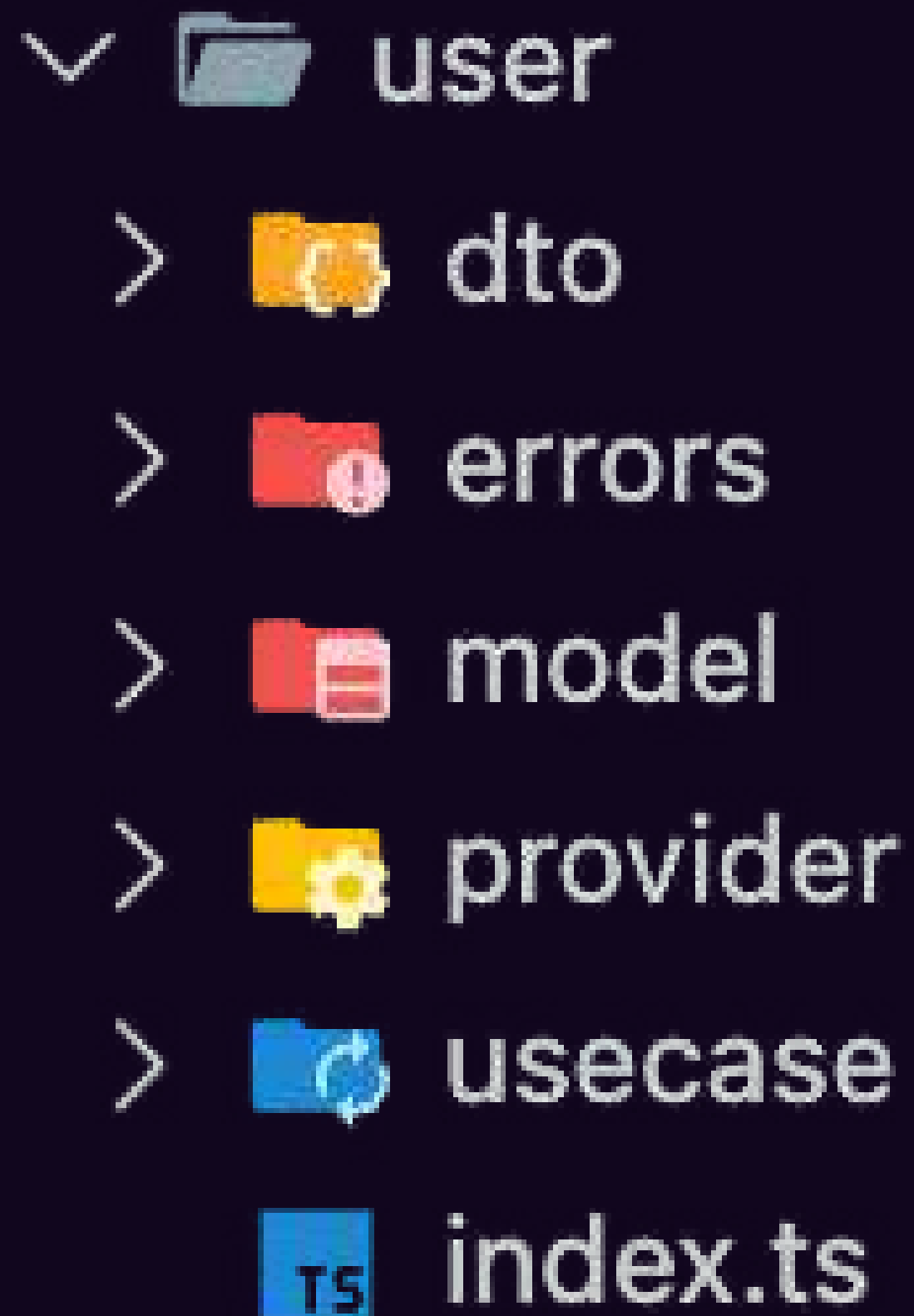
Organização por Agregados

AGREGADOS

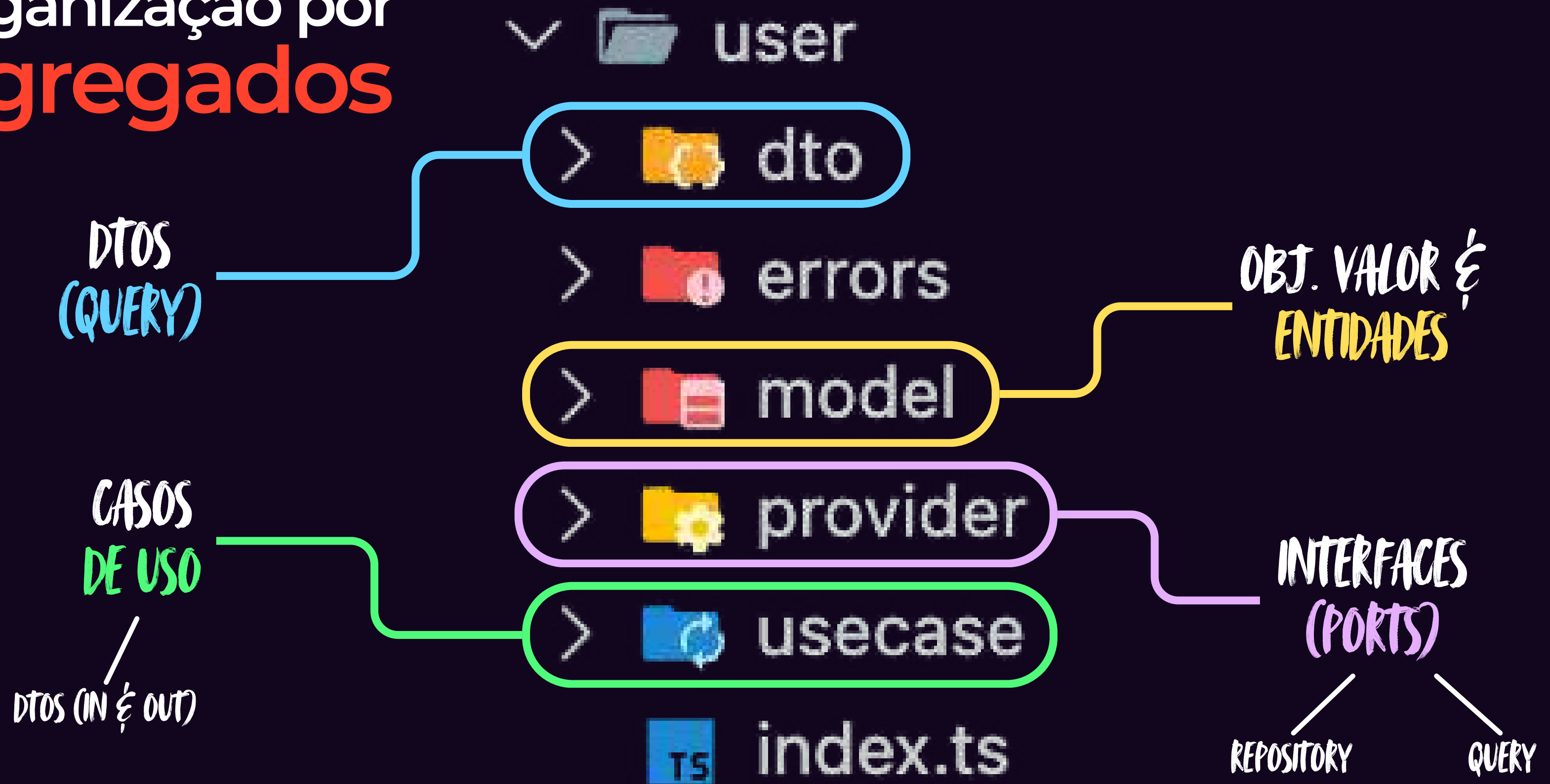


MULTI
AGREGADOS

Organização por Agregados



Organização por Agregados



VAMOS PARA O CÓDIGO...